Quantile-Based Estimation and Forecasting: A Monte Carlo and Empirical Study of Linear Models under Distributional Uncertainty

June 30, 2025

Abstract

This report investigates robust estimation techniques within the simple linear regression framework, with a specific focus on quantile-based methods as alternatives to ***Ordinary Least Squares (OLS)***. The study is bifurcated into a theoretical component and an empirical application. The theoretical section employs Monte Carlo simulations to evaluate the performance of several quantile-based slope estimators, including linear combinations with distinct weighting schemes and the ***Composite Quantile Regression (CQR)*** estimator. These estimators are tested under various error distributions (Normal, Laplace, Cauchy, Skew-Normal, and t-distributions) with accuracy assessed by the ***mean squared error (MSE)***. The empirical part assesses the out-of-sample forecasting performance of these estimators using monthly time series data on the growth rate of seasonally adjusted total CO_2 emissions. This analysis incorporates macroeconomic predictors, with dimensionality reduction achieved through ***Principal Component Analysis (PCA)***. Model performance is evaluated using a rolling window approach, with forecast accuracy measured by the ***root mean squared error (RMSE)*** and the Diebold-Mariano test for pairwise comparison. Through both simulation and practical application, this report aims to underscore the robustness and effectiveness of quantile-based estimators, especially in the presence of non-standard error structures.

Contents

0.1 Introduction

This report investigates robust estimation techniques within the simple linear regression framework, focusing on quantile-based methods. The coursework is divided into two parts: a theoretical component (Part A), which evaluates alternative slope estimators through Monte Carlo simulations, and an empirical application (Part B), which assesses the forecasting performance of these estimators using real-world data on CO_2 emissions.

In traditional linear regression analysis, the **ordinary least squares (OLS)** method is commonly used to estimate the slope coefficient. However, OLS is sensitive to violations of classical assumptions, such as normality and homoscedasticity of the error terms. In response to these limitations, **quantile regression** provides a more flexible and robust alternative, allowing for the estimation of conditional quantiles of the response variable. This makes quantile regression particularly useful in cases where the distribution of errors is skewed, heavy-tailed, or heteroscedastic.

Part A of this study compares several quantile-based slope estimators. These include linear combinations of quantile regression estimates with two distinct weighting schemes, as well as the **Composite Quantile Regression (CQR)** estimator, implemented using the *cqr.fit* function from the *cqrReg* package. The performance of each estimator is evaluated under different sample sizes and various error distributions, including Normal, Laplace, Cauchy, Skew-Normal, and t-distributions. Accuracy is measured using the **mean squared error** (**MSE**) across repeated Monte Carlo simulations.

In Part B, the focus shifts to an empirical forecasting exercise. Using monthly time series data on seasonally adjusted total CO_2 emissions, the study examines the out-of-sample forecasting accuracy of models estimated with the quantile-based methods from Part A. The target variable is the monthly growth rate of CO_2 emissions, modeled as the log-difference of the emission index. Different sets of macroeconomic and financial predictors are considered, and dimensionality is reduced using **Principal Component Analysis (PCA)**. The models are evaluated using a **rolling window approach**, with forecast accuracy assessed via the root mean squared error (RMSE) and pairwise comparison using the Diebold-Mariano test.

Through both theoretical simulations and practical forecasting applications, this report aims to highlight the robustness and effectiveness of quantile-based estimators in linear regression, particularly in the presence of non-standard error structures.

Chapter 1

Simple Linear Regression Model

The simple linear regression model serves as a foundational framework in econometrics and statistical analysis to investigate the relationship between a dependent variable and a single explanatory variable. It is formally expressed as:

$$y_i = \alpha + \beta x_i + u_i, \quad i = 1, 2, \ldots, n \tag{1.1}$$

where :

- y_i denotes the outcome variable,
- x_i is the independent variable
- α and β are unknown parameters to be estimated,
- and u_i represents the random error term.

The parameter β captures the marginal effect of a one-unit change in the explanatory variable x on the expected value of the dependent variable y.

To estimate the relationship between two variables from a data sample, the linear regression model is typically expressed as:

$$\hat{y}_t = \hat{\alpha} + \hat{\beta} x_t + \hat{u}_t \tag{1.2}$$

In this model, $\hat{y}_t$ is the predicted value of the dependent variable y_t, $\hat{\alpha}$ and $\hat{\beta}$ are the intercept and slope coefficients, x_t is the independent variable, and $\hat{u}_t$ is the residual term. The $\hat{\alpha}$ and $\hat{\beta}$ are commonly estimated using the Ordinary Least Squares (OLS) method.

The primary goal of OLS is to find the **line of best fit** by minimizing the residual sum of squares (RSS), which is the sum of the squared differences between the observed values and the predicted values:

$$\text{RSS} = \sum_t \hat{u}_t^2 = \sum_t (y_t - \hat{y}_t)^2 \tag{1.3}$$

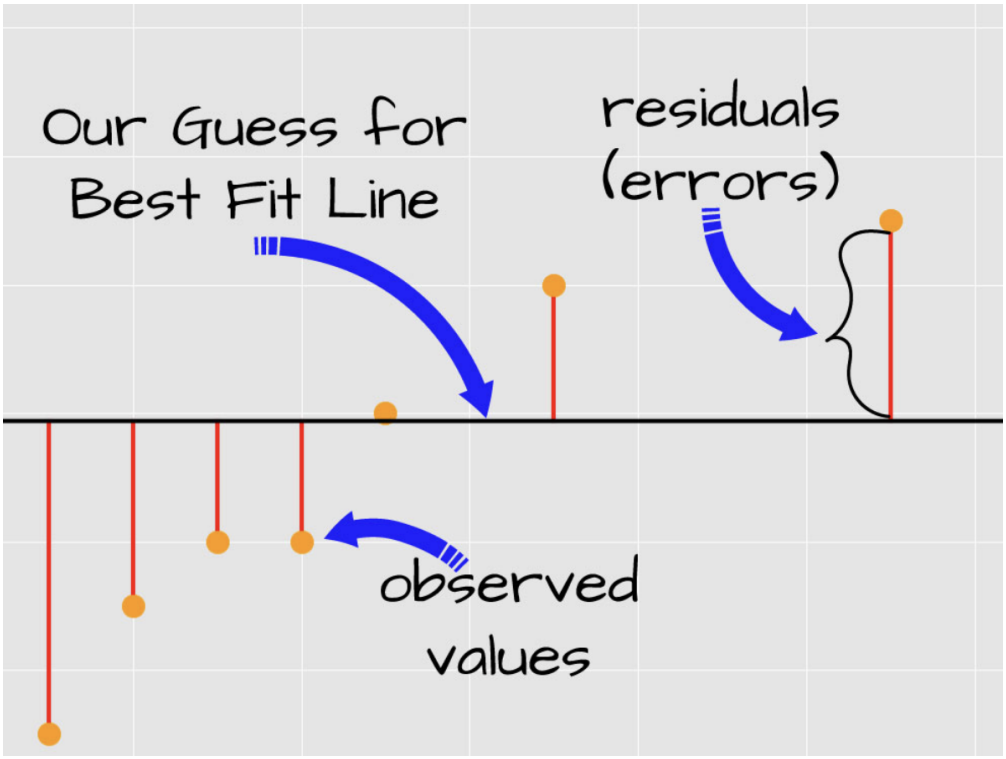

Figure 1.1: Line of Best Fit.

The OLS estimators for the slope and intercept are given by closed-form formulas. The slope coefficient $\hat{\beta}$ is calculated as:

$$\hat{\beta} = \frac{\text{Cov}(x, y)}{\text{Var}(x)} = \frac{\sum_t (x_t - \bar{x})(y_t - \bar{y})}{\sum_t (x_t - \bar{x})^2} \tag{1.4}$$

Once $\hat{\beta}$ is determined, the intercept $\hat{\alpha}$ is estimated using the sample means of x and y:

$$\hat{\alpha} = \bar{y} - (\hat{\beta} * \bar{x}) \tag{1.5}$$

Under the classical Gauss – Markov assumption, linearity of the model, random sampling of data, no perfect multicollinearity, zero conditional mean of the errors, and constant variance of the errors (homoscedasticity), the OLS estimators are **Best Linear Unbiased Estimators (BLUE)**. This means that among all linear and unbiased estimators, OLS yields the lowest variance.
If the error terms are also assumed to be normally distributed, then the OLS framework further allows for valid inference procedures, such as hypothesis testing and the construction of confidence intervals for the estimated parameters. This makes OLS a powerful and widely used method in statistical modeling and econometrics.
Despite its wide applicability and simplicity, the OLS estimator can perform poorly when the underlying assumptions are violated. In particular, the presence of outliers, heteroscedasticity, or non-normal error distributions can lead to biased or inefficient estimates. In such settings, more robust estimation techniques are needed.
Quantile regression provides one such alternative. Rather than modeling the conditional mean of the dependent variable, quantile regression estimates conditional quantiles, offering a more complete picture of the relationship between variables. It is particularly effective in capturing distributional asymmetries and is more robust to outliers and heteroscedastic errors. This makes it a valuable tool in empirical settings where classical assumptions may not hold.

Chapter 2

The Quantile Regression

The Quantile regression is a statistical method(introduced by **Koenker and Bassett in 1978**) that extends the classical linear regression framework by allowing for the estimation of conditional quantiles of the response variable, rather than merely its conditional mean.

QR offers a more complete view of the relationship between dependent variable(or explanatory variable) and the independent variable, particularly in the presence of:

- **heteroskedasticity**

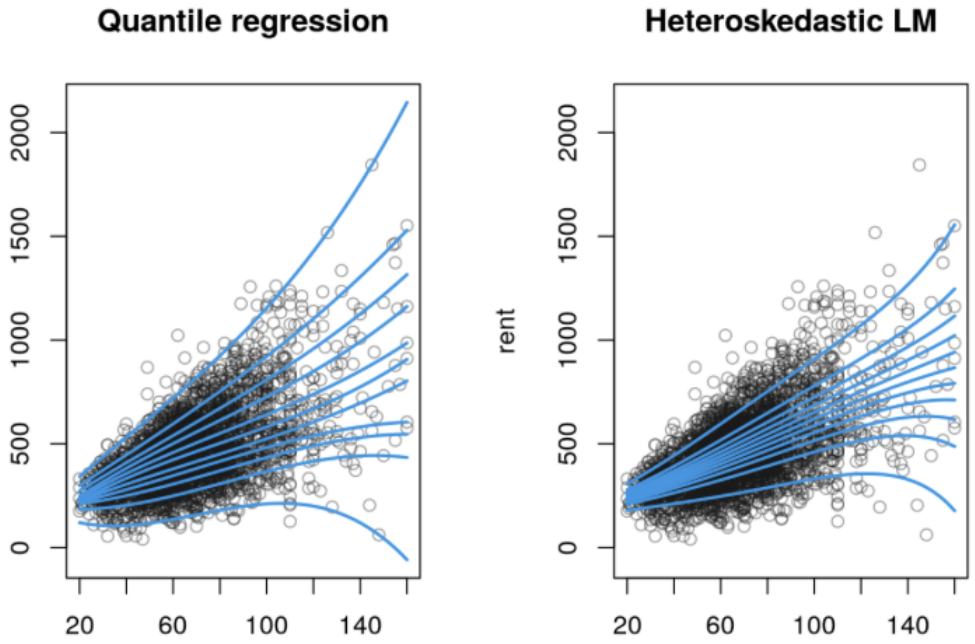

Figure 2.1: QR vs OLS.

- **non-normal error distributions**,
- or **outliers**.

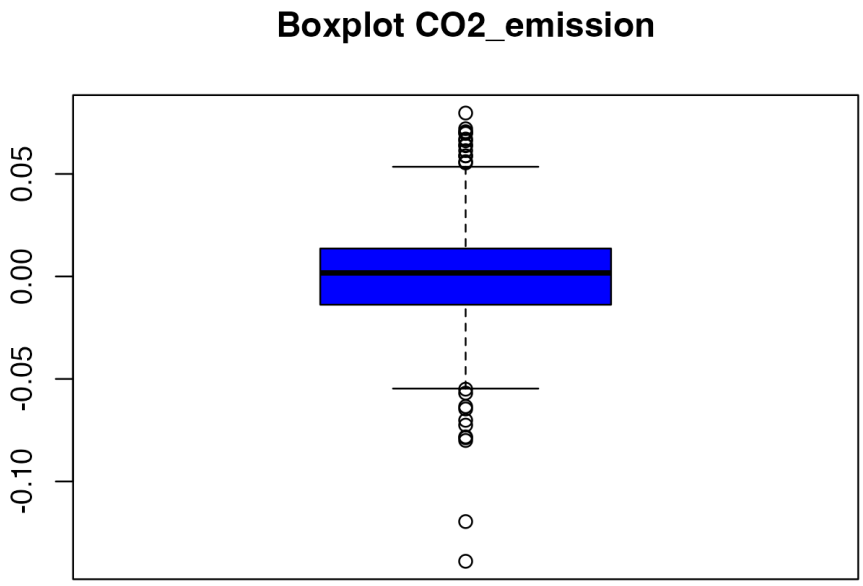

Figure 2.2: outliers into the dataset.

In the **Standard Ordinary Least Squares (OLS) approach**, the coefficients are estimated by minimizing the sum of squared residuals.

Figure 2.3: RSS Graphical Visualization.

This leads to parameter estimates that describe how the conditional mean of the dependent variable changes with respect to changes in the independent variable(s).

In contrast, quantile regression estimates the conditional quantile function, providing insights into the

impact of covariates at different points in the distribution of the outcome variable.
Formally, for a given quantile level $\tau \in (0, 1)$, the quantile regression estimator solves the following optimization problem:

$$\hat{\beta}_\tau = \arg\min_{\beta} \left(\sum_{i:y_i > \beta x_i} \tau |y_i - \beta x_i| + \sum_{i:y_i < \beta x_i} (1-\tau)|y_i - \beta x_i| \right) \tag{2.1}$$

This formula systematically penalizes estimation errors based on their sign and the chosen quantile,τ. Errors resulting from **underestimation** ($y_t > x_t^T \beta_\tau$), are weighted by τ, while errors from **overestimation** ($y_t < x_t^T \beta_\tau$) are weighted by $(1 - \tau)$.

More specifically τ is precisely the level of the quantile:

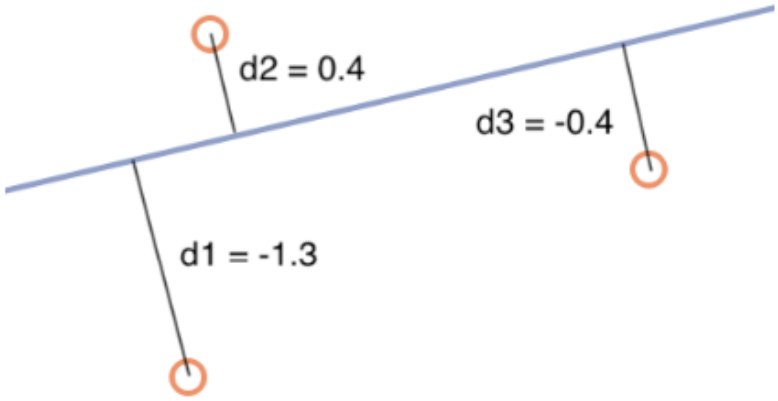

Figure 2.4: Each orange circle represents an observation while the blue line represents the quantile regression line. The black lines illustrate the distance between the regression line and each observation, which are labelled d1, d2 and d3.

Assuming that it is equal to **0.9**, you can calculate the quadratic regression loss for the data in the image above, as follows:

$$\tau(d_2) + (1-\tau)(|d_1 + d_3|) = 0.9 \cdot 0.4 + 0.1 \cdot |-1.3 + (-0.4)| = 0.53$$

"Optimizing this loss function results in an adapted line such that approximately a portion τ of the data is below it, and the rest (1-τ) is above."

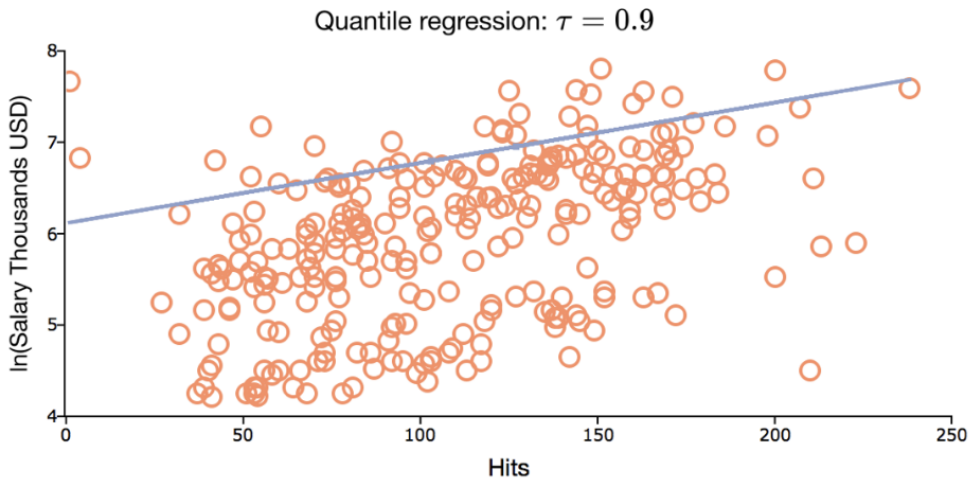

Figure 2.5: 90.11% of the observations are below the quantile regression line which was estimated with τ set to 0.9.

The **asymmetrical penalization** is the key mechanism that allows the model to target specific quantiles of the conditional distribution.

The loss function can be expressed more compactly by minimizing a **"Check Function"**, instead the minimization problem is written as:

$$\min_{\beta} \sum_{i=1}^{T} \rho_\tau \left(y_t - x_t^\top \beta_\tau \right) \tag{2.2}$$

Here, the check function $\rho_\tau(u)$ is defined as :

$$\rho_\tau(u) = \begin{cases} \tau u & \text{se } u \geq 0 \\ (1-\tau) & \text{se } u < 0 \end{cases} \tag{2.3}$$

This formulation is **central** to quantile regression. Just as OLS relies on the fundamental assumption that the **conditional expectation** of the error term is zero $E[u_t|x_t] = 0$, quantile regression operates on similar assumption regarding the error term's quantiles $Q_\tau(u_t|x_t) = 0$.
This framework allows for the estimation of the $\tau - th$ conditional quantile of a dependent variable, y, given a set of independent variables, x. The estimation model is expressed as $\hat{Q}_\tau(y|x) = x_t^T \hat{\beta}_\tau$. This equation signifies that the $\tau - th$ quantile of the dependent variable is a linear combination of the regression coefficient, β_τ. By selecting different values for τ between 0 and 1, a researcher can move beyond the central tendency and investigate the impact of covariances on different parts of the dependent variable's distribution, such as the lower or upper tails.
A noteworthy special instance of quantile regression is **median regression**, which is defined as a particular case of the broader QR framework. This occurs when the quantile, τ, is set to 0.5. This specific application is also known as the **Least Absolute Deviation (LAD)** method. Therefore, LAD is not a separate technique but is nested within quantile regression as the approach for estimating the conditional median.

2.0.1 When to use QR ?

Quantile regression can be more robust and flexible than traditional regression methods, especially when focused on extreme values or tail events. Quantile regression has applications in various fields, including economics, finance, environmental sciences, health research, and social sciences. It allows researchers to analyze the relationship between variables across different quantiles, providing a more nuanced understanding of the data and capturing the conditional distributional effects.

- **In finance**, it can be used to estimate the Value at Risk (VaR) of a portfolio of assets given a set of predictors.
- **In healthcare**, it can be used to estimate the probability of a patient developing a certain condition, given their medical history and other relevant factors.
- **In marketing**, it can be used to estimate the probability of a customer purchasing a product, given their demographic profile and past purchasing behavior.

2.0.2 SLM vs QR

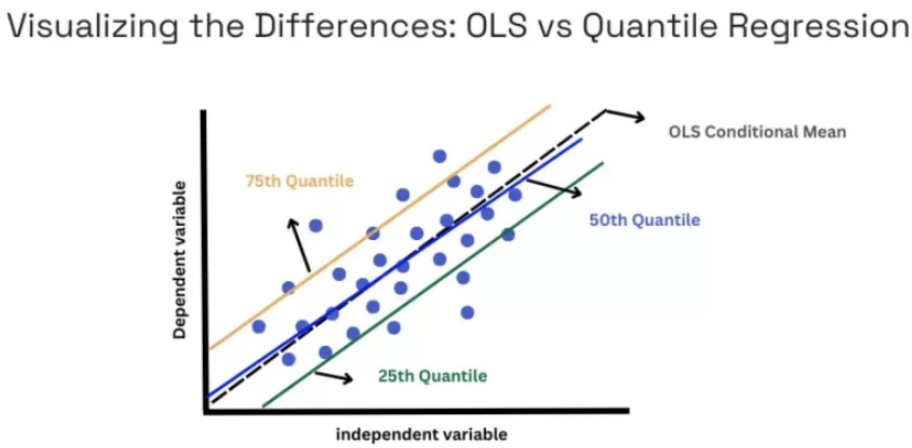

Figure 2.6: SML vs QR.

Simple Linear Regression (SLR) and ***Quantile Regression (QR)*** are both statistical techniques used to model relationships between a dependent variable and one or more independent variables, but they differ significantly in their assumptions, estimation strategies, and interpretive focus.

Simple Linear Regression estimates the conditional mean of the dependent variable given the values of the predictors. It is based on minimizing the sum of squared residuals (i.e., the L2 norm), which makes it highly sensitive to outliers and influential points.
The model assumes that the residuals are normally distributed, homoskedastic (i.e., constant variance), and independent. These assumptions are crucial for valid inference using traditional **t-tests** and **F-tests**.
Because of its focus on the mean, SLR is best suited for situations where the interest lies in understanding the average trend or behavior of the response variable.

In contrast, Quantile Regression allows for a more comprehensive analysis by estimating conditional quantiles (e.g., *the median,* 25^{th} *percentile,* 90^{th} *percentile*) of the dependent variable.
This approach uses an asymmetric absolute loss function, often referred to as the check function, which minimizes the weighted sum of absolute residuals. As a result, quantile regression is more robust to outliers and heteroskedasticity, making it suitable for analyzing data with non-constant variance or skewed distributions. Rather than providing a single line representing the average response, quantile regression can produce a family of curves that describe how the relationship between variables changes across the distribution of the outcome.

The interpretation of the coefficients in quantile regression is also different. While SLR explains how predictors affect the average outcome, QR reveals how predictors impact different points of the distribution. For instance, when studying income as a function of education, simple linear regression might show that each additional year of education increases income by a fixed amount on average. However, quantile regression

could show that this effect is stronger at higher income levels than at lower ones, capturing the heterogeneity in the returns to education across the income distribution.

In terms of estimation, SLR uses ordinary least squares, a closed-form solution that is computationally efficient.
Quantile regression, on the other hand, relies on linear programming or convex optimization techniques. While inference in quantile regression can be more complex due to the lack of strong distributional assumptions, methods such as Koenker-type robust standard errors or bootstrapping are commonly used to obtain valid confidence intervals and hypothesis tests.
In summary, simple linear regression is ideal when the goal is to understand average effects under **standard assumptions**, while quantile regression provides a more **flexible and robust** framework for exploring how covariates influence the entire distribution of the outcome variable. It is particularly useful in contexts involving skewed data, outliers, or varying effects across subpopulations.

2.1 Calculations for the analysis

2.1.1 Quantile regression setup

The analysis is based on a quantile regression approach using a dataset with 625 observations. For reasons of robustness and stability, especially with a dataset of this size, a limited number of quantiles, $K = 3$, was chosen. We adopt this strategy for explanatory and practical reasons, even if we obtain a more precise analysis by studying $K \geq 9$ The specific quantiles (τ) are calculated using the formula $\tau_k = \frac{k}{K+1}$, resulting

k	τ_k
1	0.25
2	0.5 (Median)
3	0.75

Table 2.1: Value of τ_k respect k

Instead, graphically specking we obtain :

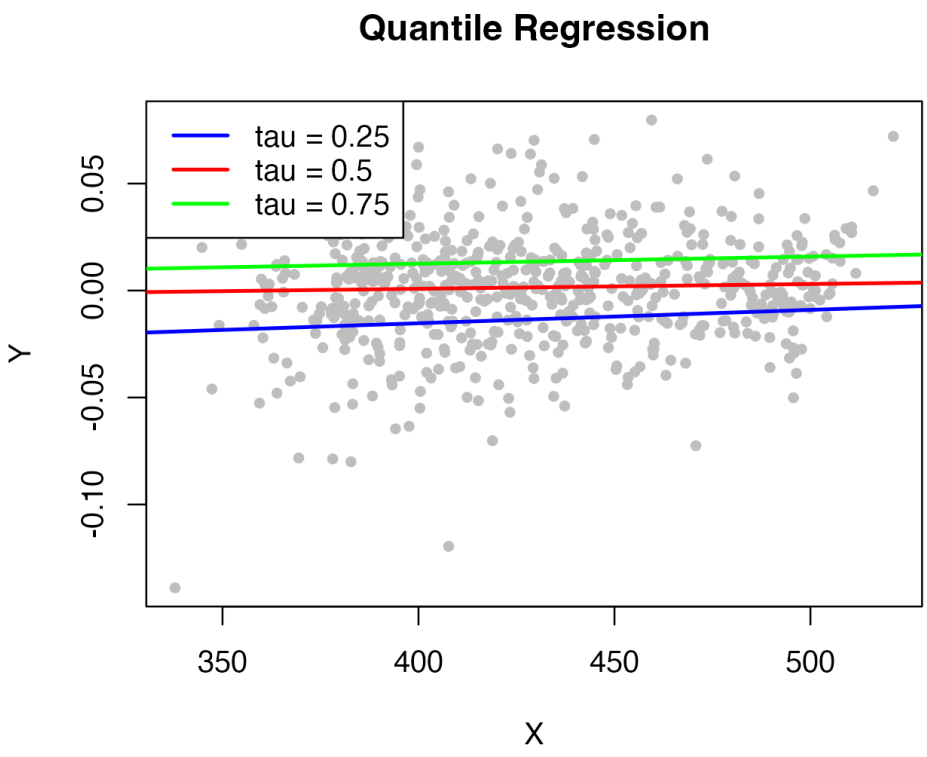

Figure 2.7: QR with different level of τ.

The ultimate goal is to create a single combined estimate for the regression coefficients (β) by calculating a weighted average of the coefficients obtained from the quantile regressions.

$$\hat{\beta} = \sum \omega_k \hat{\beta}_{\tau_k} \tag{2.4}$$

Where:

- τ_k is the value that we have already found.

2.1.2 Weighting criteria and Coefficient estimation

In order to calculate the sum of weights ($\sum \omega_k$),two different criteria were used,for combining the quantile regression coefficients.

Now to better show the variation of the regression coefficients between quantiles, we present the following graph.

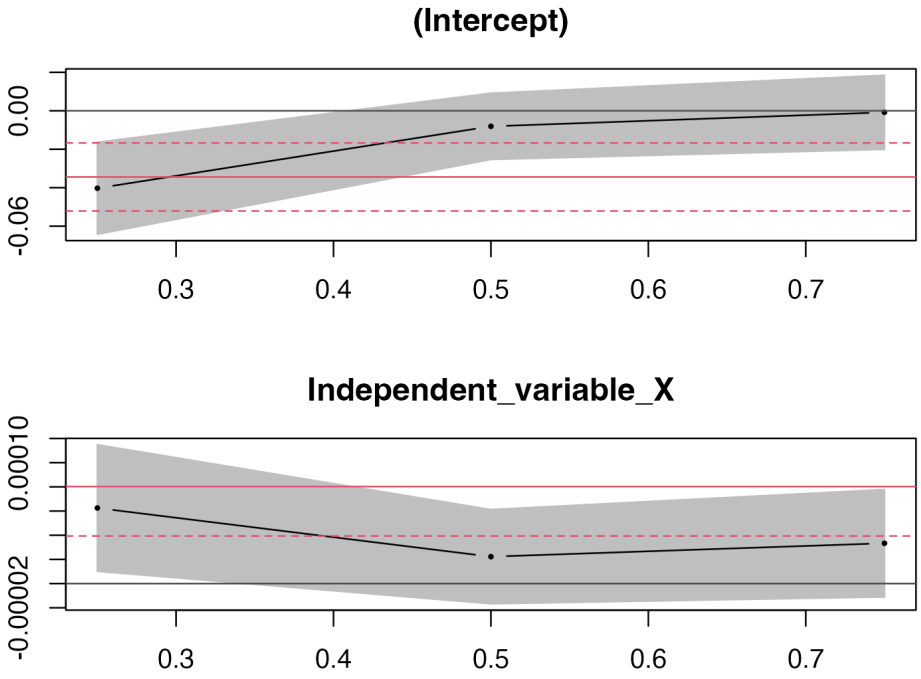

Figure 2.8: Intercept and Ind. Var of QR.

The latter illustrates the estimated intersections and gradients for the three selected quantiles ($\tau_k = 0.25$, 0.5, 0.75), based on the quantum regression applied to the CO data set.

Criterion 1: Symmetrical Weights

This method calculates the weights using the formula: $\omega_k \propto 1 - |2_{\tau_k} - 1|$, which is symmetric around the mean.
Applying it, we obtain unnormalized weights:

k	τ_k
1	$\omega_1 \propto 1 - \lvert(2 \cdot 0.25) - 1\rvert = 0.5$
2	$\omega_2 \propto 1 - \lvert(2 \cdot 0.5) - 1\rvert = 1$
3	$\omega_3 \propto 1 - \lvert(2 \cdot 0.75) - 1\rvert = 0.5$

Table 2.2: ω_k not normalized

In order to sum these values, first of all we have to normalize them, and in order to do that, we have to satisfy two conditions:

1.
$$\omega_k \geq 0 \tag{2.5}$$

2.
$$\sum \omega_k = 1 \tag{2.6}$$

After normalization, with respect to the two conditions, we found that:

- $\tau = [0.25, 0.5, 0.75]$
- $\omega_k = [0.25, 0.5, 0.25]$

Now the aim is to build a **single combined estimation of** $\hat{\beta}$, us the weighted average of the coefficients obtained from the QR.

$$\hat{\beta} = \sum \omega_k \hat{\beta}_{\tau_k} = (0.25 * \hat{\beta}_{0.25}) + (0.5 * \hat{\beta}_{0.5}) + (0.25 * \hat{\beta}_{0.75}) \tag{2.7}$$

In this formula, every $\hat{\beta}_{\tau_k}$ is a **vector** (**intercept and slope**).Using these weights and the quantile regression results from the CO2-SA.csv dataset, the combined coefficients are calculated:

$$Intercept(\alpha) = [0.25 * (-0.4028)] + [0.5 * (-0.00805)] + [0.25 * (-0.0008)] = -0.0143 \tag{2.8}$$

$$Slope(\beta) = (0.25 * 0.00006) + (0.5 * 0.00002) + (0.25 * 0.00003) = 0.0000325 \tag{2.9}$$

This slope suggests that for every one-unit increase in the independent variable, the dependent variable increases by $3.25 * 10^{-5}$.

Criterion 2: Koenker-Type Weights

[1] The second method uses a formula based on the normal distribution:

$$w_k \propto \frac{\phi\left(\Phi^{-1}(\tau_k)\right)}{\sqrt{\tau_k(1-\tau_k)}} \tag{2.10}$$

Where ϕ denotes the standard normal density function, Φ^{-1} is the inverse function of the CDF and it reflects the quantile. In order to obtain the values of the numerator of this equation, we have to use first the function $qnorm(\tau)$, and then the function *dnorm(qnorm value)*:

τ	Quantile Value	Density
0.25	-0.6744898	0.3177766
0.5	0	0.3989423
0.75	0.6744898	0.3177766

Table 2.3: Quantile and Density Values

Looking to this table, it can be confirmed that there is more density in the center of the distribution.

The tails (0.25 and 0.75) are less dense.

Using these values, the final weighted average coefficients under the second criterion were calculated as follows:

1. **Combined Intercept(α)** $= -0.03660301$
2. **Combined Slope (β)** $= 0.00008810474$

These values have been computed using the weights:

$$\omega_k = [0.3239157,\ 0.3521687,\ 0.3239157]$$

and the quantile regression coefficients obtained from the actual dataset `CO2_SA.csv`.

[1] Koenker-type weights refer to a method of heteroskedasticity-consistent weighting used in quantil regression or related robust regression contexts, particularly when dealing with non-constant variance (heteroskedasticity) in the error terms

2.1.3 Composite Quantile Regression (CQR)

As an alternative, a Composite Quantile Regression (CQR) was performed. CQR is noted as being more robust to outliers and more efficient in the presence of heteroscedasticity compared to models like Ordinary Least Squares (OLS). Due to issues with one function *cqr.fit()*, an approximation *rq()* was used, yielding the following coefficients:

- CQR Intercept (α)= -0.01639037
- Combined Slope (β)= 0.0003937057

Note: The approximation was calculated by estimating the quantile regression in $\tau = 0.25,\ 0.5,\ 0.75$ using the `rq()` function from the `quantreg` package in `R`, and then averaging the estimated coefficients:

$$\hat{\beta}_{\text{CQR}}^{\text{approx}} = \frac{1}{3}\sum_{k=1}^{3}\hat{\beta}_{\tau_k} \tag{2.11}$$

2.1.4 Comparison among the three criteria

Method	Intercept	Slope	Comments
CQR (mean)	-0.01639	3.94e-05	Simple unweighted average
Criterion 1	-0.01431	3.51e-05	Symmetric weights, intuitive
Criterion 2	**-0.03660**	**8.81e-05**	More rigorous, uses theoretical weights

Table 2.4: Comparison of C¢riteria

2.1.5 Simulation

To compare the precision of the three estimators for the slope (β), using :

1. ***Weighting Criterion 1***,
2. ***Weighting Criterion 2***,
3. and the ***CQR approximation***,

The Monte Carlo simulation with 1,000 trials was conducted. The simulation generated data based on the model $Y_t = \alpha + \beta x_t + u_t$, with errors (u_t) drawn from a standard normal distribution. The quality of the estimators was measured by the:

Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{M}\sum_{i=1}^{M}\left(\hat{\beta}_i - \beta\right)^2 \tag{2.12}$$

The MSE can be decomposed into two components:

$$\text{MSE} = \left(\text{Bias}(\hat{\beta})\right)^2 + \text{Var}(\hat{\beta}) \tag{2.13}$$

- **Bias** represents the systematic error, i.e., how far the average estimate is from the true value.
- **Variance** measures the dispersion of the estimates across repeated samples, indicating stability.

This decomposition is essential because it allows us to understand whether an estimator performs poorly due to a systematic deviation (bias) or because it fluctuates too much (variance).

The simulation concluded that under a normal error distribution, the differences between the three estimators are minimal. However, an analysis of the empirical variances indicated a clear ranking of the estimators' efficiency:

$$\boxed{Variance(CQRmean) < Variance(Criterion2) < Variance(Criterion1)}$$

It can be stated that using CQR is a better solution with respect to the other two solutions, because from the results it is more **EFFICIENT(low MSE)** and more **STABLE (low VARIANCE)**.

2.2 Monte Carlo Simulation

A Monte Carlo simulation is a way to model the probability of different outcomes in a process that cannot easily be predicted due to the intervention of random variables. It is a technique used to understand the impact of risk and uncertainty. It is also referred to as a multiple probability simulation. It help forecast a range of possible results by simulating the impact of randomness in a system. It requires assigning multiple values to an uncertain variable to achieve multiple results and then averaging the results to obtain an estimate.

Monte Carlo simulations have a vast array of applications in fields that are plagued by random variables, notably business and investing. They are used to estimate the probability of cost overruns in large projects and the probability that an asset price will move in a certain way. Today, Monte Carlo simulations are increasingly being used in conjunction with new artificial intelligence (AI) models.

2.2.1 Normal Distribution Function

This simulation evaluates and compares the performance of three different estimators derived from quantile regression (QR).

Objective :

- Analyze the mean squared error (MSE) of each estimator under a controlled simulation setup and determine which method yields the most accurate estimate of the slope parameter in a simple linear regression model.

Starting by loading the *quantregpackage* , which provides the necessary tools for performing quantile regression in R. The simulation involves 1000 replications (N=1000), where in each replication we generate a dataset with 100 observations (n=100). A fixed random seed *(set.seed(123))* is used to ensure the results are reproducible. The true underlying model for the data is a simple linear regression:

$$y = \alpha + \beta x + u \tag{2.14}$$

where:

- $\alpha = 0$ The intercept
- $\beta = 1$ The slope
- u is drawn from a normal distribution The error term

A fixed vector $x = 1$ to 100 is used as an independent variable in all simulations. The dependent variable y is then generated based on the true model, introducing randomness through normally distributed error terms.

In each simulation, quantile regression is performed at three levels: $\tau = 0.25, 0.5, 0.75$ using the $rq()$ function. The regression coefficients for each quantile are collected in a matrix. These estimates are then used to compute three different estimators for the slope coefficient based on distinct aggregation strategies. The key objective is to robustly estimate the slope coefficient β using various weighting schemes applied to these quantile-specific estimates.

Method n.1: **Criterion 1**, uses symmetric weights to place more emphasis on the median quantile. The weights are calculated using the formula:

$$\omega_k \propto 1 - |2\tau_k - 1| \tag{2.15}$$

and then normalized so they sum to 1. For the selected quantiles, this gives the normalized weights (0.25,0.5,0.25). These are applied to the estimated slope coefficients to compute an aggregated estimate for each replication:

$$\hat{\beta}_{\text{crit1}} = \sum_{k=1}^{3} \omega_k \, \hat{\beta}(\tau_k) \tag{2.16}$$

This is implemented via:

```
  beta_crit1 <- colSums(beta_matrix * weights_crit1)
beta1_crit1[i] <- beta_crit1[2]
```

Method n.2: **Criterion 2**, aims for greater statistical efficiency. Calculate weights based on the standard normal distribution using the formula:

$$w_k \propto \frac{\phi\left(\Phi^{-1}(\tau_k)\right)}{\sqrt{\tau_k(1-\tau_k)}} \tag{2.17}$$

$$\hat{\beta}_{\text{crit2}} = \sum_{k=1}^{3} w_k \, \hat{\beta}(\tau_k) \tag{2.18}$$

Optimized weights (0.3240,0.3522,0.3239), derived from theoretical efficiency considerations. It is calculated as follows:

```
beta_crit2 <- colSums(beta_matrix * weights_crit2)
beta1_crit2[i] <- beta_crit2[2]
```

Method n.3: **Composite Quantile Regression (CQR)**. Rather than applying fixed weights, this method simply averages the regression coefficients across the three quantiles, offering a straightforward aggregation method that leverages all available quantile information equally:

```
beta_cqr <- colMeans(beta_matrix)
beta1_cqr[i] <- beta_cqr[2]
```

All three estimators (***Criterion 1***, ***Criterion 2***, and ***CQR***) are stored in replications, allowing comparison of their performance in terms of Mean Squared Error (MSE) or similar metrics. This setup allows for the evaluation of the efficiency and robustness of different aggregation strategies in quantile regression under normal error assumptions.

To assess the accuracy of each method, the mean squared error (MSE) is computed for each estimator. The MSE measures the average squared deviation of the estimated slope from the true value of $\beta = 1$. It is calculated using the following formula:

$$\text{MSE} = \frac{1}{M} \sum_{i=1}^{M} \left(\hat{\beta}_i - \beta\right)^2 \tag{2.19}$$

where M is the number of simulations, $\hat{\beta}_i$ the estimated parameter in the i^{th} Monte Carlo replica and β the true value (1 in this case). A **lower MSE** indicates that the estimator is more accurate and consistent in repeated samples. The results are summarized in a comparison table showing the MSE values for ***Criterion 1***, ***Criterion 2***, and ***CQR***.

No.	Method	ME
1	Criterion 1	1.407761e-05
2	Criterion 2	1.366737e-05
3	CQR (mean)	**1.361507e-05**

Table 2.5: Mean Error Comparison

From this simulation empirical evidence is obtained about the relative performance of the three quantile-based estimators. Specifically, it is observed which method produces slope estimates that are closest to the true parameter value, on average, across the 1000 simulations. This type of analysis provides a practical understanding of estimator efficiency under finite-sample conditions, especially in situations where theoretical properties might be difficult to derive or apply.

Conlcusion

This Monte Carlo simulation demonstrates how quantile regression can be extended and optimized through composite or weighted estimators. By simulating repeated samples and evaluating MSEs using the formula above, we gain insight into which estimator performs best in practice. Such simulations are crucial in guiding applied researchers when selecting among competing estimation techniques, particularly in cases where robustness and distributional considerations play a central role.

2.2.2 Cauchy's Distribution Function

[2]

This Monte Carlo simulation extends the previous analysis by examining the performance of quantile regression-based estimators under a different error distribution, specifically the **Cauchy distribution**, which is known for its heavy tails and outliers. The simulation framework largely mirrors the earlier one that used normal errors, allowing for a direct comparison of estimator robustness across these two settings.

Before we get into the heart of the calculations made, let's introduce the Cauchy Distribution Function.

[2] A non-ideal error structure occurs when classical regression assumptions, such as normality, homoscedasticity, independence, and absence of outliers, are violated. Examples include heavy-tailed distributions (e.g., Cauchy), heteroskedasticity, autocorrelation, or the presence of influential outliers. Under such conditions, Ordinary Least Squares (OLS) estimates may become inefficient or misleading. Quantile Regression offers a robust alternative, as it does not rely on these strict assumptions and remains effective across varying distributions of the error term.

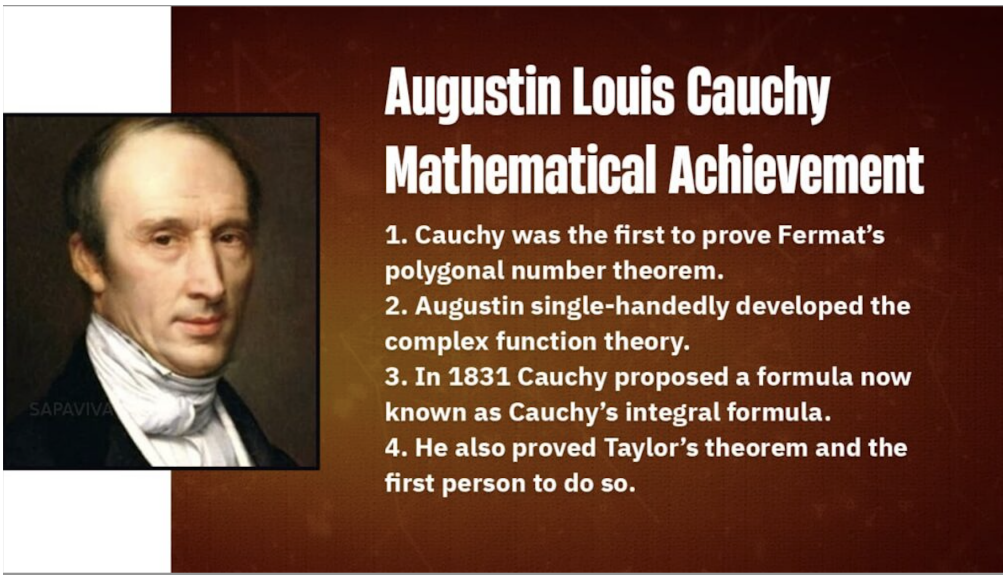

Figure 2.9: Augustin Louis Cauchy.

IN BRIEF: The Cauchy distribution function (or Lorentz distribution in the physical field) is one of the best known distributions in probabilistic statistics for its so-called "PATHOLOGICAL" properties, since both the expected value and the variance are **undefined**. This function is particularly useful in econometric analysis when one wants to evaluate the **ROBUSTNESS** of the estimators with respect to the presence of EXTREME VALUES or OUTLIERS.
Due to its structure, the Cauchy distribution challenges the classical assumptions of OLS regression, making it essential to use robust techniques such as QR.

Density Function

$$f(x; x_0, \gamma) = \frac{1}{\pi\gamma} \cdot \frac{\gamma^2}{\gamma^2 + (x - x_0)^2}$$

where:

- x_0 : location parameter
- γ : scale parameter

Back to the calculations :

Key parameters remain constant :

- The number of simulations ($N = 1000$)
- Observations per simulation ($n = 100$)
- The true linear model parameters ($\alpha = 0, \beta = 1$)
- The explanatory variable x is fixed as a sequence from 1 to 100
- The quantile regression estimations are performed at three quantiles $\tau = 0.25$, 0.5, and 0.75.

How do they differ?

The simulation differs in the generation of the error term, which now comes from a Cauchy distribution with location zero and scale one. This choice introduces extreme variability and outliers that challenge traditional estimators.

Within each repetition of the simulation, the response variable y is constructed by adding the **Cauchy-distributed errors** to the deterministic part of the model. Quantile regressions are run at the three selected quantiles, and their coefficient estimates (intercept α and slope β) are extracted. These estimates are combined using the same two weighting schemes as before, ***Criterion 1*** (weights: 0.25, 0.5, 0.25) and ***Criterion 2*** (optimized weights: 0.3240, 0.3533, 0.3239), and additionally via the composite quantile regression (***CQR***) method, which takes the simple average of the three quantile estimates.

To evaluate estimator performance, the MSE of the slope coefficient estimates is computed for each method, where

$$\text{MSE} = \frac{1}{M}\sum_{i=1}^{M}\left(\hat{\beta}_i - \beta\right)^2 \tag{2.20}$$

With $\beta = 1$ being the true slope. The MSE values under the Cauchy-distributed errors are as follows:

No.	Method	ME_cauchy
1	Criterion 1	4.182764e-05
2	Criterion 2	4.779910e-05
3	CQR (mean)	4.862267e-05

Table 2.6: Comparison of methods based on ME-cauchy.

These results demonstrate that ***Criterion 1*** yields the most accurate slope estimates under heavy-tailed error conditions, with the lowest MSE. ***Criterion 2*** and ***CQR*** follow closely but with slightly higher errors. All three methods remain relatively stable and robust in the presence of outliers, showcasing a key advantage of quantile regression techniques.

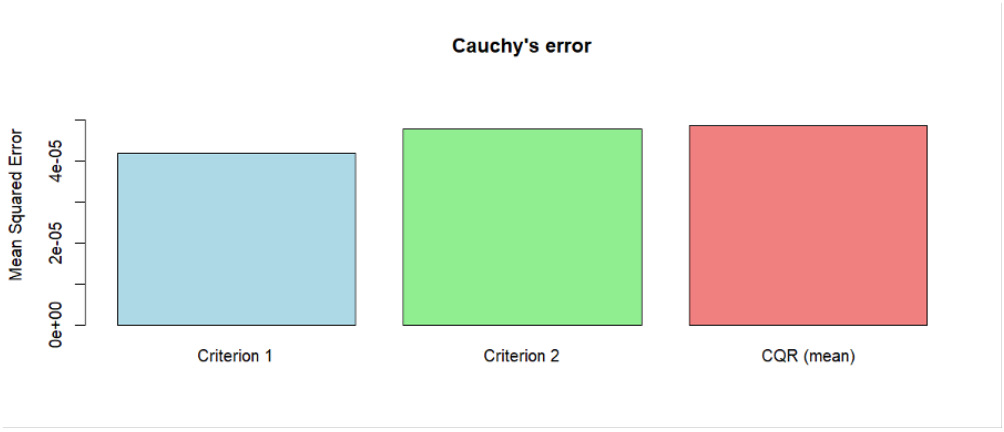

Figure 2.10: Results of the Simulation with Cauchy's error

Furthermore, a combined density plot compares the distribution of slope estimates obtained from the two error types (normal and Cauchy) for each method. This visualization reveals how the estimators behave under standard versus heavy-tailed error distributions. Generally, quantile regression methods, particularly CQR and the weighted criteria, display robustness to the extreme values induced by the Cauchy errors, maintaining tighter, more concentrated estimate distributions relative to what would be expected if ordinary least squares were applied.

Comparison to normal error

Compared to the prior simulation with normal errors, the result indicates that while all three quantile regression-based methods perform reasonably well under normal errors, their robustness becomes particularly valuable when errors have heavy tails. The weighted criteria and CQR continue to provide stable slope estimates despite the increased variability introduced by Cauchy noise, showing the practical advantage of quantile regression approaches in settings where outliers and non-normality are present.

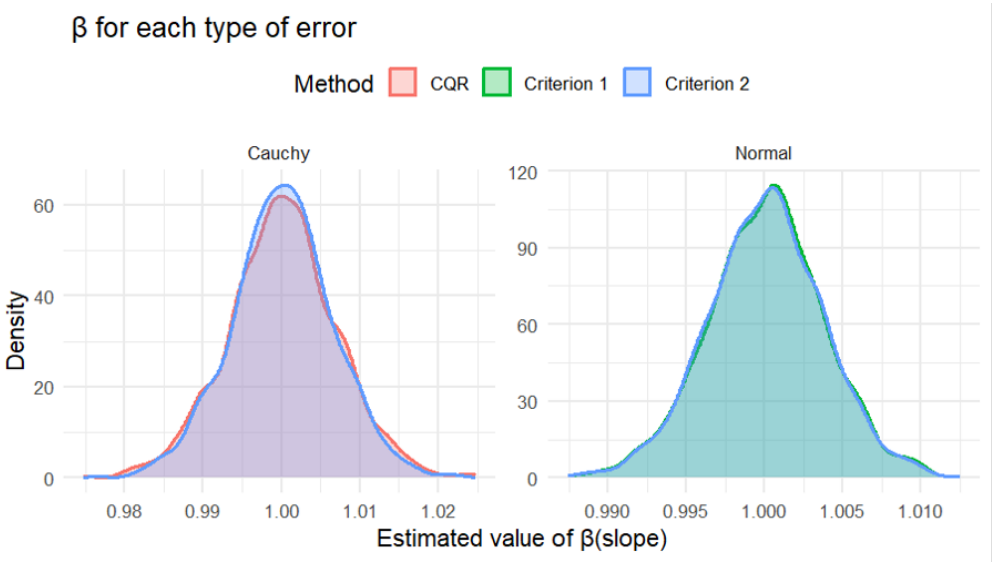

Figure 2.11: Density plot, comparing Normal and Cauchy's errors distribution

This comprehensive analysis underscores the flexibility and robustness of quantile regressin estimators. By leveraging multiple quantiles and appropriate weighting schemes, these methods can provide reliable parameter estimates in a broad range of contexts, outperforming traditional methods that may falter under non-ideal error structures.

2.2.3 t-student Distribution Function

Following the initial Monte Carlo simulations which evaluated estimator performance under both Normal (ideal) and Cauchy (extreme outlier) error distributions, this section extends the analysis to an intermediate case.
An investigation is conducted into the robustness and efficiency of the three quantile-based estimators for model errors following a t-distribution with 3 degrees of freedom(df).

This scenario is critical as it models a realistic situation where data contains significant outliers (i.e., has "heavy tails") but not to the extreme degree of the Cauchy distribution, which lacks a defined mean or variance. This allows for a more nuanced understanding of how each estimator handles data that deviates from normality.

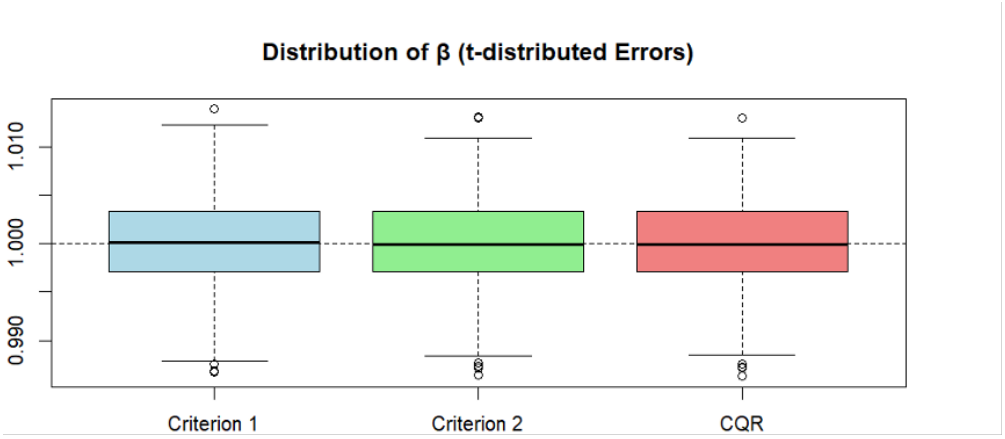

Figure 2.12: boxplot of Distribution of β

The boxplot illustrates the distribution of slope estimates β obtained from three different quantile regression methods over 1000 simulation iterations. These simulations were conducted under the assumption that the error terms follow a heavy-tailed t-distribution with 3 degrees of freedom, which introduces significant variability and outliers into the data, conditions under which traditional least squares regression may perform poorly.

The three methods compared are:

- **Criterion 1**, which uses fixed weights $=(0.25, 0.5, 0.25)$ across the 0.25, 0.5, and 0.75 quantiles respectively;
- **Criterion 2**, which uses Koenker-type weights $=(0.3239, 0.3522, 0.3239)$ designed for improved efficiency;
- **Composite Quantile Regression (CQR)**, which takes a simple average of the slope estimates across the same three quantiles.

Each box in the plot shows:

1. the median estimate
2. interquartile range (IQR)
3. the spread of the data including outliers.

A horizontal dashed line at $\beta = 1$, the true value of the slope parameter, is drawn for reference. This line allows for a *visual assessment of bias*: boxplots centered around the line suggest unbiased estimators. The relative width of the boxes indicates the variability of each method; narrower boxes suggest greater precision, while wider boxes or many outliers suggest higher estimation variance.

Overall, the boxplot provides insight into both the accuracy and robustness of each method under heavy-tailed conditions. If Criterion 2, for example, has a box that is more tightly centered around the true value with fewer extreme outliers, it suggests that Koenker-type weights lead to more stable and efficient estimates in the presence of non-Gaussian errors. This visual analysis complements the numerical evaluation of mean squared error and variance, helping to identify the most reliable estimation strategy for real-world data prone to heavy-tailed distributions or outliers.
The barplot provides a comparative visualization of the mean squared error (MSE).

The methods evaluated are:

1. **Criterion 1**, which uses weights of (0.25, 0.5, 0.25) across the 0.25, 0.5, and 0.75 quantiles;

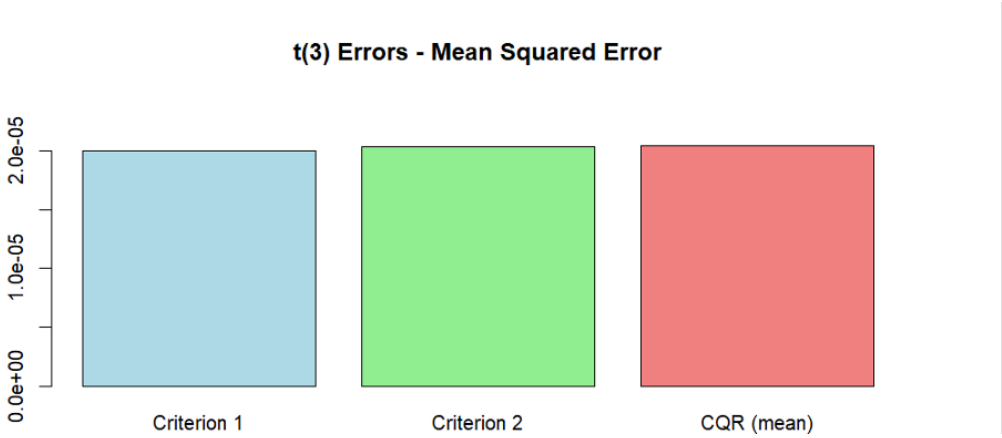

Figure 2.13: barplot of MSE

	Method	MSE_t
1	Criterion 1	2.000024e-05
2	Criterion 2	2.032522e-05
3	CQR (mean)	2.041116e-05

Table 2.7: Comparison of methods based on t-student

2. **Criterion 2**, which applies Koenker-type weights (0.3239, 0.3522, 0.3239) designed for improved efficiency;
3. **Composite Quantile Regression (CQR)**, which takes the unweighted average of slope estimates from those three quantiles.

The MSE values obtained are as follows:

- Criterion 1 yielded an MSE of approximately 2.000×10^{-5}.
- Criterion 2 produced an MSE of about 2.033×10^{-5}.
- CQR resulted in an MSE of 2.041×10^{-5}.

These results indicate that Criterion 1 slightly outperforms the other two methods in terms of estimation accuracy under heavy-tailed error conditions. The differences in MSE, while numerically small, suggest that the choice of weighting scheme can subtly influence performance, especially in models sensitive to the distributional characteristics of the error terms.

This barplot serves as a clear and concise summary of overall estimator accuracy. It complements the earlier boxplot analysis, which examined the distribution and variability of the estimates, by presenting a single performance metric that combines both bias and variance.

In addition to MSE, the empirical variances of the estimated slope coefficients were also computed to assess the stability and consistency of each method. **The variance reflects how much the slope estimates fluctuate across different simulations**, independent of any bias. The observed variances were $2.001 * 10^{-5}$ for Criterion 1,$2.034 * 10^{-5}$ for Criterion 2, and $2.042 * 10^{-5}$ for CQR. These values closely mirror the MSE results, confirming that Criterion 1 not only has the lowest MSE but also exhibits the least variability in estimates.

Evidence: This consistency reinforces the conclusion that Criterion 1 performs slightly better than the other two methods in the presence of heavy-tailed error distributions. The small differences in variance also highlight the robustness of all three methods under the specified conditions.

Monte Carlo Simulation

The Monte Carlo simulation conducted under the assumption of heavy-tailed error terms following a t-distribution with 3 degrees of freedom provides valuable insight into the performance of different quantile regression aggregation methods.

Specifically, the analysis compared three approaches:

- Criterion 1 (simple fixed weights)
- Criterion 2 (Koenker-type efficiency weights)
- Composite Quantile Regression (CQR) using an equal average of estimates.

Results

Across 1000 replications, ***Criterion 1*** consistently demonstrated the lowest mean squared error ($2.000 * 10^{-5}$) and smallest variance ($2.001 * 10^{-5}$), indicating slightly better estimation accuracy and stability compared to the other methods. Although all three approaches performed similarly in magnitude, the results suggest that Criterion 1 may offer a marginal advantage in settings with heavy-tailed, non-Gaussian noise. This reinforces the usefulness of quantile-based methods in robust regression, particularly when classical assumptions such as normally distributed errors are violated. Overall, these findings emphasize the importance of weighting strategies in composite quantile regression and support Criterion 1 as a practical and reliable choice under heavy-tailed conditions.

2.2.4 Laplace Distribution Function

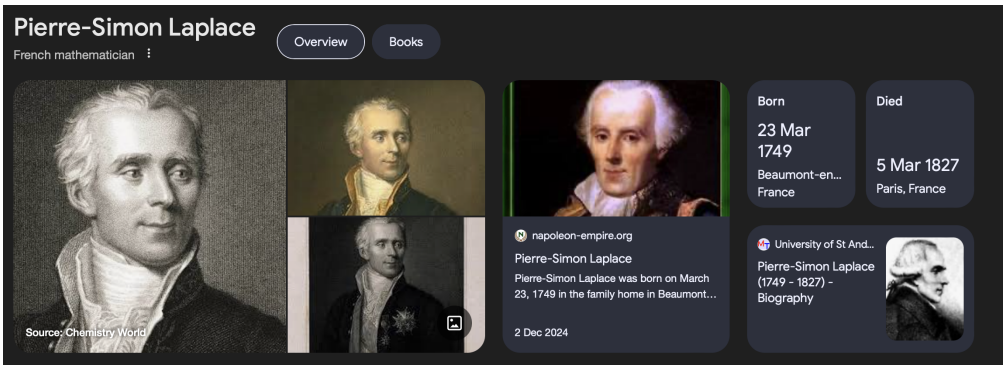

Figure 2.14: Pierre-Simon Laplace

IN BRIEF: The **Laplace distribution**, also called the **double exponential distribution**, is the distribution of differences between two independent variates with identical exponential distributions.

Density Function

$$f(x) = \frac{1}{2b} \exp\left(-\frac{|x-a|}{b}\right)$$

Where:

$$x \in (-\infty, +\infty) \quad \text{(domain of the variable)}$$
$$a(\mu) \in (-\infty, +\infty) \quad \text{(location parameter: mean)}$$
$$b > 0 \quad \text{(scale parameter: dispersion)}$$

Properties:

- The distribution is symmetric around a
- The variance is given by: $\text{Var}(X) = 2b^2$

Why this function it is also called "Double Exp" ?

The reason is quite simple to explain, in order to do it we have to use its graphical visualization:

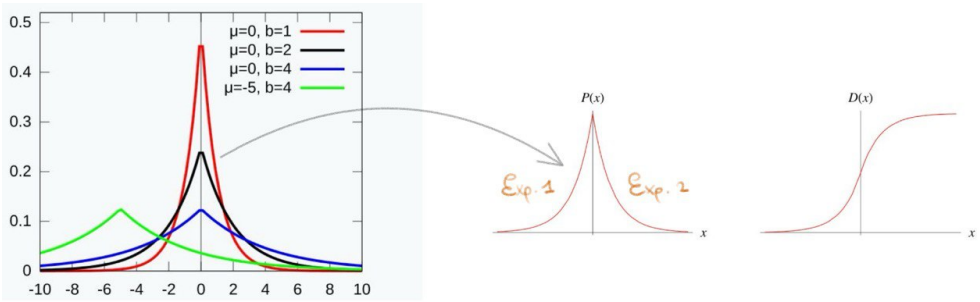

Figure 2.15: Graphical Demonstration of Double Exp

A continuous random variable X is said to follow a **Laplace** (Double exponential) distribution with two parameters μ, b if its pdf-Probability Density Function is defined as:

$$f(x) = \begin{cases} \frac{1}{2b} e^{-\frac{|x-\mu|}{b}} & \text{for } -\infty < x < \infty \\ 0 & \text{otherwise} \end{cases}$$

Setting : $\mu = 0, \quad b = 1 \quad \Rightarrow$ **Standard Laplace Distribution** $\Rightarrow$ This is precisely the operation performed by the R function **"rlaplace()"** (it inverts the above cdf-Cumulative Distribution Function).

It is denoted by:

$$Y \sim \text{Lap}(0, 1)$$

The graphical visualization reveals the following:

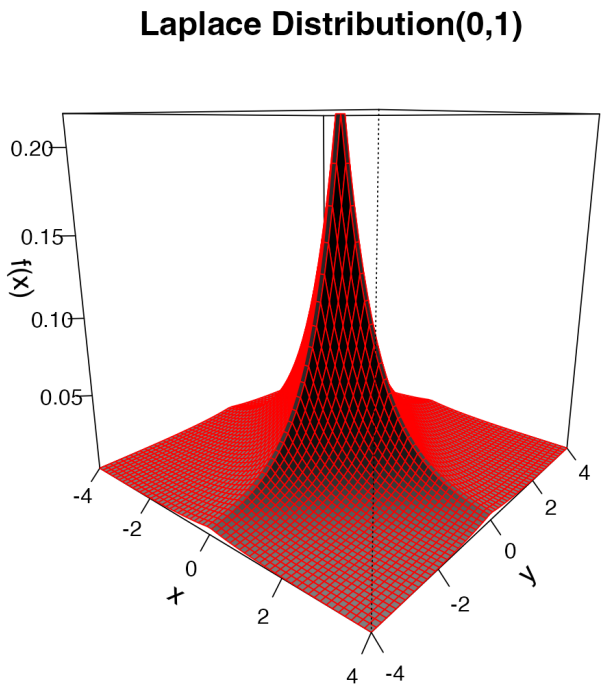

Figure 2.16: Graphical Demonstration of Standard Laplace Distribution

What can be concluded from this?

The probability distribution exhibits two defining characteristics. Firstly, it possesses a highly pronounced central peak, which signifies that most observations are concentrated around the median value. Secondly, the distribution has substantially heavier tails when compared to a Normal distribution.

In statistical terms, the shape of a distribution's peak and tails is quantified by a measure known as ***kurtosis***. The described distribution, with its distinct peak and heavy tails, is classified as leptokurtic. A leptokurtic distribution is formally defined as one having a kurtosis value greater than 3, which is the kurtosis value of the standard Normal distribution. This higher value confirms that extreme values are more probable than in a Normal distribution.

Monte Carlo Simulation

As a continuation of the previous Monte Carlo simulations conducted under Normal, Cauchy, and t-distributed errors, this section evaluates the behavior of quantile regression estimators when the model errors follow a Laplace distribution, also known as the double exponential distribution. The Laplace distribution shares symmetry with the normal distribution but exhibits sharper peaks and heavier tails, making it an appropriate model for data with frequent small deviations and occasional large outliers.

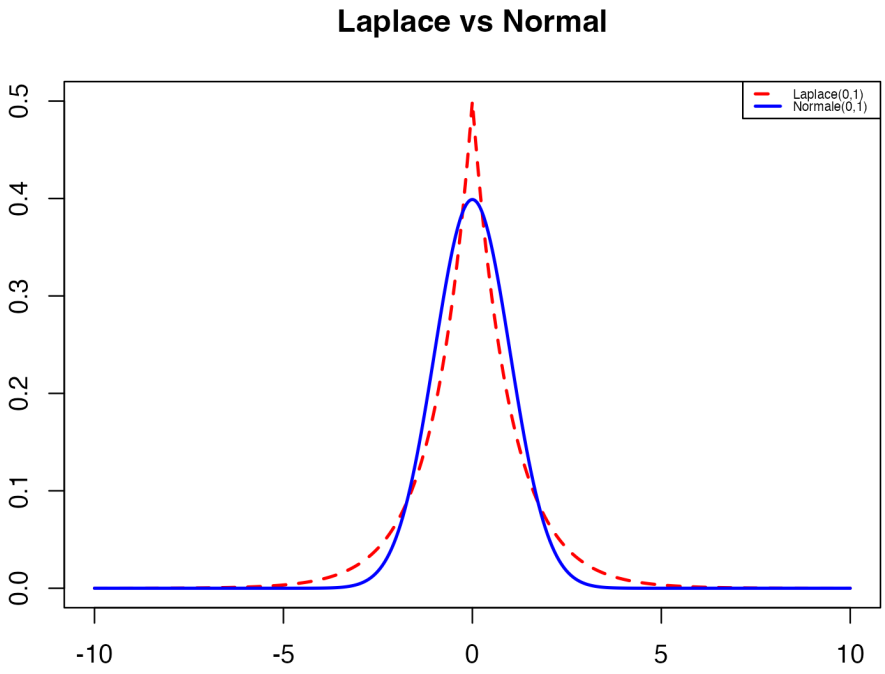

Figure 2.17: Comaprison between Laplace and Normal distribution

The same linear data-generating process is used for consistency across simulations:

SLM function

$$y_i = \alpha + \beta x_i + u_i$$

with :

- $\alpha = 0$
- $\beta = 1$
- $x_i \in 1, 2, \ldots, 100$.

The error term u_i is drawn from the standard Laplace distribution **Lap(0,1)**, generated manually using the inverse transform method to ensure precision. The simulation is repeated N=1000 times with n=100 observations per replication.

To assess robustness and efficiency, the same three estimation strategies from earlier sections are applied:

- ***Criterion 1***: Quantile regression using fixed weights $(0.25, 0.5, 0.25)$
- ***Criterion 2***: Koenker-type efficiency weights $(0.3239, 0.3522, 0.3239)$
- ***CQR (mean)***: A simple average of estimates across $\tau = 0.25, 0.5, 0.75$

Each method's performance is evaluated based on the Mean Squared Error (MSE) and the empirical variance of the estimated slope coefficient β. This allows for direct comparison with the results under Normal, Cauchy, and t-distributed errors.

Result summary obtained using:

```
print(ME_table_lap) MSE
```

Method	Criterion	ME_laplace
1	Criterion 1	0.007142093
2	Criterion 2	0.007636577
3	CQR (mean)	0.007715858

Table 2.8: Comparison of methods based on ME-Laplace

```
> print(Var_table_lap)
```

Method	Criterion	Variance_laplace
1	Criterion 1	0.007140048
2	Criterion 2	0.007633396
3	CQR (mean)	0.007712540

Table 2.9: Comparison of methods based on Variance-Laplace

Final analysis:

In alignment with the results from the t-Student and Cauchy simulations, this Laplace-based analysis further confirms the robustness of quantile regression methods in the presence of non-normal, heavy-tailed error structures. Among the methods tested, ***Criterion 1*** again emerges as the most stable and efficient estimator, with slightly better accuracy and consistency compared to Koenker-weighted and unweighted (CQR) approaches. These findings strengthen the overall conclusion that quantile-based estimation strategies are highly effective for econometric models prone to outliers and tail-risk, which are common features in real-world data.

2.2.5 Skew-t Distribution Function

Building on the previous Monte Carlo simulations under Normal, Cauchy, t-Student, and Laplace error distributions, this section examines the performance of quantile regression estimators when the errors follow a skewed t-distribution.

In Brief:This distribution combines both asymmetry and heavy tails, providing a more complex and realistic error structure that mirrors many empirical settings in economics and finance.

Linear model

$$y_i = \alpha + \beta x_i + u_i$$

$\alpha = 0$
$\beta = 1$
x $\in 1, 2, ..., 100$

The error term u_i is generated from a skewed t-distribution with:

- location$x_i = 0$
- scale $\omega = 1$
- skewness $\alpha = 5$
- degrees of freedom $\nu = 5$.

These parameters introduce moderate asymmetry and heavy-tailed behavior into the data. Each simulation consists of 100 observations and is repeated 1,000 times to compute robust performance metrics.

The estimators evaluated are consistent with prior sections:

- Criterion 1: Weighted quantile regression with fixed weights (0.25,0.5,0.25)
- **Criterion 2**: Koenker-type weights (0.3239,0.3522,0.3239)
- CQR (mean): Unweighted average across the three quantile levels $\tau = 0.25, 0.5, 0.75$

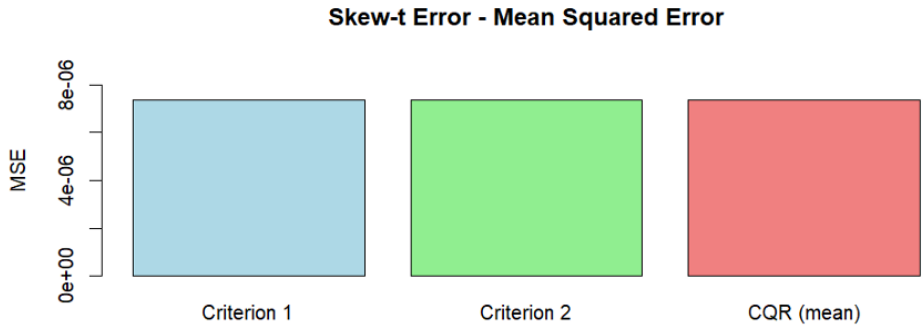

Figure 2.18: Barplot of MSE, skew-t error

The barplot titled "Barplot of MSE" presents the empirical MSE of the slope estimates β for each method across all simulation runs.

The MSE values are as follows:

- Criterion 1 yields an MSE of $7.364 * 10^{-6}$
- Criterion 2 performs slightly better with an MSE of $7.343 * 10^{-6}$
- CQR (mean) follows closely at $7.357 * 10^{-6}$.

Result: Although the differences are small, these results suggest that Koenker-type weights (Criterion 2) achieve marginally better efficiency under the skewed t-distribution, followed closely by CQR and Criterion1.

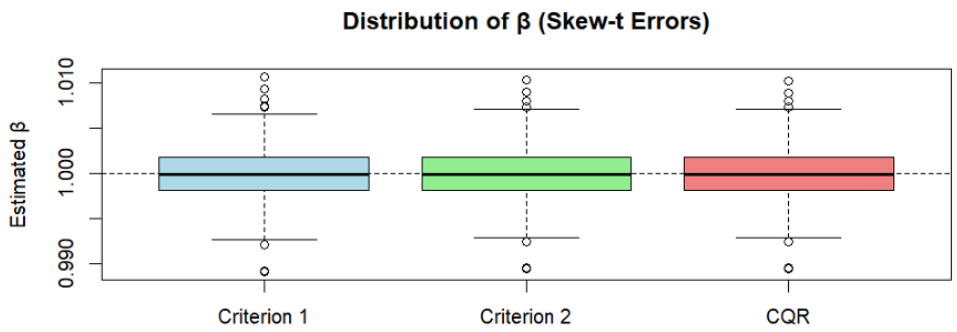

Figure 2.19: Boxplot of distribution of β, skew-t error

Complementing this, the boxplot titled **"Boxplot of distribution of β"** illustrates the distribution of estimated slope values for each method. The horizontal dashed line at $\beta = 1$ serves as a reference to assess estimator bias. All three methods produce distributions centered closely around the true value, confirming their unbiasedness.

However, there are small but notable differences in variance, which reflect the consistency of the estimators.

The empirical variances are:

- $7.366 * 10^{-6}$ for **Criterion 1**
- $7.344 * 10^{-6}$ for **Criterion 2**
- $7.358 * 10^{-6}$ for the **CQR estimator**.

Again, Criterion 2 shows the lowest variance, reinforcing its slight advantage in terms of stability.

Result: This simulation confirms that all three quantile-based estimators remain robust under skewed and heavy-tailed conditions.
The graphical results, both the MSE barplot and the slope distribution boxplot, demonstrate that while performance differences are modest, Criterion 2 performs slightly better, benefiting from its efficiency-oriented weighting scheme.

These findings are consistent with earlier results from Normal, Cauchy, t-Student, and Laplace simulations, reaffirming that composite quantile regression estimators are highly robust, and that thoughtful weighting strategies (such as Koenker-type weights) can yield incremental but meaningful improvements in estimator accuracy and stability.

2.2.6 Skew-normal Distribution Function

Continuing the Monte Carlo analysis across different error structures, this section investigates estimator performance under the skew-normal distribution. Unlike previously studied distributions, such as Normal, Cauchy, t-Student, Laplace, and skew-t, which introduced symmetry or heavy tails the skew-normal distribution adds moderate asymmetry but maintains light tails, it is well suited for modeling real-world phenomena with mild skewness and limited extreme values.

The data-generating process remains consistent:a simple linear model

$$y_i = \alpha + \beta x_i + u_i \tag{2.21}$$

where:

- $\alpha = 0$,
- $\beta = 1$
- x_i is a deterministic sequence from 1 to 100.

The error term u_i is generated from a **skew-normal distribution** with:location $x_i = 0$, scale $\omega = 1$, and skewness parameter $\alpha = 5$, using the *rsn()* function from the ***sn*** package. This configuration introduces clear asymmetry while preserving finite variance and avoiding extreme outliers. Each simulation was repeated 1,000 times using 100 observations per iteration.

The three estimation strategies examined are the same as in previous sections:

- ***Criterion 1***, which uses symmetric weights (0.25, 0.5, 0.25);
- ***Criterion 2***, which uses Koenker-type weights optimized for efficiency;

- ***CQR (mean)***, which takes a simple average across the 0.25, 0.5, and 0.75 quantiles.

Objective: The goal is to evaluate the mean squared error (MSE) and variance of the slope estimates β across all simulations.

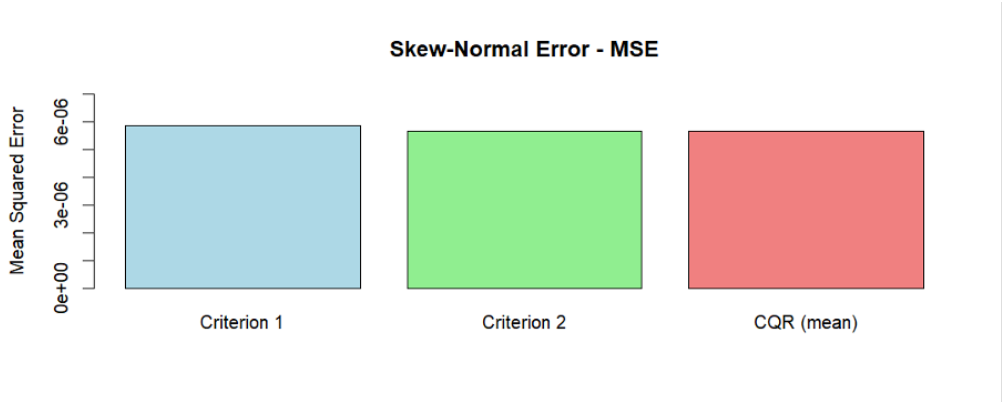

Figure 2.20: Barplot of MSE, skew-normal error

The barplot titled **"Barplot of MSE, skew-normal error"** displays the mean squared error for each method. The results are extremely close in value:

- Criterion 1 yields an MSE of $5.855 * 10^{-6}$
- Criterion 2 shows a slightly lower MSE of $5.675 * 10^{-6}$
- CQR (mean) performs best with an MSE of $5.664 * 10^{-6}$

These minimal differences indicate that all three estimators perform very well under the skew-normal setting.

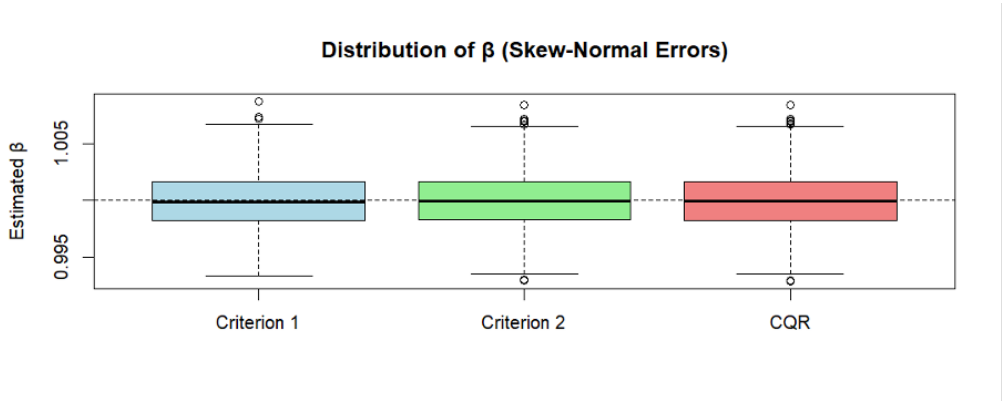

Figure 2.21: Boxplot of distribution of β, skew-normal error

The boxplot titled **"Boxplot of distribution of β, skew-normal error"** confirms that the slope estimates are centered around the true value of $\beta = 1$, with very low variance. The variances align with the MSE results:

- **Criterion 1** at $5.858 * 10^{-6}$
- **Criterion 2** at $5.678 * 10^{-6}$

- **CQR (mean)** at $5.667 * 10^{-6}$.

Analysis: To provide further insight, these results are compared to the previously analyzed Skew-t distribution, which shares the same skewness parameter but introduces heavy tails via low degrees of freedom ($\nu = 5$). The skew-t distribution allows for larger and more frequent extreme values, simulating a more volatile data environment. Under the skew-t setting, the estimators showed higher MSEs and variances:

- **Criterion 1** had an MSE of$7.364 * 10^{-6}$
- **Criterion 2** had $7.343 * 10^{-6}$
- **CQR (mean)** had $7.357 * 10^{-6}$.

Variance followed the same trend, indicating that while estimators remain unbiased under both distributions, their stability decreases when the data contain heavy-tailed outliers.

Method	MSE (Skew-t)	Variance (Skew-t)	MSE (Skew-Normal)	Variance(Skew-Normal)
Criterion 1	$\mathbf{7.364 \times 10^{-6}}$	7.366×10^{-6}	$\mathbf{5.855 \times 10^{-6}}$	5.858×10^{-6}
Criterion 2	$\mathbf{7.343 \times 10^{-6}}$	7.344×10^{-6}	$\mathbf{5.675 \times 10^{-6}}$	5.678×10^{-6}
CQR (mean)	$\mathbf{7.357 \times 10^{-6}}$	7.358×10^{-6}	$\mathbf{5.664 \times 10^{-6}}$	5.667×10^{-6}

Comparison:

The comparison reveals that skew-normal errors lead to more stable and accurate estimates due to the lack of heavy tails. While the skewness level is the same in both distributions, the tail behavior of the skew-t significantly increases variability. Still, the rankings of estimator performance remain consistent:

- CQR (mean) and Criterion 2 tend to perform slightly better in both settings
- Criterion 1, though still robust, shows marginally higher MSE and variance.

Conlusions:

This simulation confirms that composite quantile regression estimators are robust across a range of asymmetric distributions, but that their performance is sensitive to tail heaviness. Under lighter-tailed conditions like the skew-normal distribution, all three methods **perform extremely well**, with minimal differences. Under heavier-tailed distributions like the skew-t, the estimators experience **slightly higher variability**, though they continue to offer reliable, unbiased estimation. These findings reinforce the practical value of quantile regression and suggest that efficiency-optimized or averaged weighting schemes (Criterion 2 and CQR) offer modest but consistent advantages when estimating models with asymmetric error structures.

2.2.7 Comparison of all six distributions

Distribution	*Method*	*MSE*	*Variance*	*Tail Behavior*	*Skewness*
Normal	Criterion 1	1.407761e-05	1.408785e-05	Light-tailed	Symmetric
Normal	Criterion 2	1.366737e-05	1.364834e-05	Light-tailed	Symmetric
Normal	CQR(mean)	1.361507e-05	1.362177e-05	Light-tailed	Symmetric
Cauchy	Criterion 1	4.182764e-05	4.184945e-05	Very heavy-tailed	Symmetric
Cauchy	Criterion 2	4.779910e-05	4.779993e-05	Very heavy-tailed	Symmetric
Cauchy	CQR(mean)	4.862267e-05	4.865843e-05	Very heavy-tailed	Symmetric
Student-t	Criterion 1	2.000024e-05	2.000925e-05	Heavy-tailed	Symmetric
Student-t	Criterion 2	2.032522e-05	2.033639e-05	Heavy-tailed	Symmetric
Student-t	CQR(mean)	2.041116e-05	2.042265e-05	Heavy-tailed	Symmetric
Laplace	Criterion 1	0.007142093	0.007140048	Moderate heavy-tailed	Symmetric
Laplace	Criterion 2	0.007636577	0.007633396	Moderate heavy-tailed	Symmetric
Laplace	CQR(mean)	0.007715858	0.007712540	Moderate heavy-tailed	Symmetric
Skew-t	Criterion 1	7.364077e-06	7.366301e-06	Heavy-tailed	Skewed (right)
Skew-t	Criterion 2	7.342754e-06	7.343814e-06	Heavy-tailed	Skewed (right)
Skew-t	CQR(mean)	7.357312e-06	7.358215e-06	Heavy-tailed	Skewed (right)
Skew normal	Criterion 1	5.855427e-06	5.857939e-06	Light-tailed	Skewed (right)
Skew normal	Criterion 2	5.674702e-06	5.678402e-06	Light-tailed	Skewed (right)
Skew normal	CQR(mean)	5.663513e-06	5.667341e-06	Light-tailed	Skewed (right)

Table 2.10: Comparison of the different methods.

The table presents a comparative analysis of various probability distributions evaluated using three different methods:

- **Criterion 1**
- **Criterion 2**
- **Conditional Quantile Regression mean (CQR(mean)).**

The distributions examined include ***Normal, Cauchy, Student t, Laplace, Skew-t, and Skew normal***. For each distribution-method pair, the table reports the mean squared error (MSE) and variance, alongside qualitative characteristics such as tail behavior and skewness.

Analysis

The Normal distribution demonstrates very low MSE and variance values, approximately on the order of $1.4 * 10^{-5}$ across all methods. It is characterized by light-tailed behavior and symmetry, reflecting its well-known properties. Similarly, **the Student t distribution**, known for its heavier tails, shows slightly higher MSE and variance values around $2.0 * 10^{-5}$. This distribution maintains symmetry but exhibits heavy-tailed behavior, which is consistent with its capacity to model data with outliers better than the Normal distribution.

The Cauchy distribution, with its very heavy tails, results in notably higher MSE and variance values, ranging roughly from $4.18 * 10^{-5}$ to $4.86 * 10^{-5}$. Like the previous symmetric distributions, it remains symmetric but is better suited for modeling data with extreme values due to its heavier tails. In contrast, **the Laplace distribution** exhibits moderate heavy-tailed behavior and symmetric skewness, but its MSE and variance values are substantially higher, approximately 0.0071 to 0.0077, indicating greater variability or estimation error under the methods tested.

Among the skewed distributions, **the Skew-t distribution** stands out by achieving the lowest MSE and variance values, around $7.34 * 10^{-6}$ to 7.36 * 10-6, while maintaining heavy-tailed behavior and right skewness.

This suggests that the Skew-t distribution may provide more accurate modeling for skewed, heavy-tailed data relative to the other distributions assessed. **The Skew normal distribution**, which is light-tailed and also right skewed, shows slightly higher MSE and variance than the Skew-t but remains competitive, with values around $5.66 * 10^{-6}$ to $5.86 * 10^{-6}$.

In general, the results highlight clear performance differences depending on the tail behavior and skewness of the distribution. Symmetric, light-tailed distributions, such as normal, exhibit low estimation errors, while heavy-tailed distributions generally show higher variability. Skewed distributions, particularly the Skew-t, demonstrate promising accuracy for modeling asymmetric heavy-tailed data. The consistency of MSE and variance values across the three methods suggests robustness in the comparative evaluation.

Chapter 3

Empirical Application

3.1 Analysis CO_2 growth rate

3.1.1 Introduction

Understanding the dynamics of CO_2 emissions is critical for both environmental policy and macroeconomic planning. Given the increasing urgency of climate change, examining how macroeconomic factors influence the growth rate of CO_2 emissions provides valuable information on the interaction between economic activity and environmental outcomes.

Objective

This study analyzes the short-term dynamics of CO_2 emission growth in relation to key macroeconomic indicators. A cleaned data set of CO_2 emissions was transformed into percentage growth rates by logarithmic differencing, ensuring stationarity and interpretability in relative terms. To maintain analytical consistency, the first observation was removed, and the resulting growth series was used as the primary variable of interest.

Macroeconomic explanatory variables were obtained from:

- the Federal Reserve Economic Data (**FRED**)
- the Industrial Production Index (**INDPRO**)
- Unemployment Rate (**UNRATE**)
- Federal Funds Rate (**FEDFUNDS**)
- Total Nonfarm Payrolls (**PAYEMS**).

These variables, chosen for their relevance to economic cycles and policy, were logarithmically different and expressed as percentage changes to reflect relative movements over time.

To address multicollinearity and extract underlying macroeconomic factors, **Principal Component Analysis (PCA)** was applied to the standardized macroeconomic series. The first two principal components (**PC1** and **PC2**), which explained most of the variance, were retained. Due to differences in the duration of the time series, the PCA outputs were trimmed to align with the growth data CO_2, which facilitates coherent model estimation.

The analysis focused on modeling the one-period-ahead CO_2 growth rate as a function of the contemporaneous values of PC1 and PC2. Two types of regression models were employed. First, an **Ordinary Least Squares (OLS)** regression estimated the conditional mean relationship. Second, **Quantile Regression (QR)** was used to estimate conditional quantiles ($\tau = 0.25, 0.5, 0.75$), capturing heterogeneous effects across the distribution of CO_2 growth rates.

To further explore the distributional effects, QR was extended to cover ***19 quantiles*** from 0.05 to 0.95. Coefficient trajectories across quantiles were visualized using *ggplot2*, with quantiles plotted along the y-axis to enhance interpretability. OLS estimates and confidence intervals were overlaid to compare mean effects with quantile-specific sensitivities.

Finally, to evaluate the out-of-sample predictive performance of the models, a rolling window forecasting approach was implemented. The dataset was divided into:

1. a training period (two-thirds of the sample)
2. a testing period (remaining one-third).

Both OLS and QR models, trained on expanding windows, were used to forecast future CO_2 growth rates, providing insights into the models' robustness and practical applicability for environmental-economic forecasting.

3.1.2 Environment Preparation and CO_2 Data Cleaning

The analysis begins with setting the appropriate working directory and importing a dataset named "CO_2 _SA.csv" using the **read.csv()** function. This dataset contains seasonally adjusted CO_2 emissions values over time. A data cleaning step follows, where missing values (NAs) are removed using the **na.omit()** function to ensure that subsequent analysis is not distorted by incomplete observations

To make the CO_2 data suitable for time series analysis, the emissions values are first log-transformed to stabilize variance and then differenced to calculate the growth rate. Specifically, the code computes the first difference of the natural logarithm of CO_2 values and multiplies it by 100 to express it in percentage terms. This transformation is standard in macroeconomic time series analysis to make the data stationary. The resulting growth rates are then appended to the dataset as a new column called ***GrowthRate***.

3.1.3 Macroeconomic Variables

To construct the independent variables for the model, the analysis utilizes the **quantmod** package to download time series data from **FRED**. The variables selected include:

- **INDPRO**: Industrial Production Index
- **UNRATE**: Unemployment Rate
- **FEDFUNDS**: Federal Funds Rate
- **PAYEMS**: Total Nonfarm Payrolls (employment)

Each variable is downloaded using the **getSymbols()** function, which retrieves monthly historical data directly from the FRED database. These four variables were selected because PCA requires at least three independent variables to extract meaningful components, and these indicators represent core dimensions of economic activity: output, labor market conditions, monetary policy, and employment.

After confirming the successful retrieval of data using **head()** and **tail()**, the four time series are merged into a single data frame using the **merge()** function. Any resulting rows with missing values are omitted to ensure clean input for analysis.

Next, each macroeconomic time series is transformed into a growth rate series in percentage terms. This is done by taking the first difference of their natural logarithms (as with the CO_2 data), which helps normalize the data and allows for comparison across variables on a similar scale. The resulting data frame, ***macro_var_diff***, contains cleaned and transformed versions of independent macroeconomic variables, each representing the rate of change from one period to the next.

3.1.4 Calculation Part: Applying Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is a widely used statistical technique for dimensionality reduction, especially in multivariate datasets where variables may be correlated. The core idea behind PCA is to transform the original variables into a new set of uncorrelated variables called principal components. Each principal component is a linear combination of the original variables and is ordered such that the first component captures the maximum possible variance, the second component captures the next highest variance orthogonal to the first, and so on.
PCA serves two key purposes:

1. **Data simplification** – reducing a large number of correlated variables into a smaller set that still retains most of the original information.
2. **Pattern recognition** – identifying latent structures or trends in the data, such as economic regimes or dominant macroeconomic factors.

Following the preparation and cleaning of both the CO_2 growth rate data and macroeconomic indicators, the next step is to ***reduce dimensionality*** using Principal Component Analysis (PCA). This allows us to extract the most informative linear combinations of the macroeconomic variables, known as principal components, that explain the maximum amount of variance in the data.

In this analysis, PCA is applied to macroeconomic indicators such as ***industrial production, unemployment, interest rates, and employment***, with the goal of summarizing their joint behavior using a smaller number of informative components. These components are later used in further modeling, including quantile regression.

Macroeconomic indicators are retrieved directly from the **FRED database** using *quantmod*.
These include:

- industrial production **(INDPRO)**
- unemployment rate **(UNRATE)**
- federal funds rate **(FEDFUNDS),**
- employment levels **(PAYEMS)**
- the S&P 500 index **(SP500)**
- the 10-year treasury rate **(DGS10)**.

These series are merged and transformed into monthly growth rates using log-differences, which standardizes them and makes them suitable for PCA. The resulting data is cleaned again to remove any remaining NA values.

The PCA is performed using prcomp() with scaling enabled, ensuring that variables with different units or magnitudes do not distort the results. The script calculates the proportion of variance explained by each principal component and visualizes this with a scree plot, helping determine how many components to retain. A detailed breakdown of the PCA loadings, contributions, and cosine squared (cos^2) values is then extracted using *get_pca_var()* from the **factoextra package**. These metrics allow you to understand how much each macroeconomic variable contributes to the principal components and how well they are represented.

```
summary(PCA)#we see the VARIANCE in this way
Importance of components:
                          PC1    PC2     PC3     PC4     PC5     PC6
Standard deviation     2.1220 0.8712 0.64693 0.44285 0.32902 0.12234
Proportion of Variance 0.7505 0.1265 0.06975 0.03269 0.01804 0.00249
Cumulative Proportion  0.7505 0.8770 0.94678 0.97946 0.99751 1.00000
```

The standard deviation associated with each principal component indicates ***the spread or dispersion of the data*** along that component. PC1 has the highest standard deviation (2.1220), meaning it captures the most variation in the dataset. Each subsequent component explains less variability, with standard deviations progressively decreasing down to PC6, which has a very small value of 0.1223.

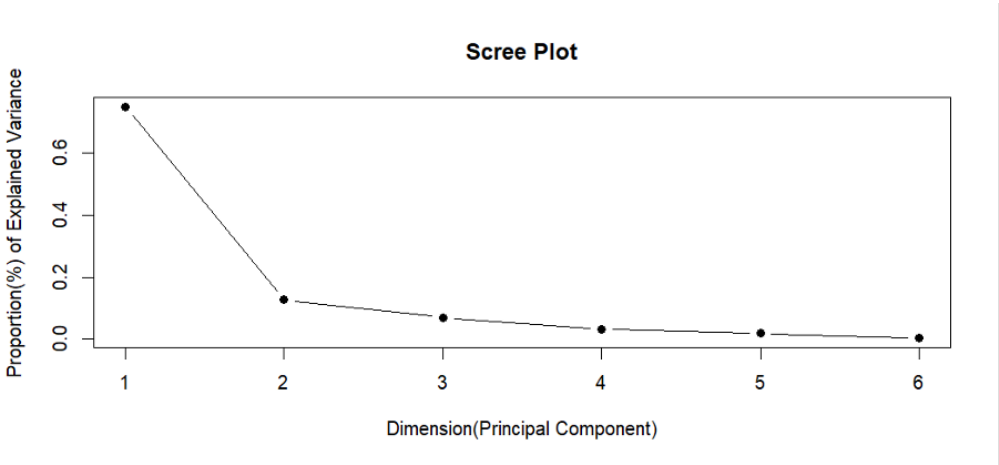

Figure 3.1: Scree plot, proportion of variance

The **scree plot** displays the proportion of variance explained by each principal component (PC) in the dataset. On the x-axis are the components (PC1 to PC6), and on the y-axis the proportion of total variance explained by each. The **goal** of this plot is to identify how many components are necessary to capture the majority of information in the data.

In this case, the proportions of explained variance are:

Component	Percentage Contribution
PC1	75.05%
PC2	12.65%
PC3	6.98%
PC4	3.27%
PC5	1.80%
PC6	0.25%

Table 3.1: Percentage contribution of principal components.

On the plot (Figure 3.1), a steep drop is observed after the first component, and a much smaller drop after the second, suggesting that the first two components alone explain approximately 87.70% of the total variance. This justifies retaining only PC1 and PC2 for further analysis, such as **plotting, clustering, and regression**. These two components summarize the essential structure of the macroeconomic variables while significantly reducing dimensionality.

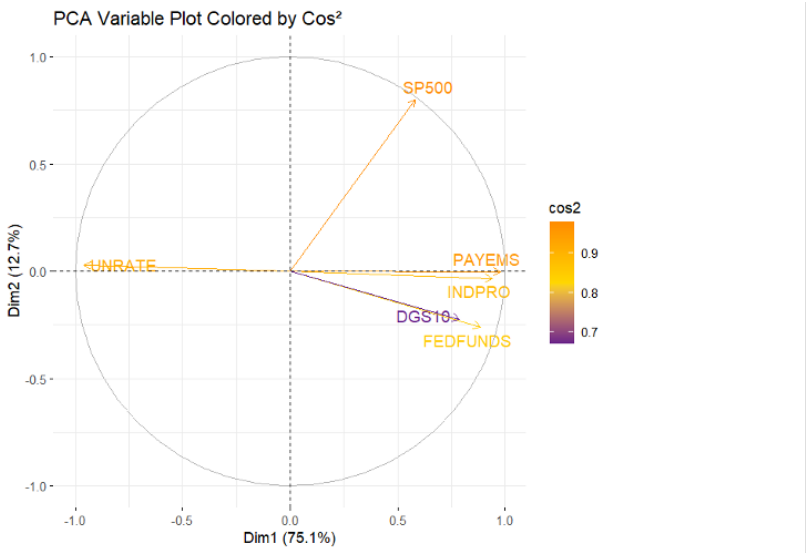

Figure 3.2: PCA variable plot

The relationships between macroeconomic variables can be visually understood through a PCA variable plot (Figure 3.2). This plot maps the variables onto the first two principal components, which together explain 87.8% of the total variance (PC1 at 75.1% and PC2 at 12.7%). Visually, the plot is defined by a strong horizontal spread. The vector for the Unemployment Rate (UNRATE) points distinctly to the left, in direct opposition to a tight cluster of vectors for PAYEMS, INDPRO, FEDFUNDS, and DGS10, all pointing to the right. This arrangement immediately highlights the primary axis of variation (PC1) as a contrast between unemployment and indicators of economic activity. Perpendicular to this main axis, the SP500 vector points strongly upward, establishing the second dimension (PC2) and visually separating the behavior of the stock market from the other variables. The bright color of all vectors, representing a high Cos^2 value, confirms that this two-dimensional chart is a reliable and accurate visual summary of the data.

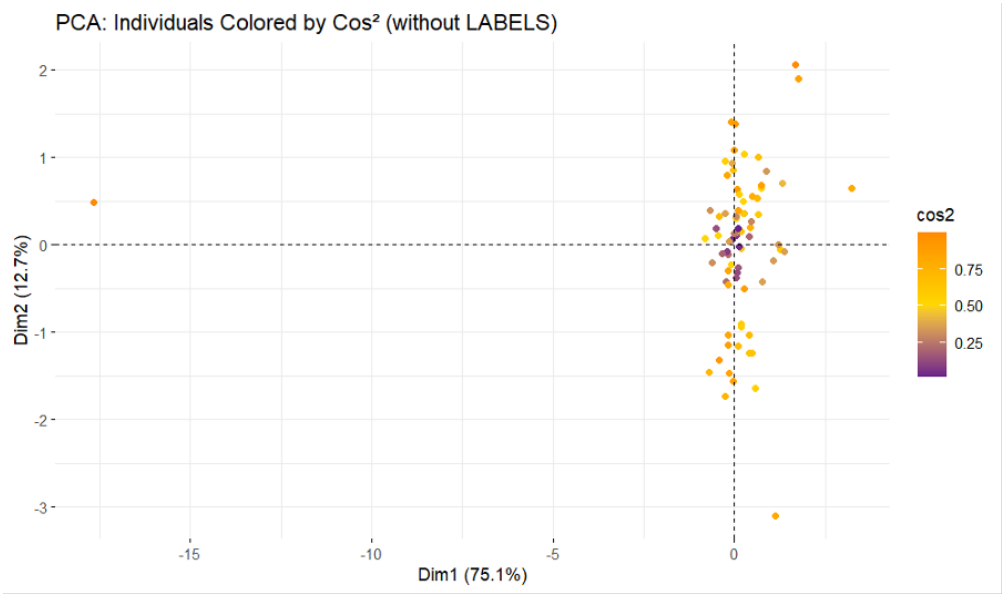

Figure 3.3: PCA individual plot

Complementing the variable plot, the individuals plot (Figure 3.3) shows how each observation (likely

representing a specific point in time, such as a month or quarter) is positioned within this newly defined two-dimensional space. The majority of the points are clustered near the center, slightly to the right of the vertical axis, representing periods of relatively average economic conditions. The most striking feature of this plot is a single, extreme outlier located far to the left on Dim1 (around −17.5). Given that the negative side of Dim1 represents **poor economic health**, this point corresponds to a period of severe economic distress (e.g., ***a major recession***) and is highly influential in defining the entire component. The vertical spread of the points, particularly those with high Cos^2 values (yellow/orange), illustrates how different time periods varied along the "financial market" axis (Dim2). Points high on the chart indicate periods of **strong stock market performance**, while points low on the chart represent times of relatively higher interest rates.

The table below shows how each macroeconomic variable relates to the first two principal components (PC1 and PC2) in terms of:

- Loadings: The correlation between each original variable and the component.
- Contribution (%): How much each variable contributes to forming the principal component.
- Cos^2: The squared cosine value, measuring how well the variable is represented by the component.

Variable	Loading PC1	Loading PC2	Contrib. PC1 (%)	Contrib. PC2 (%)	Cos^2 PC1	Cos^2 PC2
INDPRO	0.942	-0.033	19.69	0.14	0.887	0.001
UNRATE	-0.958	0.029	20.36	0.11	0.917	0.001
FEDFUNDS	0.886	-0.260	17.42	8.88	0.785	0.067
PAYEMS	0.979	-0.002	21.27	0.00	0.958	0.000
SP500	0.582	0.800	7.52	84.32	0.339	0.640
DGS10	0.787	-0.223	13.74	6.55	0.619	0.050

Table 3.2: Loading, Contribution, and Cosine Squared Values of Variables for PC1 and PC2

Looking at the loadings and contributions, PC1 is predominantly driven by four key macroeconomic indicators:

- PAYEMS (Total Nonfarm Payrolls)
- UNRATE (Unemployment Rate),
- INDPRO (Industrial Production Index)
- FEDFUNDS (Federal Funds Rate).

PAYEMS has the highest loading on PC1 (0.979), followed by UNRATE (-0.958), INDPRO (0.942), and FEDFUNDS (0.886). The negative loading of UNRATE reflects its counter-cyclical nature, meaning **unemployment tends to move inversely** with other indicators of economic activity. These variables are also well represented in PC1, as evidenced by high Cos^2 values (all above 0.78), and they contribute substantially to its formation, each accounting for between 17% and 21% of its variance. Therefore, PC1 can be interpreted as a broad component of macroeconomic activity, which captures fluctuations in production, employment, and monetary policy conditions.

In contrast, ***PC2 represents a more specific dimension of variability***, dominated almost entirely by the SP500 stock index, which has a strong loading of 0.800 and contributes over 84% to PC2. Its value Cos^2 on PC2 is 0.640, indicating that much of the SP500's variation is captured by this second dimension. Other variables, such as FEDFUNDS and DGS10 (10-year Treasury rate), contribute only marginally to PC2, with low Cos^2 values, suggesting limited representation. As a result, PC2 can be interpreted as a financial market component, primarily capturing stock market movements independent of the broader macroeconomic environment described by PC1.

Lastly, variables such as **DGS10** are only moderately represented by the first two components. While it has some correlation with PC1 (loading of 0.787), its overall Cos^2 values remain relatively low, indicating

that its variation may be influenced by dimensions not captured in PC1 or PC2. This reflects the distinct behavior of long-term interest rates, which may respond to different economic expectations or financial market dynamics.

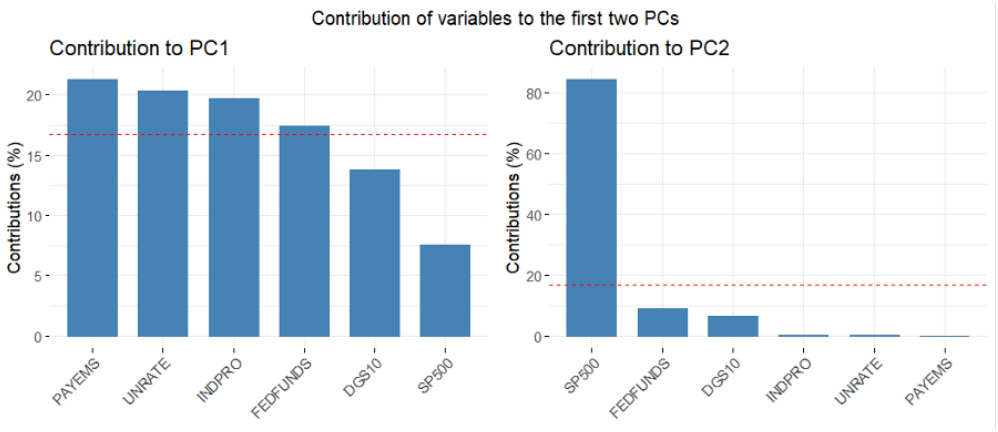

Figure 3.4: Contributions of variables to PC1 and PC2

Conclusion: The bar plots confirm the insights from the PCA summary table: PC1 is mainly driven by PAYEMS, UNRATE, INDPRO, and FEDFUNDS, all of which showed high loadings and contributions in the table, reinforcing its role as a broad macroeconomic component. PC2, as highlighted earlier, is dominated by the SP500, reflecting its distinct financial market dimension, with minimal contribution from other variables.

3.1.5 Linear and Quantile regression

With the prepared dataset containing the aligned CO_2 growth rates (**y_next**) and the first two principal components (**PC1**, **PC2**) as predictors, the relationship is estimated using both linear and quantile regression models. This dual approach offers a more comprehensive understanding of how macroeconomic factors are associated with CO_2 emissions across various segments of the distribution.

LM

The linear regression model is first estimated using the **lm()** function in R, which assumes that the conditional mean of the dependent variable can be expressed as a linear function of the predictors. The model specification is:

$$y_{t+1} = \beta_0 + \beta_1 * PC1_t + \beta_2 * PC2_t + u_t \tag{3.1}$$

The output from *summary(lm_model)* provides the estimated coefficients, their standard errors, and significance levels. This model gives a single estimate for the average effect of macroeconomic principal components on CO_2 growth. However, it may fail to capture heterogeneity across different parts of the conditional distribution.

QR

To overcome this limitation, a quantile regression (QR) approach is applied using the **rq()** function from the *quantreg* package. Unlike ordinary least squares (OLS), quantile regression estimates the conditional quantiles (e.g., **median**, **upper** and **lower** quartiles) of the dependent variable, allowing for more detailed insights into how the relationship varies across the distribution of CO_2 growth.
The model is estimated for three quantiles:

- $\tau = 0.25$ (lower quartile)
- $\tau = 0.5$ (median)

- $\tau = 0.75$ (upper quartile).

This means the quantile regression provides separate slope coefficients for each quantile, highlighting any asymmetries in the response of CO_2 growth to macroeconomic influences. The ***summary(qr_model, se = "nid")*** command is used to obtain robust standard errors for inference. These estimates are especially valuable when the effects of predictors differ for lower vs. higher CO_2 growth regimes, which a mean regression model would obscure.

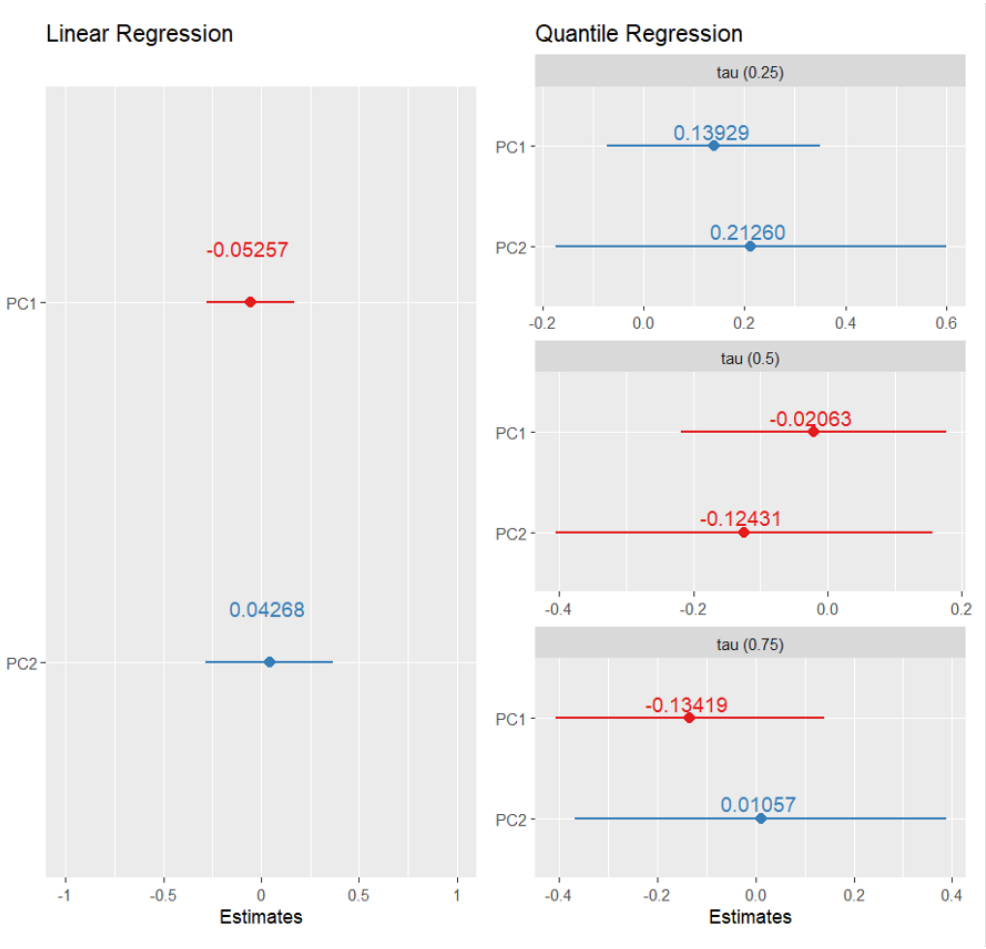

Figure 3.5: comparison of Coefficient estimates, LM and QR

The combined plot generated using the *patchwork package* presents a side-by-side comparison of coefficient estimates from two different regression models: a linear regression model and a quantile regression model.

The left panel, titled "Linear Regression", shows the results from the ordinary least squares (OLS) model. Each plotted point represents the estimated coefficient for a predictor variable, such as PC1 or PC2, while the horizontal lines indicate 95% confidence intervals. Next to each point, the coefficient value is displayed. This panel provides a summary of the average effect of each predictor on the dependent variable, CO_2 growth rates.

The right panel, titled "Quantile Regression", presents the corresponding coefficients from a quantile regression model. Similar in layout to the linear regression panel, it includes the coefficient values and confidence intervals. However, the coefficients in this panel are estimated at a specific quantile of the dependent variable distribution, commonly the median ($\tau = 0.5$) unless otherwise specified. This allows for the examination of how predictor effects may differ at various points in the outcome distribution rather than only at the mean.

Together, these two panels offer a clear visual comparison of how the relationships between predictors and the outcome variable change when analyzed through different regression frameworks. If the coefficients differ in size, or direction between the two models, it suggests heterogeneity in the effects of the predictors depending on the distribution level of CO_2 growth. This comparison enhances understanding by highlighting patterns that may be overlooked when relying solely on linear regression.

3.1.6 Quantile Regression Across Multiple Quantiles and Comparison with OLS

This section extends the quantile regression (QR) framework by analyzing how the estimated effects of macroeconomic principal components on CO_2 growth rates vary across a broader range of the conditional distribution. Instead of focusing on a limited number of quantiles (such as $\tau = 0.25, 0.5, and 0.75$), quantile regression models are estimated for ***19 quantiles***, spanning from $\tau = 0.05$ to $\tau = 0.95$ in increments of 0.05. This comprehensive approach enables a detailed evaluation of the heterogeneity in predictor effects across nearly the entire distribution of CO_2 growth rates, offering deeper insight into distributional dynamics that may be obscured in mean-based or limited-quantile analyses.

The quantile regression model is re-estimated using the **rq()** function with a vector of quantiles (taus_seq), applied to the same specification:

$$y_{t+1} = \beta_0(\tau) + \beta_1(\tau) * PC1_t + \beta_2(\tau) * PC2_t + u_t(\tau) \tag{3.2}$$

To visualize how the estimates of each coefficient change across quantiles, two types of plots are generated. First, using base R graphics through the command ***plot(summary(qr_seq))***, separate plots are produced for the intercept and the coefficients associated with PC1 and PC2. These plots display the progression of each estimate across quantiles, accompanied by confidence intervals, allowing for an assessment of how the influence of each predictor varies along the conditional distribution.

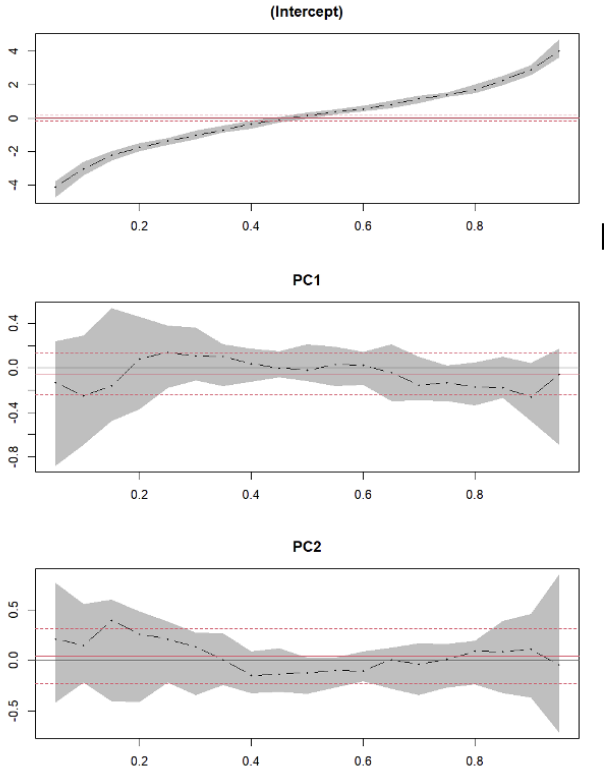

Figure 3.6: Variation of coefficients accros quantiles

Second, the **broom::tidy()** function is applied to convert the quantile regression output into a tidy data format. This transformation facilitates more flexible and detailed visualizations using ***ggplot2***. For each coefficient term, a line plot is constructed with shaded 95% confidence intervals, clearly indicating where the estimated effects are statistically significant. This approach enhances interpretability and allows for a nuanced examination of predictor behavior across the range of quantiles.

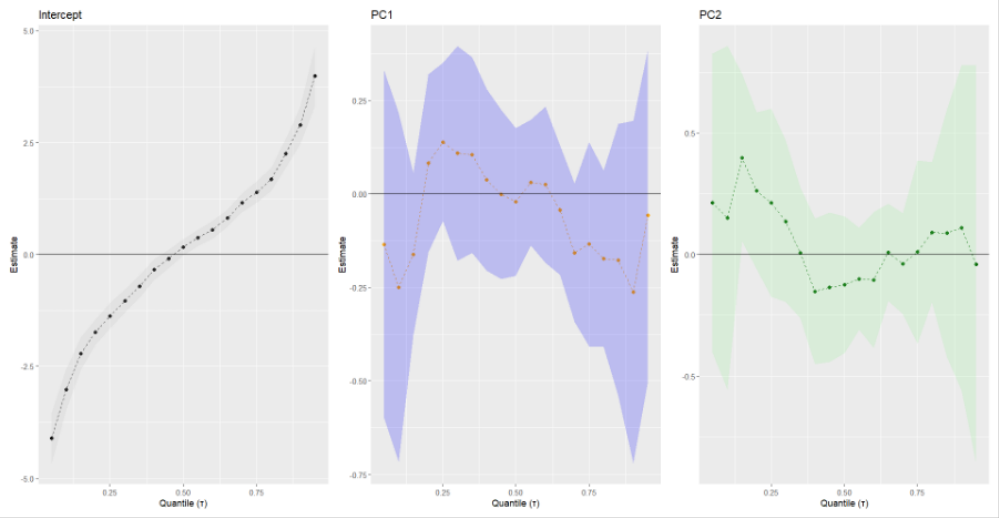

Figure 3.7: Tidy data

For improved interpretation, inverted axis plots are also constructed. In this configuration:

- the x-axis represents the estimated coefficient
- the y-axis corresponds to the quantile τ.

This visual orientation is commonly used in quantile regression analysis as it effectively emphasizes how the response behavior changes across different points of the dependent variable's distribution.

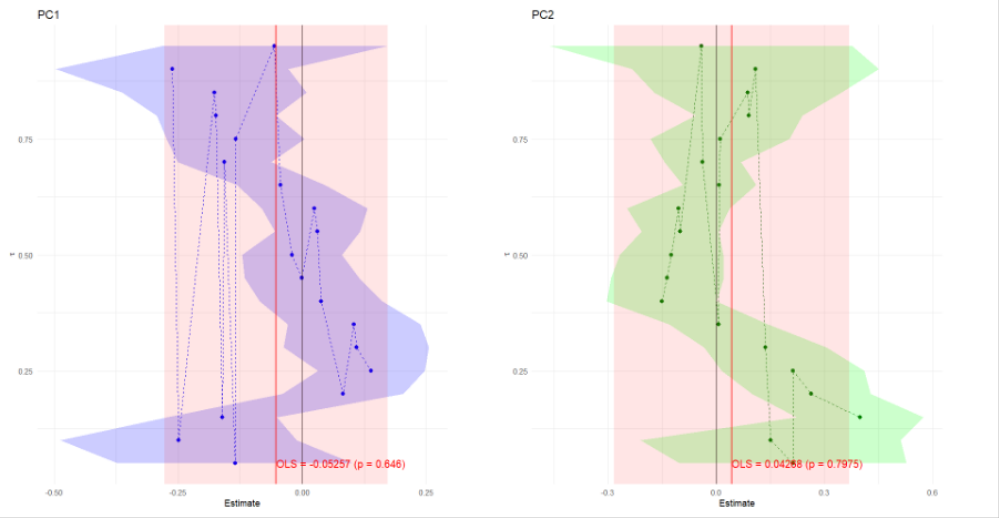

Figure 3.8: Comparison between QR and LM

To enable a comparison between the quantile regression results and the classical linear regression model (OLS), the OLS estimates, and their 95% confidence intervals are overlaid on the QR plots.

On these plots, ***vertical red lines*** represent the OLS coefficients for PC1 and PC2, while ***red shaded areas*** denote their respective confidence intervals. Furthermore, text annotations display the OLS coefficient values along with their corresponding p-values. This integrated comparison makes it possible to immediately identify where the quantile-specific effects diverge from the single, mean-based estimate provided by OLS.

From these plots, it becomes evident whether the influence of the principal components is constant across the distribution (supporting OLS) or varies significantly (supporting QR). For example, **if the QR estimates for PC1 differ considerably from the OLS line at low or high quantiles, this suggests asymmetric effects, meaning macroeconomic factors impact CO_2 growth differently at various levels of the distribution**.

This comprehensive quantile regression analysis thus strengthens the argument for using QR techniques in economic applications where distributional heterogeneity is suspected, as in the case of CO_2 emissions growth driven by macroeconomic dynamics.

3.1.7 Rolling Window Forecasting

To evaluate the out-of-sample predictive performance of the linear (OLS) and quantile regression (QR) models, we implement a rolling window forecasting procedure.

What is it?

This method simulates a real-world scenario where only historical information is available at each forecast point, ensuring that future data are not inadvertently used in training. The total sample contains ***623 monthly observations***. Following standard practice, we allocate:

- two-thirds of the sample (415 observations) to the initial training period
- the remaining one-third (208 observations) reserved for rolling predictions.

OLS forcasting

For each step from $t = 416$ to $t = 623$, we re-estimate an OLS model on an expanding window and generate a one-step-ahead forecast. This process produces a vector of 208 predicted values, which we compare to the actual realized values of y_{t+1} using **Root Mean Squared Error (RMSE)** as the performance metric.

QR Forecasting at Median and Multiple Quantiles

A parallel procedure is followed using quantile regression models. Initially, the QR model is estimated with $\tau = 0.5$ (***median regression***), and then extended to additional quantile levels $\tau \in 0.25, 0.5, 0.75$. At each rolling step, the QR model is fitted on the historical data up to time t, and the next-period forecast is computed. This produces a set of quantile-specific forecasts for each horizon.

The RMSE for each model is computed using

$$\text{RMSE} = \sqrt{\frac{1}{H}\sum_{h=1}^{H}(\hat{y}_{t+h} - y_{t+h})^2} \tag{3.3}$$

This measures the average squared error between predicted values $\hat{y}_{t+1}$ and actual values y_{t+1} over H forecast periods. Where H=20 is the size of the out-of-sample test set.
The table below summarizes the RMSE values: The **rolling window** forecast exercise produced root mean squared error (RMSE) values for both Ordinary Least Squares (OLS) and Quantile Regression (QR) models at different quantile levels. Among all models, the **OLS model** achieved the lowest RMSE, with a value of 0.11136, indicating high predictive accuracy. This low RMSE suggests that the conditional mean of the future value y_{t+1} is effectively captured through a linear relationship with the principal components PC1 and PC2. In practical terms, the OLS model's predictions are closely aligned with the actual observations, making it a strong candidate for point forecasting.

	Model	RMSE
	OLS	0.11136
0.25	QR (tau = 0.25)	3.21607
0.5	QR (tau = 0.5)	2.98390
0.75	QR (tau = 0.75)	3.30849

Table 3.3: RMSE values

In contrast, the **RMSE values for the Quantile Regression** models were significantly higher, with RMSEs of 3.21607, 2.98390, and 3.30849 for $\tau = 0.25$, 0.5, and 0.75 respectively. These results show that QR models are less accurate for point prediction when evaluated using RMSE. However, this is not unexpected. Quantile Regression is not designed to minimize squared errors; rather, it estimates specific conditional quantiles of the response variable. As a result, QR models are often more robust to outliers and provide richer distributional information, even if they perform poorly in terms of average forecasting accuracy.

Among the QR models, the one based on the ***median*** ($\tau = 0.5$) achieved the lowest RMSE, though still far above the OLS benchmark. This result may reflect a **relative symmetry** in the conditional distribution of the target variable, where the central quantile offers a slightly better fit than the tails. Still, the performance gap between QR and OLS is substantial.

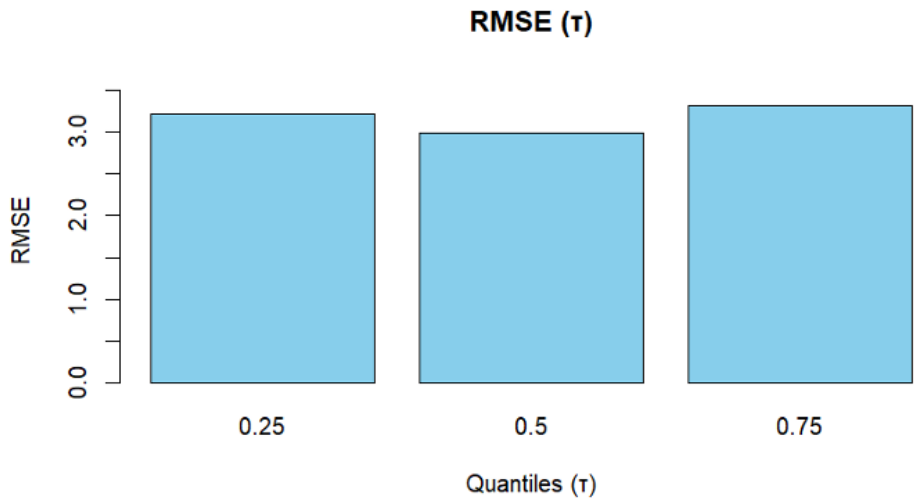

Figure 3.9: RMSE Barplot for Quantile Regression Models (QR)

The first barplot visualizes the **Root Mean Squared Error (RMSE)** for the Quantile Regression (QR) models at three different quantile levels: $\tau = 0.25$, 0.5, and 0.75. Each bar corresponds to a QR model and reflects the ***prediction error magnitude***. The bars show that all QR models perform similarly in terms of RMSE, with $\tau = 0.5$(the median) producing a slightly **lower RMSE** compared to $\tau = 0.25$ and $\tau = 0.75$. Despite this, all QR models still exhibit relatively high RMSE values (around 3.0 or more), which indicates larger forecast errors when using quantile-specific models. The uniform **"skyblue"** color highlights the group as QR models, and the relatively tall bars emphasize their inferior forecasting performance compared to OLS.

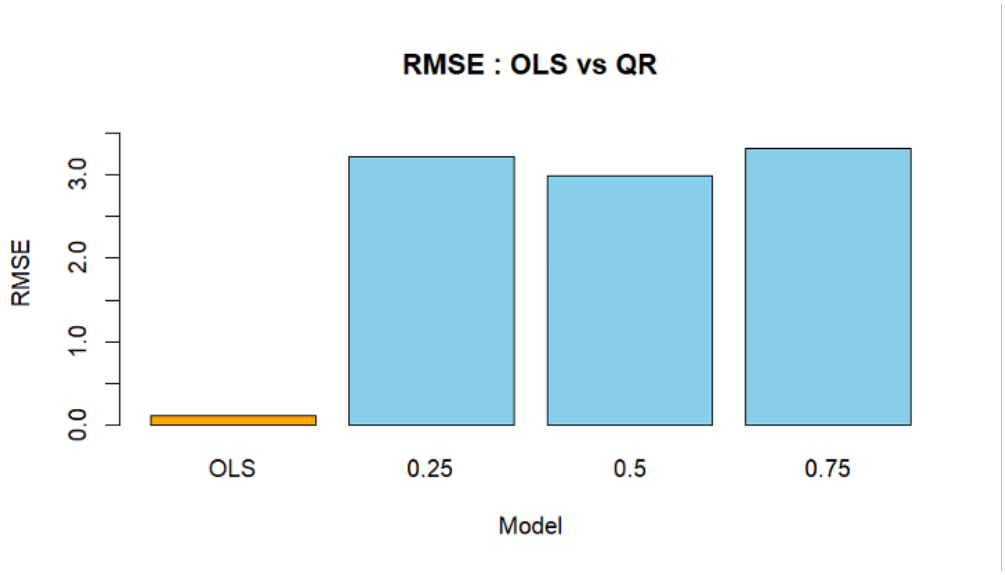

Figure 3.10: RMSE Barplot for OLS vs QR

The second barplot presents a direct comparison between OLS and all QR models ($\tau = 0.25, 0.5, 0.75$) based on their RMSE values. The orange-colored bar represents the OLS model, which is significantly lower in height than any of the blue QR bars, visually confirming that OLS achieves the smallest forecasting error. This plot clearly demonstrates that OLS outperforms all quantile regression models in terms of point prediction accuracy over the rolling window test set. The layout helps readers quickly identify the most accurate model and compare the error magnitude across techniques. The line graph is comparing RMSE

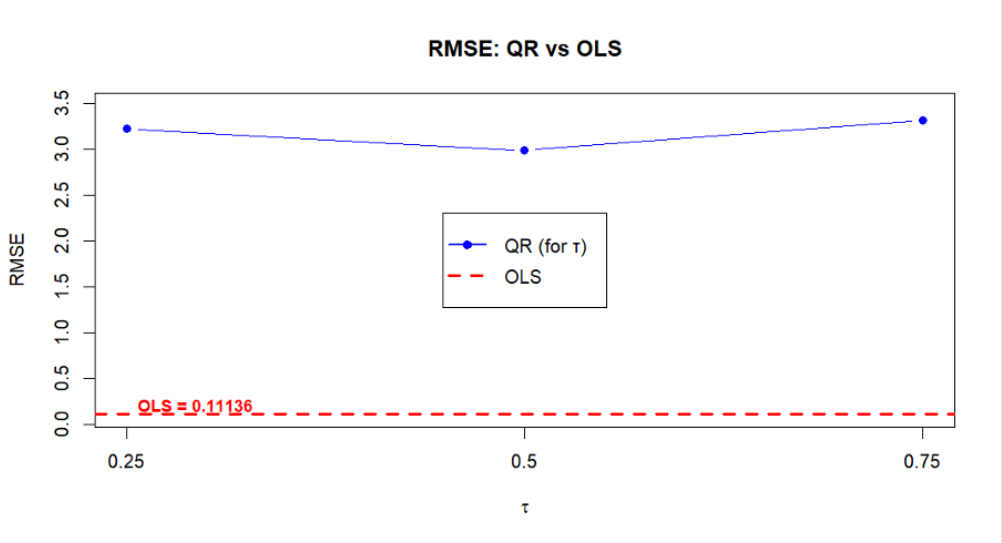

Figure 3.11: RMSE Line Plot: QR vs OLS

across quantiles (τ) for QR models, overlaid with a horizontal red dashed line indicating the RMSE of the OLS model. The blue line connects the RMSE values for QR at $\tau = 0.25$, 0.5, and 0.75, showing only minor fluctuation among these values. The dashed red line remains flat across the x-axis because the OLS model does not vary by quantile and serves as a **benchmark**.

What stands out in this plot is the visible gap between the red dashed line (OLS) and the blue QR points, reinforcing the earlier barplots: ***OLS consistently achieves a much lower RMSE***. The legend

helps distinguish between QR (blue solid line and dots) and OLS (red dashed line), and the annotated text further emphasizes the OLS value for easier interpretation.

The conclusion remains consistent: Across all visualizations,: the OLS model delivers substantially better forecast performance in terms of RMSE compared to QR models at any quantile level. While quantile regression offers more detailed insights into the distribution of potential outcomes, it is less effective for minimizing squared forecast errors in this context. The combination of bar and line plots provides both an intuitive and quantitative understanding of model performance differences.

3.1.8 Diebold-Mariano Test

The Diebold-Mariano (DM) test provides a formal statistical framework to compare the predictive accuracy of two competing models. In this context, it is used to assess whether the forecast errors of the Ordinary Least Squares (OLS) model differ significantly from those of the Quantile Regression (QR) models at different quantile levels, specifically, $\tau = 0.25$, 0.5 (**the median**), and 0.75. The null hypothesis of the test states that the expected difference in loss (typically measured as squared forecast errors) between the two models is zero, implying equal predictive accuracy.

The DM test was applied using a ***one-step-ahead forecast horizon*** (h = 1) and a ***quadratic loss function*** (power = 2), which penalizes larger errors more heavily. For each comparison between the OLS model and a QR model, the test calculates a p-value indicating the likelihood that the observed differences in prediction errors could have occurred by chance.

	τ	P-value OLS vs. QR	Significance $\alpha = 0.05$
1	OLS (ref.)	NA	
0.25	0.25	0.01114	YES
0.5	0.5	0.84400	NO
0.75	0.75	0.00032	YES

Table 3.4: Comparison of OLS and QR in terms of significance at different tau values.

Initially, the comparison was conducted using QR at **the median level** ($\tau = 0.5$).
The DM statistic for this comparison was:

- The DM statistic was **-0.19703**
- a p-value of **0.844**.

This high p-value indicates that there is no statistically significant difference in forecast accuracy between the OLS model and the QR model at the median quantile. ***In conclusion***:the null hypothesis cannot be rejected, leading to the conclusion that both models perform similarly in terms of predictive accuracy when predicting the conditional median of the response variable.

To further investigate the performance of QR across different quantile levels, the DM test was extended to include $\tau = 0.25$ and $\tau = 0.75$ in addition to $\tau = 0.5$. The forecast errors from the OLS model were compared against those from each QR model. The results were stored and analyzed collectively.

For $\tau = 0.75$, the DM test yielded:

- a DM statistic of -3.663
- a p-value of 0.0003166

which is highly significant at the 5% significance level ($\alpha = 0.05$). This strongly suggests that the QR model at $\tau = 0.75$ provides significantly better forecasts than the OLS model, particularly in capturing lower quantiles (i.e., the left tail of the distribution). This result implies that **Quantile Regression is especially effective at modeling lower-end outcomes**, where OLS might fail to capture relevant distributional asymmetries.

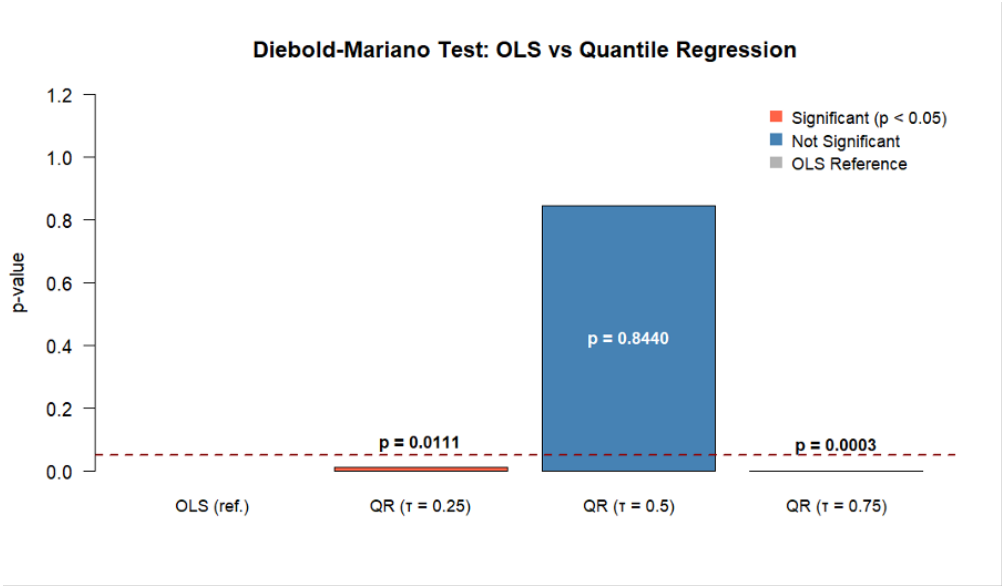

Figure 3.12: DM test: OLS vs QR

Result: are visually summarized in a barplot, where each bar represents a model comparison. The OLS model is used as the reference and is shown in gray. The QR models are represented using colored bars, either steelblue for non-significant results ($p-value \geq 0.05$) or tomato red for statistically significant results ($p-value < 0.05$). A dashed red line at the 0.05 level marks the conventional threshold for significance. The p-values are also labeled directly above each bar for clarity.

In conclusion, the DM test confirms that Quantile Regression offers statistically significant advantages over OLS at the lower and upper quantiles ($\tau = 0.25$ and $\tau = 0.75$), making it a more robust choice when modeling the extremes of the distribution.

3.2 Analysis FRED-MD dataset

3.2.1 Introduction

In empirical economic analysis, understanding how independent variables influence not just the mean, but the entire distribution of a dependent variable is essential. Traditional linear regression methods, such as **Ordinary Least Squares (OLS)**, estimate the conditional mean of the outcome variable. However, this approach may overlook important distributional heterogeneity. **Quantile Regression (QR)** extends the classical regression framework by estimating conditional quantiles of the response variable. This makes it especially useful in labor economics, where wage determinants often have different effects across income levels. This report explores and compares linear and quantile regression models using the Wage dataset from the ISLR package in R. The analysis focuses on the relationship between age, job class, and wage outcomes across the wage distribution.

3.2.2 Data and Setup

The analysis begins by installing and loading a range of R packages necessary for the procedures:

- ISLR for accessing the Wage dataset
- quantreg for performing quantile regression
- visualization packages such as sjPlot and ggplot2.

The forecast package is also included for potential forecasting tests, like the ***Diebold-Mariano test***. After loading the required packages, the contents of the Wage dataset are explored to understand the available variables.

A linear regression model is then constructed using the lm() function, where wage is regressed on age and job class. This model serves as a benchmark against which the quantile regression models will be compared.

3.2.3 Quantile regression

To investigate the heterogeneity in the relationship between predictors and wages, quantile regression models are estimated at three distinct quantile levels:

- $\tau = 0.1$: ***low-income earners***
- $\tau = 0.5$: ***median wage earners***
- $\tau = 0.9$: ***high-income earners***

The rq() function from the quantreg package is utilized to perform this analysis. Age and job class are selected as independent variables, enabling an assessment of how these characteristics influence wages across different segments of the income distribution.

To facilitate comparison of the coefficients obtained from the linear and quantile regression models, the ***plot_models()*** function from the ***sjPlot*** package is employed. This visualization incorporates all four models, linear regression and quantile regressions at $\tau = 0.1$, 0.5, and 0.9, and presents the estimated coefficients alongside confidence intervals and significance indicators for p-values. This comparative plot provides a clear overview of how the effects of predictors vary across the wage distribution.

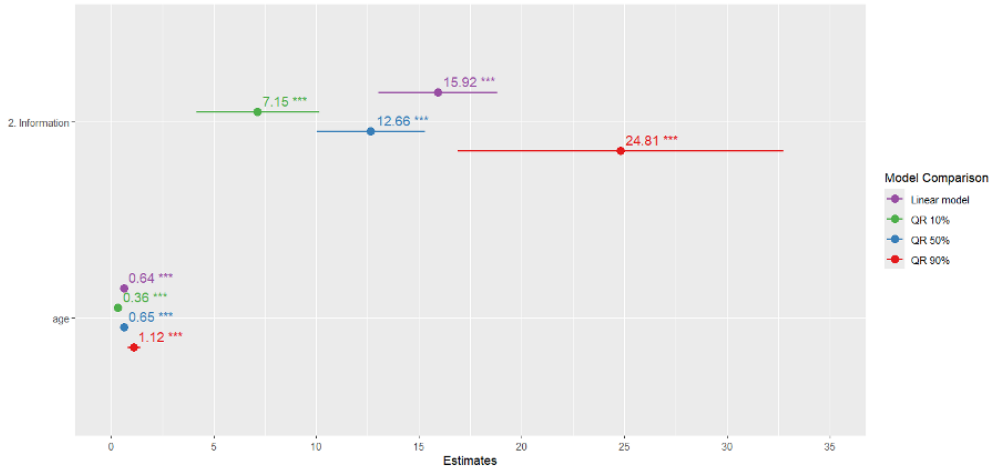

Figure 3.13: comparison of Coefficients

From the plot, it is evident that the effect of age varies significantly across quantiles. At lower quantiles (e.g., $\tau = 0.1$), age has a relatively small effect on wage, suggesting that age does not substantially affect wages for the lowest earners. However, at higher quantiles (e.g., $\tau = 0.9$), age has a much larger positive effect, indicating that more experienced workers benefit more at the upper end of the wage distribution.

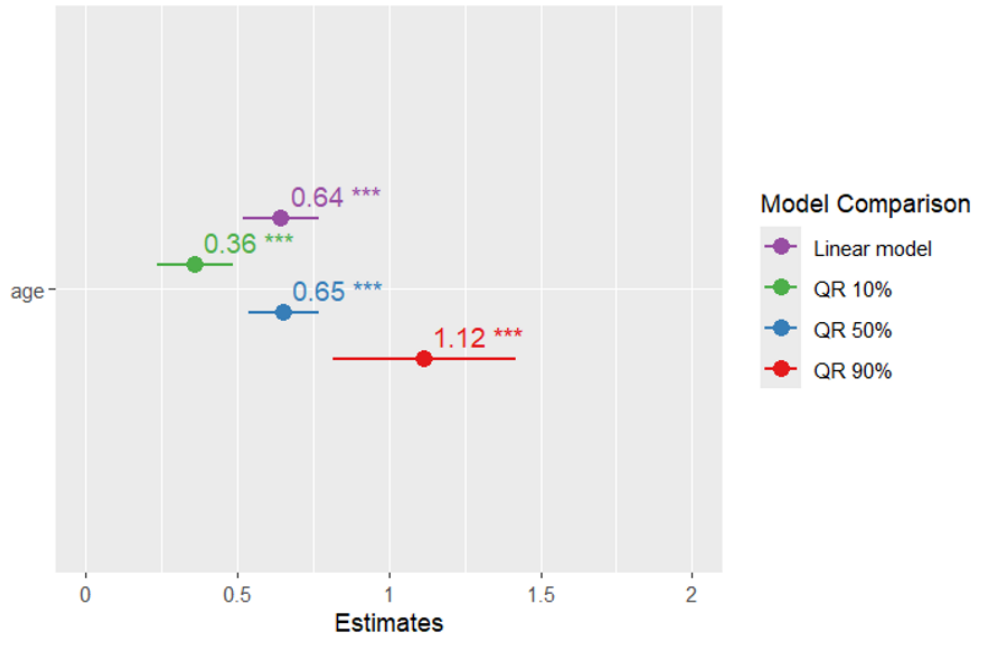

Figure 3.14: Coefficient estimates without dummy variables

The visualization is further refined by removing the dummy variable corresponding to the "Information" category in job class through the use of the ***rm.terms*** argument. This provides a cleaner comparison of coefficients without redundant information. The plot also shows that job class has a significant effect at certain quantiles, emphasizing its role in wage determination.

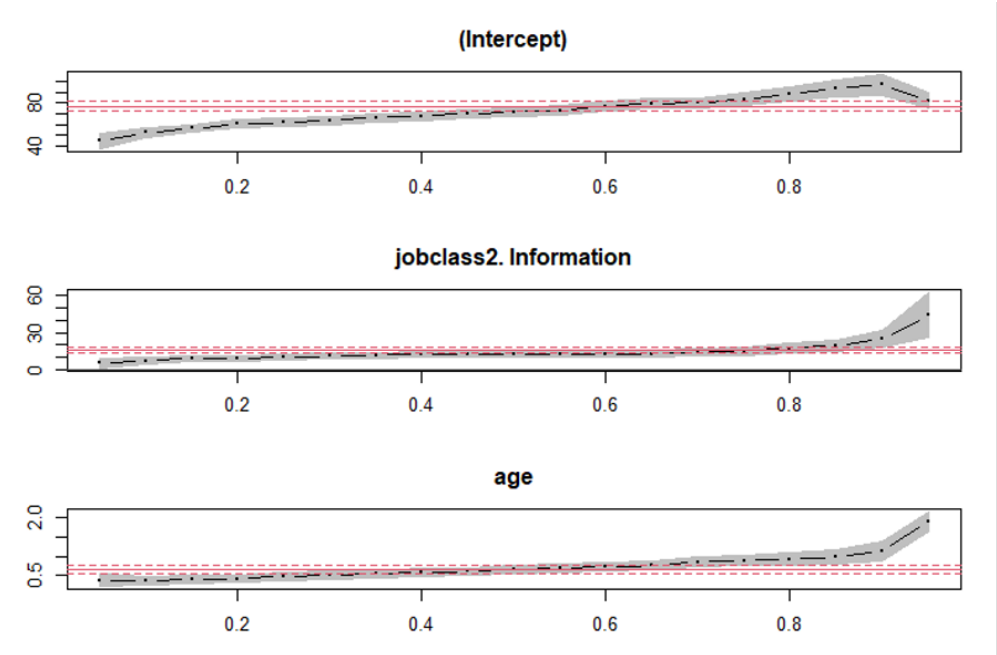

Figure 3.15: Quantile Regression Process Plot

The final plot generated using ***summary(qr_all)*** % > % ***plot()*** is known as a Quantile Regression Process Plot. This graph illustrates how the estimated coefficients for each predictor in the model, such as

age and jobclass, change across different quantiles of the wage distribution, ranging from the 5^{th} percentile ($= 0.05$) to the 95^{th} percentile ($= 0.95$). Unlike standard linear regression, which provides a single average effect, quantile regression allows us to explore how predictors influence different points of the outcome distribution, providing a more complete picture of the relationship between variables.

In the plot, the x-axis represents the quantile values (τ), while the y-axis shows the estimated coefficients for each predictor at those quantiles. Each line corresponds to a different predictor and shows how its effect on wages changes across the income spectrum. The shaded areas around each line are 95% confidence intervals, which help to assess the statistical significance of the coefficients. **If the confidence interval for a coefficient does not include zero at a given quantile, it suggests that the effect is statistically significant at that level**.

A key insight from the plot typically comes from observing the behavior of the coefficient for age. It may appear relatively flat or insignificant at lower quantiles, indicating that age has a minimal impact on wages for low earners. However, as the quantile increases, the coefficient for age might become larger and more significant, suggesting that age (and by implication, experience) has a greater impact on wages among higher earners. This trend reveals that age-related wage gains are not evenly distributed and are more pronounced at the upper ends of the income distribution.

Similarly, the coefficient for jobclass, particularly when comparing information-based jobs to industrial ones, can highlight important differences. If the jobclass coefficient remains consistently positive and significant across most quantiles, this suggests that being in the "Information" job class is associated with a persistent wage premium regardless of where the worker falls in the wage distribution. However, variations in the coefficient of magnitude across quantiles may also indicate that **the benefit of working in certain job classes is more substantial for some income groups than others**.

Overall, this plot is valuable because it exposes heterogeneity in the effects of predictors across the wage distribution, patterns that would remain hidden in a standard linear regression model. By capturing these nuances, the quantile regression process plot offers deeper insight into how different variables influence wages at different levels, enriching the overall analysis and leading to more targeted economic interpretations.

3.2.4 At the end of the story

This analysis demonstrates the value of quantile regression as a tool for understanding the full distributional impact of explanatory variables on an outcome of interest. While linear regression offers a single average estimate, quantile regression reveals that predictors like age and job class have varying impacts at different points in the wage distribution. Specifically, age is more influential for high earners, whereas its effect is negligible for low-income workers. This nuanced insight is crucial for policymakers and economists looking to understand inequality, wage dynamics, and the structure of the labor market. In general, quantile regression provides a more detailed and informative perspective on economic relationships than mean-based approaches alone.

Conclusion

This report provides a comprehensive examination of quantile-based estimation techniques in linear regression, contrasting them with traditional **Ordinary Least Squares (OLS)** methods through both theoretical simulations and empirical applications. The study effectively highlights the robustness and utility of quantile regression, particularly in scenarios with non-standard error distributions.

The first part of the report utilized Monte Carlo simulations to assess various quantile-based slope estimators under several error distributions, including ***Normal***, ***Laplace***, ***Cauchy***, ***Skew-Normal***, and ***t-distributions***. The findings from these simulations consistently demonstrated that while all tested quantile-based methods (Symmetrical Weights, Koenker-Type Weights, and Composite Quantile Regression) are robust, their relative performance varies with the error structure. For instance, the estimator with symmetrical weights (**Criterion 1**) performed ***best*** under heavy-tailed distributions like Cauchy and t-student, while Koenker-type weights (**Criterion 2**) and CQR showed slight advantages in skewed distributions. This underscores the importance of considering the underlying data characteristics when selecting an estimation strategy.

The second part shifted to empirical applications, first by analyzing the growth rate of CO_2 emissions and then by examining wage determinants using the FRED-MD and ISLR datasets, respectively. In the CO_2 analysis, while OLS produced a lower Root Mean Squared Error (RMSE) in out-of-sample forecasting, the Diebold-Mariano test revealed that quantile regression models provided statistically significant improvements in predictive accuracy at the upper and lower quantiles ($\tau = 0.25$ and $\tau = 0.75$). This suggests that quantile regression is more adept at capturing the dynamics at the extremes of the CO_2 growth distribution.

The analysis of wage data further solidified the value of quantile regression. It revealed that the effects of predictors such as age have significantly different impacts across the wage distribution, a nuance entirely missed by the single average estimate provided by OLS. Specifically, age and experience proved more influential for high earners than for those at the lower end of the income scale.

In conclusion, this report successfully demonstrates that **quantile-based estimators offer a flexible and robust alternative to OLS**, providing deeper insights into the relationships within data, especially in the presence of outliers, heteroscedasticity, and skewed distributions. Through rigorous simulations and practical examples, the study makes a compelling case for the adoption of quantile regression in empirical economic and financial analysis to uncover distributional heterogeneities that are often of primary interest.

Prepared by

Khan Balawal *269612*

Osifcinova Theodorika *263672*

Sava Andreea Alexandra *264054*

References

- **Alpharithms – Simple Linear Regression Modeling**
  https://www.alpharithms.com/simple-linear-regression-modeling-502111/
- **MathWorld – Laplace Distribution**
  https://mathworld.wolfram.com/LaplaceDistribution.html
- **MathWorld – Cauchy Distribution**
  https://mathworld.wolfram.com/CauchyDistribution.html
- **Koenker, R. – Quantile Regression (Lecture Notes)**
  http://www.econ.uiuc.edu/~roger/research/intro/rq3.pdf
- **Aptech – The Basics of Quantile Regression**
  https://www.aptech.com/blog/the-basics-of-quantile-regression/
- **University of Virginia – Getting Started with Quantile Regression**
  https://library.virginia.edu
- **EPA – Basic Analyses Overview**
  https://www.epa.gov/caddis/basic-analyses
- **scikit-learn – Quantile Regression Example**
  https://scikit-learn.org
- **Alfasoft – Quantile Regression: Flexible Alternative**
  https://alfasoft.com
- **ScienceDirect – Quantile Regression Topic Overview**
  https://www.sciencedirect.com
- **R-bloggers – Quantile Regression in R**
  https://www.r-bloggers.com
- **CRAN – quantreg Package Vignette**
  https://cran.r-project.org/web/packages/quantreg/vignettes/rq.pdf
- **RPubs – Quantile Regression Intro by ibn_abdullah**
  https://rpubs.com/ibn_abdullah/rquantile
- **RPubs – Quantile Regression in Practice by Karolina Szczesna**
  https://rpubs.com/KarolinaSzczesna/862710
- **STHDA – PCA in R: prcomp vs princomp**
  https://www.sthda.com
- **R Project – Official Website**
  https://www.r-project.org
- **STHDA (FR) – Boxplot con ggplot2**
  https://www.sthda.com

R Code

How to do QR ?:

```r
#HOW TO DO QUANTILE REGRESSION ?
#Let's see it step by step here

#First of all we need to install -> "quantreg package"
#This packages allows us to perform easily the QR-quantile regression
install.packages("quantreg")
library(quantreg)

#Set the diractory -> in simple terms we are loading our data_set located in our pc
setwd("/Users/balawalkhan/Desktop/econometrics project ")
my_data = read.csv("CO2_SA.csv")#we use the "read.csv()" in order to import the data_set

#Let's check if we have upload the file correctly :
View(my_data)#this function shows all data_set
head(my_data)#this function shows historichal data
tail(my_data)#this function shows recent data

#At this point we have to clean the data -> "DATA CLEANNING PROCESS"
my_data = na.omit(my_data)#we remove the NAs values
View(my_data)#check

outliers = boxplot(my_data$GrowthRate, main = "Boxplot CO2_emission", col = "blue")#("   ") -> OUTLIERS(values that we don't want)
#Details on boxplot :
#1.the central line gives us the "MEDIAN" values
#2.Upper line -> 3rd Quartile that we indicate with Q3
#3.Lower line -> 1st Quartile that we indicate with Q1
outliers$out#we see all outliers (anomal values) that we have in the boxplot (given by the small circles)
#NOTE : Quantile regression is robust to outliers ,so, we can safely leave them !



#Recall the variables as :
Dependent_variable_Y = my_data$GrowthRate
Independent_variable_X = my_data$Value
summary(Dependent_variable_Y)
summary(Independent_variable_X)


#QUANTILE REGRESSION IN PRACTICE:
```

```r
#Instead of using the function "lm()-Linear Model" we will use -> "rq()-Regression Quantile"
#tau = c(0.25, 0.5, 0.75)
quantile25 = rq(Dependent_variable_Y ~ Independent_variable_X, data = my_data, tau = 0.25)
summary(quantile25)#summary with CI
summary(quantile25, se = "nid")#classical summary with p-value;st.err...


#To have a better understand let's see it on the Scatter Plot :
plot(Independent_variable_X, Dependent_variable_Y,
     main = "Quantile Regression (tau = 0.25)",
     xlab = "Independent Variable X",
     ylab = "Dependent Variable Y",
     pch = 20, col = "gray")

#Let's add the quantile regression line to the 25th percentile
#A quantile tau = 0.25 corresponds exactly to the 25th percentile
abline(quantile25, col = "blue", lwd = 2)#note "lwd-line width" can be (1(Thin line), 2(thicker), 3( more thicker) )

#In order to give a better data interpretation we can also include in the "LEGEND" in our plot -> OPTIONAL POINT
legend("topleft", legend = "Quantile 25%", col = "blue", lty = 1, lwd = 2)



#But if we want to have ALL QUANTILES IN A SINGLE GRAPH we will use a vector :
taus = c(0.25, 0.5, 0.75)
#having done this we will proceed with the regression using "rq()" ad before for 25th percentile
quantiles <- rq(Dependent_variable_Y ~ Independent_variable_X, data = my_data, tau = taus)
summary(quantiles, se = "nid")
plot(quantiles)#simple plot in order to see what the OLS method cannot capture
#this plot shows that the effect of X (coefficient) changes between the 25th, 50th and 75th percentile.
#and this is an important implication because  the OLS (Ordinary Least Squares) assumes that the effect of X on Y is constant throughout the distribution of Y.

#In order to analyze the diferrences among the coefficients we use the ANOVA TEST :
anova(quantiles)#You are checking whether the coefficients (gradients) estimated by quantile regression are equal for all quantiles (0.25, 0.5, 0.75).
#we observe a P-value(0.3024) > Significance Level(0.05) with (1-p) =0.95 -> NO STATISTICAL SIGNIFICANCE to say that the coefficients are different between the quantiles


#as before we plot them:
plot(Independent_variable_X, Dependent_variable_Y,
     main = "Quantile Regression",
     xlab = "X",
     ylab = "Y",
     pch = 20,
     col = "gray")

#Regressions line for each quantile with different colors
colors <- c("blue", "red", "green")
```

```r

#In this case we need to extract each tau in order to trace the regression using the cicle "for()"
coeffs <- coef(quantiles)
#trace each line
for (i in 1:length(taus)) {
  intercept <- coeffs[1, i]
  slope <- coeffs[2, i]
  abline(a = intercept, b = slope, col = colors[i], lwd = 2)
}

#also here we can add the legend ,since it is optional
legend("topleft",
       legend = c("tau = 0.25", "tau = 0.5", "tau = 0.75"),
       col = colors, lty = 1, lwd = 2)




#After all these passages we have to calculate the weights as :
# Step 1: Compute weights proportional to 1 - |2   - 1|
raw_weights = 1 - abs(2 * taus - 1)
weights =  raw_weights / sum(raw_weights)  # normalize so that sum = 1
print(weights)

#Now still for the 1st Criterion we calculate the aggregate Beta as :
#Step 2: Compute weighted average of coefficients
beta_agg =  coeffs %*% weights  # matrix multiplication
print(beta_agg)

#Let's cross now to Step 2 apllying the 2nd Criterion :
#NUMERATOR :
#Step 1 :Extract phi^-1 from N(0, 1) by using the function : "qnorm()"
print(list(qnorm(taus)))
#Step 2 : calculate the Density Function -> "dnorm(qnorm_result)"
print(list(dnorm(qnorm(taus))))
#We create a new variable in order to simplify the calculations then :
numerator =  dnorm(qnorm(taus))

#DENOMINATOR will be :
denominator <- sqrt(taus * (1 - taus))

#At this point we are ready to calculate the 2nd weight :
weight_2 = numerator / denominator
print(weight_2)
#As before we have to NORMALIZE them
weights_2_normalized <- weight_2/ sum(weight_2)
print(weights_2_normalized)


#Also here we compute the aggregate Beta :
aggregate_beta <- coeffs %*% weight_2
print(aggregate_beta)

#IMPORTANT NOTE : The operator %*% is already a weighted sum, so we do not use "sum()"



```

```

#Reached this point, we have to work with the COMPOSITE QUANTILE REGRESSION :
#we need the following library in order to use -> "cqr.fit()"
library(cqrReg)

#The model that we use is : "Grothwrate = Beta(0) + Beta(1)*Value  + u(t)"
#The CQR uses more quantiles at the same time to improve robustness and efficiency
    .
#Let's do the regression :
model_cqr = cqr.fit(Dependent_variable_Y ~ Independent_variable_X,  tau = taus,
    method = "Nelder-Mead")
#Since cqr.fit gives us PROBLEMS -> NULL VALUE
#We adopt another strategy using the function  "rq()"
model_rq = rq(Dependent_variable_Y ~ Independent_variable_X,  tau = taus)
summary(model_rq, se = "nid")

coeffs_rq = coef(model_rq)
#Average of the coefficients between the 3 quantiles :
beta_cqr_approx <- rowMeans(coeffs)
print(beta_cqr_approx)


#COMPARION :
data_comparison = data_frame(Criterion = c("Weight_1", "Weight_2", "APPROX_CQR"),
    INTERCEPT = c(-0.0143, -0.0162, -0.0164), SLOPE = c(0.0000325, 0.0000389,
    0.0000394))
View(data_comparison)
```

QR + weights calculation:

```

#HOW TO DO QUANTILE REGRESSION ?
#Let's see it step by step here

#First of all we need to install -> "quantreg package"
#This packages allows us to perform easily the QR-quantile regression
install.packages("quantreg")
library(quantreg)

#Set the directory -> in simple terms we are loading our data_set located in our
    pc
setwd("/Users/balawalkhan/Desktop/econometrics project ")
my_data = read.csv("CO2_SA.csv")#we use the "read.csv()" in order to import the
    data_set

#Let's check if we have upload the file correctly :
View(my_data)#this function shows all data_set
head(my_data)#this function shows historical data
tail(my_data)#this function shows recent data

#At this point we have to clean the data -> "DATA CLEANNING PROCESS"
my_data = na.omit(my_data)#we remove the NAs values
View(my_data)#check

outliers = boxplot(my_data$GrowthRate, main = "Boxplot CO2_emission", col = "blue"
    )#("   ") -> OUTLIERS(values that we don't want)
#Details on boxplot :
#1.the central line gives us the "MEDIAN" values
```

```r
#2.Upper line -> 3rd Quartile that we indicate with Q3
#3.Lower line -> 1st Quartile that we indicate with Q1
outliers$out#we see all outliers (anomal values) that we have in the boxplot (
    given by the small circles)
#NOTE : Quantile regression is robust to outliers ,so, we can safely leave them !



#Recall the variables as :
Dependent_variable_Y = my_data$GrowthRate
Independent_variable_X = my_data$Value
summary(Dependent_variable_Y)
summary(Independent_variable_X)


#QUANTILE REGRESSION IN PRACTICE:
#Instead of using the function "lm()-Linear Model" we will use -> "rq()-Regression
    Quantile"
#tau = c(0.25, 0.5, 0.75)
quantile25 = rq(Dependent_variable_Y ~ Independent_variable_X, data = my_data, tau
    = 0.25)
summary(quantile25)#summary with CI
summary(quantile25, se = "nid")#classical summary with p-value;st.err...


#To have a better understand let's see it on the Scatter Plot :
plot(Independent_variable_X, Dependent_variable_Y,
     main = "Quantile Regression (tau = 0.25)",
     xlab = "Independent Variable X",
     ylab = "Dependent Variable Y",
     pch = 20, col = "gray")

#Let's add the quantile regression line to the 25th percentile
#A quantile tau = 0.25 corresponds exactly to the 25th percentile
abline(quantile25, col = "blue", lwd = 2)#note "lwd-line width" can be (1(Thin
    line), 2(thicker), 3( more thicker) )

#In order to give a better data interpretation we can also include in the "LEGEND"
    in our plot -> OPTIONAL POINT
legend("topleft", legend = "Quantile 25%", col = "blue", lty = 1, lwd = 2)



#But if we want to have ALL QUANTILES IN A SINGLE GRAPH we will use a vector :
taus = c(0.25, 0.5, 0.75)
#having done this we will proceed with the regression using "rq()" ad before for
    25th percentile
quantiles <- rq(Dependent_variable_Y ~ Independent_variable_X, data = my_data, tau
    = taus)
summary(quantiles, se = "nid")
plot(quantiles)#simple plot in order to see what the OLS method cannot capture
#this plot shows that the effect of X (coefficient) changes between the 25th, 50th
    and 75th percentile.
#and this is an important implication because  the OLS (Ordinary Least Squares)
    assumes that the effect of X on Y is constant throughout the distribution of Y.
plot(summary(quantiles, se ="nid"))
#In order to analyze the diferrences among the coefficients we use the ANOVA TEST
    :
```

```r
anova(quantiles)#You are checking whether the coefficients (gradients) estimated by quantile regression are equal for all quantiles (0.25, 0.5, 0.75).
#we observe a P-value(0.3024) > Significance Level(0.05) with (1-p) =0.95 -> NO STATISTICAL SIGNIFICANCE to say that the coefficients are different between the quantiles


#as before we plot them:
plot(Independent_variable_X, Dependent_variable_Y,
     main = "Quantile Regression",
     xlab = "X",
     ylab = "Y",
     pch = 20,
     col = "gray")

#Regressions line for each quantile with different colors
colors <- c("blue", "red", "green")

#In this case we need to extract each tau in order to trace the regression using the cicle "for()"
coeffs <- coef(quantiles)
#trace each line
for (i in 1:length(taus)) {
  intercept <- coeffs[1, i]
  slope <- coeffs[2, i]
  abline(a = intercept, b = slope, col = colors[i], lwd = 2)
}

#also here we can add the legend ,since it is optional
legend("topleft",
       legend = c("tau = 0.25", "tau = 0.5", "tau = 0.75"),
       col = colors, lty = 1, lwd = 2)




#After all these passages we have to calculate the weights as :
# Step 1: Compute weights proportional to 1 - |2   - 1|
raw_weights = 1 - abs(2 * taus - 1)
weights =  raw_weights / sum(raw_weights)  # normalize so that sum = 1
print(weights)

#Now still for the 1st Criterion we calculate the aggregate Beta as :
#Step 2: Compute weighted average of coefficients
beta_agg =  coeffs %*% weights  # matrix multiplication
print(beta_agg)

#Let's cross now to Step 2 applying the 2nd Criterion :
#NUMERATOR :
#Step 1 :Extract phi^-1 from N(0, 1) by using the function : "qnorm()"
print(qnorm(taus))
#Step 2 : calculate the Density Function -> "dnorm(qnorm_result)"
print(dnorm(qnorm(taus)))
#We create a new variable in order to simplify the calculations then :
numerator =  dnorm(qnorm(taus))

#DENOMINATOR will be :
denominator <- sqrt(taus * (1 - taus))

```

```r
#At this point we are ready to calculate the 2nd weight :
weight_2 = numerator / denominator
print(weight_2)
#As before we have to NORMALIZE them
weights_2_normalized <- weight_2/ sum(weight_2)
print(weights_2_normalized)


#Also here we compute the aggregate Beta :
aggregate_beta <- coeffs %*% weight_2
print(aggregate_beta)

#IMPORTANT NOTE : The operator %*% is already a weighted sum, so we do not use "sum()"




#Reached this point, we have to work with the COMPOSITE QUANTILE REGRESSION(CQR):
#we need the following library in order to use -> "cqr.fit()"
library(cqrReg)

#The model that we use is : "Grothwrate = Beta(0) + Beta(1)*Value  + u(t)"
#The CQR uses more quantiles at the same time to improve robustness and efficiency.
#Let's do the regression :
model_cqr = cqr.fit(Dependent_variable_Y ~ Independent_variable_X,  tau = taus, method = "Nelder-Mead")
#Since cqr.fit gives us PROBLEMS -> NULL VALUE
#We adopt another strategy using the function  "rq()"
model_rq = rq(Dependent_variable_Y ~ Independent_variable_X,  tau = taus)
summary(model_rq, se = "nid")

coeffs_rq = coef(model_rq)
#Average of the coefficients between the 3 quantiles :
beta_cqr_approx <- rowMeans(coeffs)
print(beta_cqr_approx)


#COMPARION :
data_comparison = data.frame(Criterion = c("Weight_1", "Weight_2", "APPROX_CQR"), INTERCEPT = c(-1.431442e-02, -3.660301e-02, -1.639037e-02 ), SLOPE = c(3.509606e-05, 8.810474e-05, 3.937057e-05 ))
View(data_comparison)
```

Monte Carlo Simulation with Normal Error:

```r
    #MONTE CARLO SIMULATION
library(quantreg)

#we are going to set the needed parameters
N=1000#n   of Simulations
n=100#n of observations
set.seed(123)#in order to reproduce with same results

#parameter of simple regression model
Alpha = 0
```

```r
Beta = 1

x <- 1:n #it's a fixed vector

#now we have to save the data
beta1_crit1_norm <- numeric(N)
beta1_crit2_norm <- numeric(N)
beta1_cqr_norm   <- numeric(N)

#calculation that we have already done
tau = c(0.25, 0.5, 0.75)#our quantiles
weights_crit1 = c(0.25, 0.5, 0.25)#normalized weights using the 1st criterion
weights_crit2 = c( 0.3239157, 0.3521687, 0.3239157)#normalized weights using the 2
    nd criterion

#Core of the Monte Carlo Simulation -> for()

for (i in 1:N) {
  error_term <- rnorm(n)#we extract normal errors from a Normal Distribution
  y <- Alpha + Beta * x + error_term

  data_temp <- data.frame(x = x, y = y)#we create this data_frame in order to use
      the function "rq()"

  beta_list <- lapply(tau, function(tau_k) {
    coef(rq(y ~ x, data = data_temp, tau = tau_k))
  })#for each quantile (tau) we estimate the QR-quantile regression

  beta_matrix <- do.call(rbind, beta_list)#then we build a matrix 3x2 in which we
      have the applied method

  beta_crit1_norm <- colSums(beta_matrix * weights_crit1)
  beta1_crit1_norm[i] <- beta_crit1_norm[2]

  beta_crit2_norm <- colSums(beta_matrix * weights_crit2)
  beta1_crit2_norm[i] <- beta_crit2_norm[2]

  beta_cqr_norm <- colMeans(beta_matrix)
  beta1_cqr_norm[i] <- beta_cqr_norm[2]
}

#We calculate now the ME for each method
ME_crit1_norm <- mean((beta1_crit1_norm - Beta)^2)
ME_crit2_norm <- mean((beta1_crit2_norm - Beta)^2)
ME_cqr_norm   <- mean((beta1_cqr_norm - Beta)^2)

#Results that we have to compare
ME_table_norm <- data.frame(
  Method = c("Criterion 1", "Criterion 2", "CQR (mean)"),
  ME_norm = c(ME_crit1_norm, ME_crit2_norm, ME_cqr_norm)
)

print(ME_table_norm)

#Let's give first the TABLE RAPRESENTATION:
#Calculate the variances :
var_crit1 <- var(beta1_crit1_norm)
var_crit2 <- var(beta1_crit2_norm)
var_cqr   <- var(beta1_cqr_norm)
```

```r


Var_table <- data.frame(
  Method = c("Criterion 1", "Criterion 2", "CQR (mean)"),
  Variance = c(var_crit1, var_crit2, var_cqr)
)
print(Var_table)









#We will use a BOXPLOT to have a better idea of what we have done
cols = c("lightblue", "lightgreen", "lightcoral")
barplot(
  height = ME_table_norm$ME_norm,
  names.arg = ME_table_norm$Method,
  col = cols,
  main = "Normal's error",
  ylab = "Mean Squared Error",
  ylim = c(0, max(ME_table_norm$ME_norm) * 1.2)
)


boxplot(
  beta1_crit1_norm,
  beta1_crit2_norm,
  beta1_cqr_norm,
  names = c("Criterion 1", "Criterion 2", "CQR"),
  col = c("lightblue", "lightgreen", "lightcoral"),
  main = "Distribution of      ",
  ylab = "Estimated      "
)
abline(h = 1, col = "black", lty = 2)
```

Laplace Distribution:

```r

#MONTE CARLO SIMULATION : LAPLACE ERROR
library(quantreg)
install.packages("rmutil")#here we need this type of package
library(rmutil)
install.packages("VGAM")
library(VGAM)

#we are going to set the needed parameters
N=1000#n   of Simulations
n=100#n of observations
set.seed(123)#in order to reproduce with same results

#parameter of simple regression model(same process as before for Normal error)
Alpha = 0
```

```r
Beta = 1
x <- 1:n #it's a fixed vector

#Les's use still the cicle "for()"-> CORE OF MONTE CARLO SIMULATION
#With respect the Normal Distribution Laplace Distribution it is more SENSITIVE TO OUTLIERS
#Before extracting the Laplace error's and lunching the cicle for(),let's give firt the difference between the two distributions

#Set the Laplace Density Function :
laplace_density_function <- function(x, mu = 0, b = 1) {
  return(1/(2*b) * exp(-abs(x - mu)/b))
}


#We have that x     (      ,+   ) for both distributions.
#In practice we choose the range [ 10,10] to represent densities since, for both distributions Normal(0,1) and Laplace(0,1), more than 99.9% of the mass of probability falls within this range. This ensures a complete and effective representation of the tail behaviour.
#P( 3  .29   Z3   .29) 0  .999 (quantiles for the normal)
x_range <- seq(-10, 10, 0.01)# constant increase of 0.01

#Plot first Laplace Distribution :
plot(x_range,
     rmutil::dlaplace(x_range, m = 0, s = 1),
     type = "l",
     col = "red",
     lty = 2,
     lwd = 2,
     ylab = "Density",
     xlab = "x",
     main = "Laplace vs Normal")


#Now we add to the plot the Normal Distribution :
lines(x_range, dnorm(x_range, mean = 0, sd = 1), col = "blue", lwd = 2)
legend("topright",
       legend = c("Laplace(0,1)", "Normale(0,1)"),
       col = c("red", "blue"),
       lty = c(2, 1),
       lwd = 2,
       cex = 0.5)
#The result of this comparison is that Laplace Distribution has heavy tails compared to the Normal Distribution.

#in order to a better idea of this type of distribution let's plot it's 3D Version :
x <- seq(-4, 4, length.out = 60)
y <- seq(-4, 4, length.out = 60)


z <- outer(x, y, function(x, y) {
  rmutil::dlaplace(x, m = 0, s = 1) * rmutil::dlaplace(y, m = 0, s = 1)
})#it allows us to build the density of the function

par(mar = c(2, 2, 4, 4))

# Plot wireframe 3D
```

```r
persp(x, y, z,
      theta = 40, phi = 15,
      border = "red", col = "white",
      xlab = "x", ylab = "y", zlab = "f(x)",
      ticktype = "detailed", shade = 0,
      lwd = 0.7,
      cex.axis = 0.8,
      main = "Laplace Distribution(0,1)")

#Now we can proceed with the extraction of the errors :
#MANUALLY : For our calcules we will use this one !
rlaplace <- function(n) {
  u <- runif(n, -0.5, 0.5)#Uniform Distribution
  return(-sign(u) * log(1 - 2 * abs(u)))
}#This function generates the L(o,1)

#DIRECT METHOD :
install.packages("rmutil")
library(rmutil)
x <- rmutil::rlaplace(n = 100, m = 0, s = 1)#Explicit form
x <- rlaplace(n, m = 0, s = 1)#Implicit form(just for demonstration)
#where :
#1.n = n   of observations
#2.m = Position parameter -> if m = 0 : Distribution centered in 0 ; if m = (+x or
    -x) -> Distibution on right or left
#3.s = How much the data is dispersed compared to the average.We have set s =1 ->
   the distribution takes the classical form -> "STANDARD LAPLACE"


#Also here we need to save the results in empty vectors
beta1_crit1_lap <- numeric(N)
beta1_crit2_lap <- numeric(N)
beta1_cqr_lap   <- numeric(N)

#Quantiles and weights(normalized) :
tau <- c(0.25, 0.5, 0.75)
weights_crit1 <- c(0.25, 0.5, 0.25)
weights_crit2 <- c(0.3239157, 0.3521687, 0.3239157)

#ALL the following code will be the same as Normal case extraction!
x <- 1:n #recall it
for (i in 1:N) {
  error_term <- rlaplace(n)#it generates Laplace errors
  y <- Alpha + Beta * x + error_term
  data_temp <- data.frame(x = x, y = y)

  beta_list <- lapply(tau, function(tau_k) {
    coef(rq(y ~ x, data = data_temp, tau = tau_k))
  })

  beta_matrix <- do.call(rbind, beta_list)

  beta_crit1_lap <- colSums(beta_matrix * weights_crit1)
  beta1_crit1_lap[i] <- beta_crit1_lap[2]

  beta_crit2_lap <- colSums(beta_matrix * weights_crit2)
  beta1_crit2_lap[i] <- beta_crit2_lap[2]

  beta_cqr_lap <- colMeans(beta_matrix)
```

```
  beta1_cqr_lap[i] <- beta_cqr_lap[2]
}

#Mean Squarred Errors :
ME_crit1_lap <- mean((beta1_crit1_lap - Beta)^2)
ME_crit2_lap <- mean((beta1_crit2_lap - Beta)^2)
ME_cqr_lap   <- mean((beta1_cqr_lap - Beta)^2)

ME_table_lap <- data.frame(
  Method = c("Criterion 1", "Criterion 2", "CQR (mean)"),
  ME_laplace = c(ME_crit1_lap, ME_crit2_lap, ME_cqr_lap)
)
print(ME_table_lap)

#Variances of estimators
Var_table_lap <- data.frame(
  Method = c("Criterion 1", "Criterion 2", "CQR (mean)"),
  Variance_Laplace = c(
    var(beta1_crit1_lap),
    var(beta1_crit2_lap),
    var(beta1_cqr_lap)
  )
)
print(Var_table_lap)

#Graphical visualization of ME
cols <- c("lightblue", "lightgreen", "lightcoral")
barplot(
  height = ME_table_lap$ME_laplace,
  names.arg = ME_table_lap$Method,
  col = cols,
  main = "Laplace's Error",
  ylab = "Mean Squared Error",
  ylim = c(0, max(ME_table_lap$ME_laplace) * 1.2)
)


#Estimates rapresentation
boxplot(
  beta1_crit1_lap,
  beta1_crit2_lap,
  beta1_cqr_lap,
  names = c("Criterion 1", "Criterion 2", "CQR"),
  col = cols,
  main = "Distribution of       (Laplace Errors)",
  ylab = "Estimated      "
)
abline(h = 1, col = "black", lty = 2)
```

Cauchy Distribution:

```
    #This first part is the same as before (Monte Carlo with Normal Errors)
library(quantreg)

set.seed(123)
N <- 1000
n <- 100
```

```r

Alpha <- 0
Beta <- 1

x = 1:n

tau = c(0.25, 0.5, 0.75)

weights_crit1 = c(0.25, 0.5, 0.25)
weights_crit2 = c(0.3240, 0.3522, 0.3239)

beta1_crit1_cauchy <- numeric(N)
beta1_crit2_cauchy <- numeric(N)
beta1_cqr_cauchy  <- numeric(N)


#What changes with respect normarl errors is wrote here.

for (i in 1:N) {
  #We generate Cauchy Errors
  error_term <- rcauchy(n, location = 0, scale = 1)

  #Simulation
  y <- Alpha + Beta * x + error_term

  #Data_set
  data_temp <- data.frame(x = x, y = y)

  #We do QR-quantile regression
  beta_list <- lapply(tau, function(tau_k) {
    suppressWarnings(coef(rq(y ~ x, data = data_temp, tau = tau_k)))
  })

  #Matrix with each coefficient
  beta_matrix <- do.call(rbind, beta_list)

  #Compound Betas
  beta_crit1_cauchy <- colSums(beta_matrix * weights_crit1)
  beta_crit2_cauchy <- colSums(beta_matrix * weights_crit2)
  beta_cqr_cauchy   <- colMeans(beta_matrix)

  #Slopes( 1 )
  beta1_crit1_cauchy[i] <- beta_crit1_cauchy[2]
  beta1_crit2_cauchy[i] <- beta_crit2_cauchy[2]
  beta1_cqr_cauchy[i]   <- beta_cqr_cauchy[2]
}


ME_crit1_cauchy <- mean((beta1_crit1_cauchy - Beta)^2)
ME_crit2_cauchy <- mean((beta1_crit2_cauchy - Beta)^2)
ME_cqr_cauchy   <- mean((beta1_cqr_cauchy - Beta)^2)

#Cauchy Results:
ME_table_cauchy <- data.frame(
  Method = c("Criterion 1", "Criterion 2", "CQR (mean)"),
  ME_cauchy = c(ME_crit1_cauchy, ME_crit2_cauchy, ME_cqr_cauchy)
)

print(ME_table_cauchy)
```

```r

#In order to obtain a better visualization let's use the BOXPLOT

Bar_col = c("lightblue", "lightgreen", "lightcoral")

barplot(
  height = ME_table_cauchy$ME_cauchy,
  names.arg = ME_table_cauchy$Method,
  col = Bar_col,
  main = "Cauchy's error",
  ylab = "Mean Squared Error",
  ylim = c(0, max(ME_table_cauchy$ME_cauchy) * 1.2)
)




#In addition,we are going to give a comparison between the two distribution that we have used
#aggregate data_frame:
df_dens <- data.frame(
  beta = c(beta1_crit1_norm, beta1_crit2_norm, beta1_cqr_norm,
           beta1_crit1_cauchy, beta1_crit2_cauchy, beta1_cqr_cauchy),
  Method = rep(c("Criterion 1", "Criterion 2", "CQR"), each = N * 2),
  Distribution = rep(c("Normal", "Normal", "Normal", "Cauchy", "Cauchy", "Cauchy"), each = N)
)


library(ggplot2)#library for graphical visualization

ggplot(df_dens, aes(x = beta, color = Method, fill = Method)) +
  geom_density(alpha = 0.3, size = 1.1) +
  facet_wrap(~Distribution, scales = "free") +
  labs(
    title = "       for each type of error",
    x = "Estimated value of      (slope)",
    y = "Density"
  ) +
  theme_minimal(base_size = 13) +
  theme(legend.position = "top")
```

t-Student:

```r
     library(quantreg)

N <- 1000
n <- 100
set.seed(123)

Alpha <- 0
Beta <- 1
x <- 1:n

#Save results in empty vectors
beta1_crit1_t <- numeric(N)
beta1_crit2_t <- numeric(N)
```

```r
beta1_cqr_t   <- numeric(N)

#Quantiles and weights
tau <- c(0.25, 0.5, 0.75)
weights_crit1 <- c(0.25, 0.5, 0.25)
weights_crit2 <- c(0.3239157, 0.3521687, 0.3239157)


for (i in 1:N) {
  error_term <- rt(n, df = 3)  #We set df(degree of freedom) = 3
  y <- Alpha + Beta * x + error_term
  data_temp <- data.frame(x = x, y = y)

  beta_list <- lapply(tau, function(tau_k) {
    coef(rq(y ~ x, data = data_temp, tau = tau_k))
  })

  beta_matrix <- do.call(rbind, beta_list)

  beta_crit1_t <- colSums(beta_matrix * weights_crit1)
  beta1_crit1_t[i] <- beta_crit1_t[2]

  beta_crit2_t <- colSums(beta_matrix * weights_crit2)
  beta1_crit2_t[i] <- beta_crit2_t[2]

  beta_cqr_t <- colMeans(beta_matrix)
  beta1_cqr_t[i] <- beta_cqr_t[2]
}


#MSE
ME_crit1_t <- mean((beta1_crit1_t - Beta)^2)
ME_crit2_t <- mean((beta1_crit2_t - Beta)^2)
ME_cqr_t   <- mean((beta1_cqr_t - Beta)^2)

ME_table_t <- data.frame(
  Method = c("Criterion 1", "Criterion 2", "CQR (mean)"),
  MSE_t = c(ME_crit1_t, ME_crit2_t, ME_cqr_t)
)
print(ME_table_t)

#Variances (empirical)
Var_table_t <- data.frame(
  Method = c("Criterion 1", "Criterion 2", "CQR (mean)"),
  Variance_t = c(
    var(beta1_crit1_t),
    var(beta1_crit2_t),
    var(beta1_cqr_t)
  )
)
print(Var_table_t)



cols <- c("lightblue", "lightgreen", "lightcoral")
#Barplot of MSE
barplot(
  height = ME_table_t$MSE_t,
  names.arg = ME_table_t$Method,
```

```
  col = cols,
  main = "t(3) Errors - Mean Squared Error",
  ylab = "MSE",
  ylim = c(0, max(ME_table_t$MSE_t) * 1.2)
)

#Boxplot of estimates
boxplot(
  beta1_crit1_t,
  beta1_crit2_t,
  beta1_cqr_t,
  names = c("Criterion 1", "Criterion 2", "CQR"),
  col = cols,
  main = "Distribution of       (t-distributed Errors)",
  ylab = "Estimated      "
)
abline(h = 1, col = "black", lty = 2)
```

Skew Normal:

```
    install.packages("sn")#For this Distribution with respect quantreg we have to
       use a new library -> "sn"
library(sn)

rsn(n, xi = 0, omega = 1, alpha = 5)#the core function of this distribution

#Now we proceed with the extrapolation of errors as other cases that we have
    already studied
library(quantreg)


N <- 1000
n <- 100
set.seed(123)

Alpha <- 0
Beta <- 1
x <- 1:n

beta1_crit1_skew <- numeric(N)
beta1_crit2_skew <- numeric(N)
beta1_cqr_skew   <- numeric(N)

tau <- c(0.25, 0.5, 0.75)
weights_crit1 <- c(0.25, 0.5, 0.25)
weights_crit2 <- c(0.3239157, 0.3521687, 0.3239157)

for (i in 1:N) {
  error_term <- rsn(n, xi = 0, omega = 1, alpha = 5)#Skew-Normal errors
  y <- Alpha + Beta * x + error_term
  data_temp <- data.frame(x = x, y = y)

  beta_list <- lapply(tau, function(tau_k) {
    coef(rq(y ~ x, data = data_temp, tau = tau_k))
  })

  beta_matrix <- do.call(rbind, beta_list)
```

```r

  beta_crit1_skew <- colSums(beta_matrix * weights_crit1)
  beta1_crit1_skew[i] <- beta_crit1_skew[2]

  beta_crit2_skew <- colSums(beta_matrix * weights_crit2)
  beta1_crit2_skew[i] <- beta_crit2_skew[2]

  beta_cqr_skew <- colMeans(beta_matrix)
  beta1_cqr_skew[i] <- beta_cqr_skew[2]
}



ME_crit1_skew <- mean((beta1_crit1_skew - Beta)^2)
ME_crit2_skew <- mean((beta1_crit2_skew - Beta)^2)
ME_cqr_skew   <- mean((beta1_cqr_skew - Beta)^2)

ME_table_skew <- data.frame(
  Method = c("Criterion 1", "Criterion 2", "CQR (mean)"),
  MSE_skew = c(ME_crit1_skew, ME_crit2_skew, ME_cqr_skew)
)
print(ME_table_skew)

Var_table_skew <- data.frame(
  Method = c("Criterion 1", "Criterion 2", "CQR (mean)"),
  Variance_skew = c(
    var(beta1_crit1_skew),
    var(beta1_crit2_skew),
    var(beta1_cqr_skew)
  )
)
print(Var_table_skew)




cols <- c("lightblue", "lightgreen", "lightcoral")
#Barplot of MSE
barplot(
  height = ME_table_skew$MSE_skew,
  names.arg = ME_table_skew$Method,
  col = cols,
  main = "Skew-Normal Error - MSE",
  ylab = "Mean Squared Error",
  ylim = c(0, max(ME_table_skew$MSE_skew) * 1.2)
)

#Boxplot of estimates
boxplot(
  beta1_crit1_skew,
  beta1_crit2_skew,
  beta1_cqr_skew,
  names = c("Criterion 1", "Criterion 2", "CQR"),
  col = cols,
  main = "Distribution of       (Skew-Normal Errors)",
  ylab = "Estimated      "
)
abline(h = 1, col = "black", lty = 2)
```

Skew t:

```
    library(quantreg)
library(sn)

N <- 1000
n <- 100
set.seed(123)

Alpha <- 0
Beta <- 1
x <- 1:n

beta1_crit1_skewt <- numeric(N)
beta1_crit2_skewt <- numeric(N)
beta1_cqr_skewt   <- numeric(N)

#Quantiles and weights
tau <- c(0.25, 0.5, 0.75)
weights_crit1 <- c(0.25, 0.5, 0.25)
weights_crit2 <- c(0.3239157, 0.3521687, 0.3239157)

for (i in 1:N) {
  error_term <- rst(n, xi = 0, omega = 1, alpha = 5, nu = 5)
  y <- Alpha + Beta * x + error_term
  data_temp <- data.frame(x = x, y = y)

  beta_list <- lapply(tau, function(tau_k) {
    coef(rq(y ~ x, data = data_temp, tau = tau_k))
  })

  beta_matrix <- do.call(rbind, beta_list)

  beta_crit1_skewt <- colSums(beta_matrix * weights_crit1)
  beta1_crit1_skewt[i] <- beta_crit1_skewt[2]

  beta_crit2_skewt <- colSums(beta_matrix * weights_crit2)
  beta1_crit2_skewt[i] <- beta_crit2_skewt[2]

  beta_cqr_skewt <- colMeans(beta_matrix)
  beta1_cqr_skewt[i] <- beta_cqr_skewt[2]
}



ME_crit1_skewt <- mean((beta1_crit1_skewt - Beta)^2)
ME_crit2_skewt <- mean((beta1_crit2_skewt - Beta)^2)
ME_cqr_skewt   <- mean((beta1_cqr_skewt - Beta)^2)

ME_table_skewt <- data.frame(
  Method = c("Criterion 1", "Criterion 2", "CQR (mean)"),
  MSE_skewt = c(ME_crit1_skewt, ME_crit2_skewt, ME_cqr_skewt)
)
print(ME_table_skewt)

Var_table_skewt <- data.frame(
  Method = c("Criterion 1", "Criterion 2", "CQR (mean)"),
  Variance_skewt = c(
```

```
    var(beta1_crit1_skewt),
    var(beta1_crit2_skewt),
    var(beta1_cqr_skewt)
  )
)
print(Var_table_skewt)



cols <- c("lightblue", "lightgreen", "lightcoral")
# MSE
barplot(
  height = ME_table_skewt$MSE_skewt,
  names.arg = ME_table_skewt$Method,
  col = cols,
  main = "Skew-t Error - Mean Squared Error",
  ylab = "MSE",
  ylim = c(0, max(ME_table_skewt$MSE_skewt) * 1.2)
)

#Boxplot of estimate
boxplot(
  beta1_crit1_skewt,
  beta1_crit2_skewt,
  beta1_cqr_skewt,
  names = c("Criterion 1", "Criterion 2", "CQR"),
  col = cols,
  main = "Distribution of       (Skew-t Errors)",
  ylab = "Estimated      "
)
abline(h = 1, col = "black", lty = 2)
```

PART B of the REPORT:

```
    #PART B of CourseWork :


#All packages that are needed
install.packages("ISLR")#it provides A WAGE SAMPLE DATASET (example)
install.packages("tidyverse")
install.packages("quantreg")
install.packages("sjPlot")#Usfull for graphical visualization of our results
install.packages("strengejacke")#in addition to sjPlot package required by RStudio
install.packages("quantmod")
install.packages("dplyr")
install.packages("rstatix")
install.packages("ggplot2")
install.packages("forecast")#required package for D-M Test


#Libraries
library(ISLR)
library(tidyverse)
library(quantreg)
library(sjPlot)
library(quantmod)
library(dplyr)
```

```r
library(rstatix)
library(ggplot2)
library(forecast)

#Let's first show how do we compute the linear regression model in order to compare it with the QR(objective of the analysis)
#before we want to know our variables in the data set so we will use the function "ls()"
ls("package:ISLR")
ISLR::Wage#using thi instruction we can asses to the differents elements as Wage in this case
lr <- lm(wage ~ jobclass + age, Wage)#this function allows us to calculate the linear regression model in a direct way


#Now let's compute the QR :
#In this case we have to set the level of tau = [0.1, 0.5, 0.9],where :
#tau = 0.1 -> wages of low earners
#tau = 0.5 -> MEDIAN wages
#tau = 0.9 -> wages of Top 10% earners
#For this model we need the function "rq()"
qr10 <- rq(wage ~ jobclass + age, Wage, tau = 0.1)
qr50 <- rq(wage ~ jobclass + age, Wage, tau = 0.5)
qr90 <- rq(wage ~ jobclass + age, Wage, tau = 0.9)


#Graphical Visualization of lm and rq results
#Plotting we try to answer to the following questions :

#1."If age increases by 1 year, how much does it increase on average?
#2.If the worker has a certain type of jobclass (e.g. information), how much does it increase wage?"
plot_models(lr, qr10, qr50, qr90,
            show.values = TRUE ,
            m.labels = c("Linear model", "QR 10%", "QR 50%", "QR 90%"),
            legend.title = "Model Comparison"
            )
#The above plot allows to compare the Models(rq vs lm) and what we see is : different predictors has different magnitude of estimates
#In particular we observe that:
#High estimates are displayed in a better way,while low estimates are underapresented.
#On the plot of course are rapresented :
#Explanatory Var (or Indep.var) -> Age
#Dependent Var -> Wage (what we are trying to observe)

rm.terms = "jobclass[2. Information]"#we use it in order to remove the DUMMY VARIABLE
#So(COEFFICIENTS PLOT from sjPLot) :
plot_models(lr, qr10, qr50, qr90,
             rm.terms = "jobclass [2. Information]",
             show.values = TRUE ,
             m.labels = c("Linear model", "QR 10%", "QR 50%", "QR 90%"),
             legend.title = "Model Comparison"
)
#Giving an interpretation :
#Each coloured point in the graph represents an estimate of the coefficient.
#The horizontal bars are CI(confidence intervals)
#the stars indicated the p-value level :
```

```r
#1. *** -> Highly significant
#2. ** -> Very significant
#3. * -> significant
#A p-value of less than 0.001 indicates that the probability of observing this result by chance is very low.
#Since p < 0.001 is lower than the traditional significance level (e.g.    = 0.05), we can reject the Null hypothesis and say that the result is highly significant.
#What is observed is that for low wages the age counts little ,while it appears to have relevance with advancing age.
#In order to see different quantiles behaviour:
qr_all <- rq(wage ~ jobclass + age, data = Wage,
             tau = seq(from = 0.05, to = 0.95, by = 0.05))

seq(0.5, 0.95, by = 0.05)#range of tau
summary(qr_all) %>% plot()



#Note: Clean the Environment before proceeding ,but you have to recall the libraries !

#1.ENVIRONMENT PREPARING:

#After this brief preamble we can move on our task (CO_2 Dataset):
#set the Directory and then import the data
setwd("/Users/balawalkhan/Desktop/econometrics project ")
co2_data = read.csv("CO2_SA.csv")#we use the "read.csv()" in order to import the data_set

#At this point we have to clean the data -> "DATA CLEANNING PROCESS"
co2_data = na.omit(co2_data)#we remove the NAs values
View(co2_data)#check
growth_rate <- diff(log(co2_data$Value)) * 100
#Where :
#1.the function "log()" calculate the natural logarithm of the CO2 emission values
#2.the function "diff()" calculate the difference between each logarithmic value and the previous one
co2_data <- co2_data[-1, ]#we just remuve the first row
co2_data$GrowthRate <- growth_rate




#2.EXTRACTING PART

#The above code is what we have already done,but now we focus on the PART B of the coursework:
#For this part we need another package ->"quantmod"
install.packages("quantmod")
library(quantmod)
#Let's IMPORT the Independent variable from FRED-MD(Federal Reserve Economic Data-Monthly Data)
#we will use the function "getSymbols()" in order to exctract the data,where:
#INDPRO -> ticker (Industria Production Index),in other word, it has historichal series
getSymbols("INDPRO", src = "FRED")
```

```r
#How many variables we will extract from FRED-MD ?-> 4 Macroeconomics Variables, because in order to apply PCA(Principal Component Analysis) that require >= 3 Ind.var.
getSymbols("UNRATE", src = "FRED")#Unenployment Rate
getSymbols("FEDFUNDS", src = "FRED")#Federal Funds Rate
getSymbols("PAYEMS", src = "FRED")#All Employees: Total Nonfarm Payrolls
#check
head(INDPRO)
tail(INDPRO)
head(UNRATE)
tail(UNRATE)
head(FEDFUNDS)
tail(FEDFUNDS)
head(PAYEMS)
tail(PAYEMS)

#Aggregate all above extracted var. in a single frame by using the function "merge()":
macro_var <- na.omit(merge(INDPRO, UNRATE, FEDFUNDS, PAYEMS))#we have also cleaned the code
#Since we want to interpret in a better way the data ,we transform them in a Percentage values:
macro_var_diff <- diff(log(macro_var)) * 100 #logarithmic difference between two consecutive periods
#clean:
macro_var_diff <- na.omit(macro_var_diff)
colnames(macro_var_diff) <- c("INDPRO", "UNRATE","FEDFUNDS","PAYEMS")



#3:CALCULANTION PART

#After this process the task ask us to apply the PCA.
#To complete the PCA calculation we will use the Direct method given by the function "prcomp()"
PCA = prcomp(macro_var_diff, scale. = TRUE)
print(PCA)
summary(PCA)

n_of_rows <- min(nrow(co2_data), nrow(PCA))#allows us to find the compatible value between the two dataset
#We do this passage because often the two dataset do NOT have the same number of rows.
print(n_of_rows)
#ALTERNATIVE code to get the same result:
length(co2_data$GrowthRate)#result = 624 -> Maximum of lines that you can use without error in the two datasets.
nrow(PCA$x) #result = 850
#At this point we must cut the PCA components to make them match with GrowthRate.

PCA_components <- as.data.frame(PCA$x[1:n_of_rows, 1:2])#PCA$x contains all the main components calculated by PCA.
colnames(PCA_components) <- c("PC1", "PC2")#we take just them instead of PC3 and PC4

#In this way we are trying to construct : y_{t+1} =  0  +  1 *PC1 +  2 *PC2 + ut

```

```r
aligned_growth <- co2_data$GrowthRate[1:n_of_rows]

#Let's define y_{t+1}
y_next = aligned_growth[-1]
#and now x_t
x_t = PCA_components[-n_of_rows, ]

data_frame <- data.frame(y_next, x_t)
data_frame


#Reaching this point we are in a situtaion to conduct lm and qr models :

#1.LINEAR REGRESSION
lm_model <-lm(y_next ~., data = data_frame)
summary(lm_model)

#2.QUANTILE REGRESSION
#we need the following library :
library(quantreg)#recall
taus = c(0.25, 0.5, 0.75)
qr_model <- rq(y_next ~., data = data_frame, tau = taus)
summary(qr_model, se = "nid")



#4.GRAPHICAL RAPRESENTATION

library(sjPlot)#recall
install.packages("patchwork")
library(patchwork)#necessary to aggregate the plots

#We build two different graphs:
p1 <- plot_model(lm_model,
                 show.values = TRUE,
                 digits = 5,
                 value.offset = 0.15,
                 title = "Linear Regression")

p2 <- plot_model(qr_model,
                 show.values = TRUE,
                 digits = 5,
                 value.offset = 0.15,
                 title = "Quantile Regression")

#Aggregate lm and qr
print(p1 + p2)



#5.DIFFERENT LEVELS OF TAU

#What happens if we set different levels of tau?
taus_seq <- seq(0.05, 0.95, by = 0.05)  # 19 quantiles
taus_seq
length(taus_seq)#We try 19 quantiles

#QR on tau = 19 values
qr_seq <- rq(y_next ~ PC1 + PC2, data = data_frame, tau = taus_seq)
```

```r


plot(summary(qr_seq), parm = 1)# Intercept
plot(summary(qr_seq), parm = 2)# PC1
plot(summary(qr_seq), parm = 3)# PC2

# Alternative plot using ggplot:
qr_tidy <- broom::tidy(qr_seq, se.type = "nid")#tidy + standard errors

#Intercept
g0 <- ggplot(qr_tidy[qr_tidy$term == "(Intercept)", ], aes(x = tau, y = estimate))
    +
  geom_line(linetype = "dashed") +
  geom_point() +
  geom_ribbon(aes(ymin = estimate - 1.96 * std.error,
                  ymax = estimate + 1.96 * std.error),
              alpha = 0.2, fill = "grey") +
  geom_hline(yintercept = 0, color = "black") +
  labs(title = "Intercept", x = "Quantile (  )", y = "Estimate")

#PC1
g1 <- ggplot(qr_tidy[qr_tidy$term == "PC1", ], aes(x = tau, y = estimate)) +
  geom_line(linetype = "dashed", color = "orange") +
  geom_point(color = "orange") +
  geom_ribbon(aes(ymin = estimate - 1.96 * std.error,
                  ymax = estimate + 1.96 * std.error),
              alpha = 0.2, fill = "blue") +
  geom_hline(yintercept = 0, color = "black") +
  labs(title = "PC1", x = "Quantile (  )", y = "Estimate")

#PC2
g2 <- ggplot(qr_tidy[qr_tidy$term == "PC2", ], aes(x = tau, y = estimate)) +
  geom_line(linetype = "dashed", color = "darkgreen") +
  geom_point(color = "darkgreen") +
  geom_ribbon(aes(ymin = estimate - 1.96 * std.error,
                  ymax = estimate + 1.96 * std.error),
              alpha = 0.2, fill = "lightgreen") +
  geom_hline(yintercept = 0, color = "black") +
  labs(title = "PC2", x = "Quantile (  )", y = "Estimate")


g0 + g1 + g2

#To be more precise for the analysis let's invert the axes (   on y-axis):
#Inverted PC1
p_pc1 <- qr_tidy %>%
  dplyr::filter(term == "PC1") %>%
  ggplot(aes(x = estimate, y = tau)) +
  geom_point(color = "blue", size = 2) +
  geom_line(linetype = "dashed", color = "blue") +
  geom_ribbon(aes(xmin = estimate - std.error,
                  xmax = estimate + std.error),
              fill = "blue", alpha = 0.2) +
  geom_vline(xintercept = 0, color = "black") +
  labs(y = expression(tau), x = "Estimate", title = "PC1") +
  theme_minimal()

#Inverted PC2
p_pc2 <- qr_tidy %>%
```

```
  dplyr::filter(term == "PC2") %>%
  ggplot(aes(x = estimate, y = tau)) +
  geom_point(color = "darkgreen", size = 2) +
  geom_line(linetype = "dashed", color = "darkgreen") +
  geom_ribbon(aes(xmin = estimate - std.error,
                  xmax = estimate + std.error),
              fill = "green", alpha = 0.2) +
  geom_vline(xintercept = 0, color = "black") +
  labs(y = expression(tau), x = "Estimate", title = "PC2") +
  theme_minimal()

#Inverted axes
p_pc1 + p_pc2

#Now we add the OLS
conf_lm <- confint(lm_model)#CIs calculation
summary_lm <- summary(lm_model)

#(PC1) OLS value extraction
data_lm_PC1 <- data.frame(
  estimate = coef(lm_model)["PC1"],
  LCL = conf_lm["PC1", 1],
  UCL = conf_lm["PC1", 2],
  p = summary_lm$coefficients["PC1", 4]
)

#(PC2) OLS value extraction
data_lm_PC2 <- data.frame(
  estimate = coef(lm_model)["PC2"],
  LCL = conf_lm["PC2", 1],
  UCL = conf_lm["PC2", 2],
  p = summary_lm$coefficients["PC2", 4]
)

#add OLS to PC1
plot_PC1_OLS <- p_pc1 +
  geom_vline(xintercept = data_lm_PC1$estimate, color = "red", size = 1, alpha =
      0.5) +
  annotate("rect", ymin = -Inf, ymax = Inf,
           xmin = data_lm_PC1$LCL, xmax = data_lm_PC1$UCL,
           fill = "red", alpha = 0.1) +
  annotate("text", x = data_lm_PC1$estimate + 0.0003, y = 0.05,
           label = paste0("OLS = ", round(data_lm_PC1$estimate, 5),
                          " (p = ", round(data_lm_PC1$p, 4), ")"),
           color = "red", hjust = 0, size = 4.2)+
  theme(plot.margin = margin(5.5, 60, 5.5, 5.5))

#add OLS to PC2
plot_PC2_OLS <- p_pc2 +
  geom_vline(xintercept = data_lm_PC2$estimate, color = "red", size = 1, alpha =
      0.5) +
  annotate("rect", ymin = -Inf, ymax = Inf,
           xmin = data_lm_PC2$LCL, xmax = data_lm_PC2$UCL,
           fill = "red", alpha = 0.1) +
  annotate("text", x = data_lm_PC2$estimate + 0.0003, y = 0.05,
           label = paste0("OLS = ", round(data_lm_PC2$estimate, 5),
                          " (p = ", round(data_lm_PC2$p, 4), ")"),
           color = "red", hjust = 0, size = 4.2)+
  theme(plot.margin = margin(5.5, 60, 5.5, 5.5))
```

```r

#Complete Graphs
print(plot_PC1_OLS + plot_PC2_OLS)#set width = 1600 and height = 600 when export them as a file (png,jpeg...)



#6.ROLLING WINDOW FORECAST

#In this part we will do the prediction just by using historical data
#Why ? -> Because when we build a regression model we do not have access to the future data ,so, in order to understand how much our model it is good for predictions
#Enviromment preparing :

#Let's recall y_{t+1},PC1 and PC2 ,putting them in a data frame
n <- nrow(data_frame)#n   of row in our previous data frame
n# result = 623
#The task ask us : Use rolling windows 2/3 of the sample for training.
#In order to compute this passage we will use the function "floor()" -> CORE PASSAGE
training_size <- floor(2 * n / 3)#we have n = 623 observations ,we take them and we obtain that the training size will be = 415.33
#floor function it useful because it takes the integer number (e.g. 3.7-> 3 and not 4)
training_size#so,according to "floor" we obtain 415 and not 415.33
testing_size = n - training_size#This calculates how many observations are left for the test set, that is, to evaluate the predictions.
testing_size#208 observation that are left (we will use this resuts for the predictions)

#create an empty vector in which we save the forecasts
ols_empty_vector <-numeric(testing_size)
ols_empty_vector#check

#extract the real values obtained previously [ y_{t+1}]
real_values <- data_frame$y_next[(training_size + 1 ) : n]#(train_size + 1):n generates a sequence from the first out-of-sample observation (after the training set) to the end of the dataset.

#Rolling forecast(OLS) ->  We create a cycle that goes from the first out-of-sample forecast to the last (shift = + one month).
for (i in 1:testing_size) {
  train_end <- training_size + i - 1
  model_ols <- lm(y_next ~ ., data = data_frame[1:train_end, ])
  x_new <- data_frame[train_end + 1, c("PC1", "PC2")]
  x_new <- c(1, as.numeric(x_new))
  ols_empty_vector[i] <- sum(coef(model_ols) * x_new)
}

ols_not_empty = ols_empty_vector
ols_not_empty


#Rolling (QR)
qr_empty_vector <- numeric(testing_size)

for (i in 1:testing_size) {
```

```r
  train_end <- training_size + i - 1
  model_qr <- rq(y_next ~ ., data = data_frame[1:train_end, ], tau = 0.5)
  x_new <- data_frame[train_end + 1, c("PC1", "PC2")]
  x_new <- c(1, as.numeric(x_new))
  qr_empty_vector[i] <- sum(coef(model_qr) * x_new)
}

qr_not_empty = qr_empty_vector
qr_not_empty

#Now we have the necessary staff ,so we can move foreword with the calculation of
    "RMSE-ROOT MEAN SQUARED ERROR"
#Write the function as follows:
RMSE_OLS = sqrt(mean((ols_not_empty - real_values)^2))

RMSE_QR = sqrt(mean((qr_not_empty - real_values)^2))#only for tau = 0.5(MEDIAN)
#check
RMSE_OLS#how wrong the linear model is on average
RMSE_QR#how wrong the quantile model is on average

#Fixing different levels of tau = [0.25, 0.5(MEDIAN), 0.75]
taus = c(0.25, 0.5, 0.75)
qr_model_taus_empty <- list()#we use the function "list()" due to different levels
    of tau

#As before let's run the cycle
for (tau_val in taus) {
  qr_previsons <- numeric(testing_size)

  for (i in 1:testing_size) {
    train_end <- training_size + i - 1
    model_qr <- rq(y_next ~ ., data = data_frame[1:train_end, ], tau = tau_val)
    x_new <- data_frame[train_end + 1, c("PC1", "PC2")]
    x_new <- c(1, as.numeric(x_new))
    qr_previsons[i] <- sum(coef(model_qr) * x_new)
  }

  qr_model_taus_empty[[as.character(tau_val)]] <- qr_previsons
}


qr_model_taus__not_empty = qr_model_taus_empty
qr_model_taus__not_empty#list of values

RMSE_qr_multi_tau <- sapply(qr_model_taus__not_empty, function(pred) {
  sqrt(mean((pred - real_values)^2))
})

RMSE_qr_multi_tau

#Table:
Comparison_table_of_RMSE <- data.frame(
  Model = c("OLS", paste0("QR (tau = ", names(RMSE_qr_multi_tau), ")")),
  RMSE = round(c(RMSE_OLS, RMSE_qr_multi_tau), 5)
)

View(Comparison_table_of_RMSE)

#Barplot:
```

```r
barplot(RMSE_qr_multi_tau,
        col = "skyblue",
        border = "black",
        main = "RMSE (  )",
        xlab = "Quantiles (  )",
        ylab = "RMSE",
        ylim = c(0, max(RMSE_qr_multi_tau) * 1.1))


#OLS:
RMSE_all <- c(RMSE_OLS, RMSE_qr_multi_tau)
names(RMSE_all)[1] <- "OLS"
barplot(RMSE_all,
        col = c("orange", rep("skyblue", length(RMSE_qr_multi_tau))),
        border = "black",
        main = "RMSE : OLS vs QR",
        xlab = "Model",
        ylab = "RMSE",
        ylim = c(0, max(RMSE_all) * 1.1))



#Better Graph Interpretation:
#Run the code selecting it entirely block!
taus_numeric <- as.numeric(names(RMSE_qr_multi_tau))
rmse_vals <- RMSE_qr_multi_tau
y_min <- min(rmse_vals, RMSE_OLS) * 0.95
y_max <- max(rmse_vals, RMSE_OLS) * 1.05
plot(taus_numeric, rmse_vals,
     type = "b", pch = 19, col = "blue",
     xlab = expression(tau), ylab = "RMSE",
     main = "RMSE: QR vs OLS",
     ylim = c(y_min, y_max),
     xaxt = "n")
axis(1, at = taus_numeric, labels = names(RMSE_qr_multi_tau))
abline(h = RMSE_OLS, col = "red", lty = 2, lwd = 3)
text(x = min(taus_numeric),
     y = RMSE_OLS + 0.09,
     labels = paste("OLS =", round(RMSE_OLS, 5)),
     col = "red", font = 2, cex = 0.9, pos = 4)
legend("center", legend = c("QR (for   )", "OLS"),
       col = c("blue", "red"),
       lty = c(1, 2), pch = c(19, NA), lwd = c(2, 3),
       bg = "white", box.lwd = 0.5)



#7.DIEBOLD-MARIANO TEST

#This model it is useful to copare two different model (in our case OLS vs QR).
#In particular it verfies the differences in the errors
install.packages("forecast")#required package
library(forecast)

#To conduct this test we have to build the "LOSS DIFFERNTIAL" :

#forecast errors for each model
Error_OLS <- real_values - ols_not_empty
```

```r
Error_QR <- real_values - qr_not_empty#as before we see first tau = 0.5(MEDIAN)

#Only after the calculation of error we are able to determine the loss
    differential -> we use "dm.test()" (function available directly in R in the
    forecast package)
DM_model <- dm.test(Error_OLS, Error_QR, h = 1 , power = 2)
#1. h -> represents the forecast horizon, that is how many steps forward you are
    trying to predict.
#2. power -> it defines the loss function that we are going to implement (
    QUADRATIC LOSS FUNCTION)
DM_model
#if H0 : E[L(error_ols) - L(error_qr)] == 0, it means that the model are equal

#with taus = [0.25, 0.5 , 0.75]
Error_OLS <- real_values - ols_not_empty#recall
DM_test_p_values <- numeric(length(qr_model_taus__not_empty))
names(DM_test_p_values) <- names(qr_model_taus__not_empty)
#cycle
for (tau_val in names(qr_model_taus__not_empty)) {
  pred_qr <- qr_model_taus__not_empty[[tau_val]]
  Error_QR_tau <- real_values - pred_qr
  #D-M Test
  Dm_result <- dm.test(Error_OLS, Error_QR_tau, h = 1, power = 2)
  DM_test_p_values[tau_val] <- Dm_result$p.value

}

Dm_result


#Table:
DM_test_table <- data.frame(
  Tau = names(DM_test_p_values),
  `p-value (OLS vs QR)` = round(DM_test_p_values, 5),
  `Significativo (  =0.05)` = ifelse(DM_test_p_values < 0.05, "YES", "NO")
)

DM_test_table_extended <- rbind(
  data.frame(
    Tau = "OLS (ref.)",
    `p-value (OLS vs QR)` = NA,
    `Significativo (  =0.05)` = ""
  ),
  DM_test_table
)

print(DM_test_table_extended)
View(DM_test_table_extended)



#BARPLOT
#run as a block
pvals_ext <- c(NA, DM_test_p_values)
model_labels <- c("OLS (ref.)", paste0("QR (   = ", names(DM_test_p_values), ")"))
bar_colors <- c("gray70", ifelse(DM_test_p_values < 0.05, "tomato", "steelblue"))
pval_labels <- c("", paste0("p = ", formatC(DM_test_p_values, format = "f", digits
    = 4)))
bar_centers <- barplot(pvals_ext,
```

```
                        col = bar_colors,
                        border = "black",
                        ylim = c(0, 1.2),
                        names.arg = model_labels,
                        main = "Diebold-Mariano Test: OLS vs Quantile Regression",
                        ylab = "p-value",
                        las = 1,
                        cex.names = 0.9)

# SIGNIFICANCE LEVEL (    = 0.05 )
abline(h = 0.05, col = "darkred", lty = 2, lwd = 2)
pval_y_pos <- ifelse(pvals_ext[-1] < 0.1, pvals_ext[-1] + 0.08, pvals_ext[-1] / 2)
label_colors <- ifelse(bar_colors[-1] == "steelblue", "white", "black")
#p-value top on the bars
text(x = bar_centers[-1],
     y = pval_y_pos,
     labels = pval_labels[-1],
     cex = 0.9,
     font = 2,
     col = label_colors)
legend("topright",
       legend = c("Significant (p < 0.05)", "Not Significant", "OLS Reference"),
       fill = c("tomato", "steelblue", "gray70"),
       border = NA,
       bty = "n",
       cex = 0.9,
       x.intersp = 0.6)
```

PCA DEEPLY ANALYSIS:

```
    #PCA ANALYSIS IN DETAIL

library(corrplot)
library(quantmod)
library(forecast)
library(dplyr)
library(ggplot2)
library(quantreg)
library(sjPlot)
library(broom)
library(patchwork)
library(factoextra)
library(gridExtra)



setwd("/Users/balawalkhan/Desktop/econometrics project ")
co2_data <- read.csv("CO2_SA.csv")
co2_data <- na.omit(co2_data)#remove NAs

#Log CO_2 Calculation
growth_rate = diff(log(co2_data$Value)) * 100
co2_data <- co2_data[-1, ]#It removes the firt line

#Check what we have done
str(co2_data)
head(co2_data)
```

```r


#Let's extract the MACRO VARIABLES from "FRED":
getSymbols(c("INDPRO", "UNRATE", "FEDFUNDS", "PAYEMS", "SP500", "DGS10"), src = "
    FRED")
macro_data <- merge(INDPRO, UNRATE, FEDFUNDS, PAYEMS, SP500, DGS10)
macro_data <- na.omit(macro_data)
macro_data_diff <- diff(log(macro_data)) * 100
macro_data_diff <- na.omit(macro_data_diff)
colnames(macro_data_diff) <- c("INDPRO", "UNRATE", "FEDFUNDS", "PAYEMS", "SP500",
    "DGS10")


#Now the main objective it is to appply the PCA on the extracted variables (our x_
    t):
PCA <- prcomp(macro_data_diff, scale. = TRUE)
summary(PCA)#we see the VARIANCE in this way
# >Var[st.dev.] =  >Informations
# <Var[st.dev.] = <Informations
#Just take in consideration the proportion of variance using the following line of
     code
prop_var <- (PCA$sdev^2) / sum(PCA$sdev^2)
round(prop_var, 4)
#Graphical Visualization(SIMPLE PLOT): we will use just a simple plot
plot(prop_var, type = "b", pch = 19,
     xlab = "Dimension(Principal Component)",
     ylab = "Proportion(%) of Explained Variance",
     main = "Scree Plot")

#since we are interested to go deeply into the analysis we are going also to
    integrate a MATRIX of PCA explanation:
#Objective :
#1.Which variables most affect PC1 and PC2(for example)?
#2.How much "weight" has each variable in each component?
#3.Which component best represents each variable?
#The SOLUTION in given by -> "get_pca_var()"
PCA_vars <- get_pca_var(PCA)
#Coordinate:
print("Loadings (coordinate):")
round(PCA_vars$coord[, 1:2], 3)
#Contribution in Percentage:
print("Contributes (%):")
round(PCA_vars$contrib[, 1:2], 2)
#how well a variable is represented by PC1 and PC2:
print("Cos :")
round(PCA_vars$cos2[, 1:2], 3)
#1.if Cos^2 = 1 -> The variable is well represented.
#2.if Cos^2 = 0 -> The variable is not well represented.


#Create a UNIQUE TABLE of RESULTS:
PCA_Summary_df <- data.frame(
  Variable = rownames(PCA_vars$coord),
  Loading_PC1 = round(PCA_vars$coord[, 1], 3),
  Loading_PC2 = round(PCA_vars$coord[, 2], 3),
  Contrib_PC1 = round(PCA_vars$contrib[, 1], 2),
  Contrib_PC2 = round(PCA_vars$contrib[, 2], 2),
  Cos2_PC1 = round(PCA_vars$cos2[, 1], 3),
  Cos2_PC2 = round(PCA_vars$cos2[, 2], 3)
```

```
)
print(PCA_Summary_df)




#GRAPHICAL VISUALIZATION FOR COS^2:
corrplot(PCA_vars$cos2[, 1:2], is.corr = FALSE)
fviz_cos2(PCA, choice = "var", axes = 1:2,
          title = "Cos2: Quality of Representation on PC1 and PC2")

#BIPLOT -> more useful
fviz_pca_var(PCA,
             col.var = "cos2",
             gradient.cols = c("darkorchid4", "gold", "darkorange"),
             repel = TRUE,
             title = "PCA Variable Plot Colored by Cos ")
#From the PCA biplot it emerges that the variables SP500, PAYEMS, INDPRO and FEDFUNDS are the most influential in explaining the variance observed in the data, with a very high Cos2 confirming the good quality of their representation in the first two main components.
#In contrast, the variable DGS10 is less represented on the plan PC1-PC2, while UNRATE is well represented but shows an opposite trend compared to the others, indicating a counter-cyclical relationship.


#GRAPHICAL VISUALIZATION FOR CONTRIBUTIONS:
a <- fviz_contrib(PCA, choice = "var", axes = 1, top = Inf,
                  title = "Contribution to PC1")
b <- fviz_contrib(PCA, choice = "var", axes = 2, top = Inf,
                  title = "Contribution to PC2")
grid.arrange(a, b, ncol = 2, top = "Contribution of variables to the first two PCs")
#The variables above the threshold (red line) are more important than the average for that component.


#At this point the objective is to try to understand which months/periods are most represented by the main components.
PCA_ind_extract <- get_pca_ind(PCA)
#Check if we have done correctly:
head(PCA_ind_extract$coord)
head(PCA_ind_extract$cos2)
head(PCA_ind_extract$contrib)
tail(PCA_ind_extract$coord)
tail(PCA_ind_extract$cos2)
tail(PCA_ind_extract$contrib)

#GRAPHICAL VISUALIZATION OF PCA_INDIVIDUALS(WITHOUT LABELS):
fviz_pca_ind(PCA,
             col.ind = "cos2",
             gradient.cols = c("darkorchid4", "gold", "darkorange"),
             repel = FALSE,
             label = "none",
             title = "PCA: Individuals Colored by Cos  (without LABELS)")

#GRAPHICAL VISUALIZATION OF PCA_INDIVIDUALS(WITH LABELS):
fviz_pca_ind(PCA,
```

```
133               col.ind = "cos2",
134               gradient.cols = c("darkorchid4", "gold", "darkorange"),
135               repel = TRUE,
136               title = "PCA: Individuals Colored by Cos ")
137
138 #CONTRIBUTION OF INDIVIDUALS:
139 fviz_contrib(PCA, choice = "ind", axes = 1:2, top = 20,
140               title = "Contribution of Individuals to PC1 + PC2")
141
142
143 #k-means Method: it groups periods
144 set.seed(123)#in order to replicate the same result at each simulation of data
145 kmeans_result <- eclust(PCA_ind_extract$coord, "kmeans", k = 4, nstart = 25)
146
147 fviz_cluster(kmeans_result, data = PCA_ind_extract$coord,
148               ellipse.type = "convex", geom = "point",
149               main = "K-Means Clustering of Periods Based on PCA")
```